BURST / `neutt`

A Manual for the JAX/GPU Port of Neutrino Boltzmann Transport in Big Bang Nucleosynthesis

Standard Model, $\Delta N_{\rm eff}$, and flavour mixing

This manual documents the neutrino-transport sector of `neutt`, the differentiable JAX/GPU re-implementation of the Fortran code **BURST** (Grohs, Fuller, Kishimoto, Paris & Vlasenko 2016). It covers the Standard-Model weak-decoupling calculation of $N_{\rm eff}$, the neutrino spectral distortions, finite-temperature quantum-electrodynamics (QED) corrections (through the next-to-leading-order (NLO) $\mathcal{O}(e^3)$ term, on by default), the coupling to the **LINX** BBN abundance code, and the neutrino flavour-mixing refinement (the QKE, or quantum kinetic equation, density-matrix transport), with emphasis on the reasons behind the difficult numerical design choices.
Throughout, the JAX implementation is mapped back to its Fortran BURST ancestor, and every place where the port omits a BURST feature, or takes a different numerical route, is flagged in a [**Port note**] box.

Generated October 25, 2026.

Contents

1 Introduction and scope **4**

1.1 What BURST is . . . 4
1.2 What `neutt` is . . . 4
1.3 Scope of this manual . . . 4
1.4 How the Fortran mapping is presented . . . 5
1.5 Notation and conventions . . . 5
1.6 JAX concepts used in this manual . . . 6
1.7 The canonical Standard-Model result . . . 7
1.8 Roadmap: how to read this manual . . . 7

2 Physics background **8**

2.1 Weak decoupling and neutrino spectral distortions . . . 8
2.2 The Boltzmann transport equation and the collision integral . . . 8
2.3 The two-temperature description and $N_{\rm eff}$ . . . 9
2.4 Finite-temperature QED corrections . . . 10
2.5 BBN observables and the $n \leftrightarrow p$ rates . . . 10

3 Numerical methods background **10**

3.1 The kernel-tensor reformulation . . . 10
3.2 Quadrature: what rule is used where . . . 11
3.3 Off-grid interpolation in log space . . . 12
3.4 The net/frs decomposition and the precision gate . . . 12
3.5 ODE integration: diffrax, Tsit5, Kvaerno5, PID control . . . 13
3.6 The exponential-midpoint hybrid solver . . . 14
3.7 float64, NaN guards, and differentiability . . . 15
3.8 The GPU-throughput strategy . . . 15

4 Installation, environment, and a first run **16**

4.1 Where the code lives . . . 16
4.2 Environment (Perlmutter) . . . 16
4.3 Quickstart 1: standalone $N_{\rm eff}$ . . . 17
4.4 Quickstart 2: a repeated transport solve . . . 17
4.5 Quickstart 3: coupled BBN (abundances + $N_{\rm eff}$) . . . 17
4.6 Building the Fortran oracle (optional, for cross-checks) . . . 18

5 Core module reference **18**

5.1 `constants`: physical constants . . . 18
5.2 `grid`: the comoving-energy grid, quadrature, occupations, and moments . . . 19
5.2.1 The ε-grid . . . 19

5.2.2 Equilibrium occupations . . . 20
5.2.3 Gauss nodes and rescaling . . . 20
5.2.4 Momentum-space moments . . . 21
5.3 `interp`: log-space occupation interpolation . . . 21
5.4 `thermo`: plasma thermodynamics, finite-T QED, and the background . . . 22
5.4.1 The free-gas equation of state . . . 22
5.4.2 Finite-T QED mass renormalization . . . 23
5.4.3 The finite-T QED equation of state . . . 24
5.4.4 NLO QED (the `qed_nlo` default) . . . 24
5.4.5 The decoupling "clock" . . . 25

6 The Standard-Model transport kernels 26
6.1 File inventory and Fortran mapping . . . 26
6.2 `assemble`: precision gate and RHS . . . 26
6.3 The grid-only kernels: `nunu` and `nunubar` . . . 28
6.4 The lepton kernels: `nuer1`, `nuer2`, `nuepma` . . . 29
6.5 The scheduled (`_sched`) variants, and what `use_sched` is . . . 30
6.6 Conservation sum rules . . . 31

7 The coupled decoupling solver 32
7.1 The system of equations . . . 32
7.2 `make_decoupling_solver` . . . 33
7.3 The RHS builders: `make_rhs` and `make_split_rates` . . . 35
7.4 The solver menu . . . 35
7.5 Standalone demos . . . 35
7.6 Outputs and I/O: getting at the spectra . . . 37

8 The exponential-midpoint hybrid solver 38
8.1 `make_hybrid_solver` . . . 38
8.2 The returned `solve` . . . 39
8.3 Handoff qualification and silent fallback . . . 39
8.4 Operating envelope, which solver, when . . . 39
8.5 Validation headline . . . 40

9 Coupling to LINX for abundances 40
9.1 `make_coupled_bbn` . . . 41
9.2 The `run` closure . . . 41
9.3 Full vs. semi coupling . . . 42
9.4 The $\Delta N_{\rm eff}$ convention . . . 42
9.5 The transport↔LINX seam (`transport_adapter`) . . . 43
9.6 Coupled-BBN demos . . . 44

10 Parameters, tolerances, and knobs **44**
10.1 Traced vs. build-time . . . 44
10.2 Tolerances and step control (change with care) . . . 47
10.3 Starting transport at a lower temperature . . . 47

11 Validation and testing **48**
11.1 The validation philosophy . . . 48
11.2 Canonical numbers . . . 48
11.3 Running the test suite . . . 48

12 Caveats and safeguards **49**
12.1 Features in BURST not present in the port . . . 50
12.2 Troubleshooting quick reference . . . 50

13 The QKE flavour-mixing sector **51**
13.1 The density-matrix state . . . 51
13.2 The Hamiltonian and what is on by default . . . 52
13.3 The operator split: `hybrid` by default . . . 53
13.4 The collision acting on the coherences . . . 56
13.5 The off-grid tail closure . . . 57
13.6 Headline results . . . 58
13.7 The public API and status . . . 59

14 Extending the code **60**
14.1 Increasing the precision of N_{eff} . . . 60
14.2 Gradients and gradient-based inference . . . 61
14.3 Scanning the parameters of an existing model . . . 61
14.4 Adding a channel, a background term, or an option . . . 62
14.5 Fixed structure . . . 63

A BURST ↔ `neutt` mapping index **63**
A.1 Auxiliaries and grid . . . 63
A.2 Background and finite-T QED . . . 64
A.3 Transport kernels and assembly . . . 65
A.4 Solver, driver, and coupling . . . 66
A.5 Flavour mixing (QKE) . . . 67
A.6 Summary of port-only additions . . . 67

B Glossary of symbols **68**

C References **72**

1 Introduction and scope

1.1 What BURST is

BURST is a Fortran Big Bang Nucleosynthesis (BBN) code written by Evan Grohs, George Fuller, and collaborators. Its defining feature, relative to the standard BBN codes, is that it does not approximate the neutrino seas as a single thermal bath with one temperature T_ν. Instead it solves the **full momentum-resolved Boltzmann transport equation** for each neutrino species, with the complete $2 \to 2$ weak collision integrals, simultaneously with the nuclear reaction network and the plasma thermodynamics. This yields the out-of-equilibrium neutrino spectral distortions that develop during weak decoupling, and a correspondingly self-consistent effective number of neutrino species $N_{\rm eff}$. The methodology is described in

> E. Grohs, G. M. Fuller, C. T. Kishimoto, M. W. Paris, A. Vlasenko, "Neutrino energy transport in weak decoupling and big bang nucleosynthesis," Phys. Rev. D **93**, 083522 (2016), arXiv:1512.02205.

We refer to this throughout as **the BURST paper**. The companion review E. Grohs & G. M. Fuller, "Big Bang Nucleosynthesis" (arXiv:2301.12299) is the best single reference for the surrounding cosmology, and we assume the reader is comfortable with its level.

1.2 What `neutt` is

`neutt` is a from-scratch re-implementation of the BURST neutrino sector in JAX, written to run on GPU and to be differentiable. (The package's former names are `jaxburst` and `newt`.) The motivation is twofold:

1. **Speed.** The Fortran BURST takes $\mathcal{O}(600\text{–}1000)$ CPU-hours per cosmology, almost all of it in the transport collision integrals. Recasting those integrals as dense tensor contractions (Section 3.1) lets a single GPU evaluate them, and lets many cosmologies be batched with `vmap` (JAX's automatic vectorization, Section 1.6).
2. **Differentiability and embeddability.** A transport solver that JAX can compile (`jit`) and differentiate (`grad`), with a clean `transport(background)`→`spectra` seam, can be dropped into a gradient-based Markov-chain Monte Carlo (MCMC) sampler, or used to replace the thermal-T_ν approximation inside the differentiable BBN code **LINX** (Giovanetti et al. 2024, arXiv:2408.14538).

The guiding principle of the port is **accuracy parity first**: the collision operator is a near-cancellation of large gain and loss terms, so the port runs in float64, reproduces BURST's conservation "sum rules," and is validated bit-for-bit against an instrumented build of the original Fortran wherever possible. Speed is pursued only where it costs no accuracy.

1.3 Scope of this manual

The manual (Sections 2–14) covers the Standard-Model neutrino transport and everything needed to drive it: the background plasma thermodynamics, finite-temperature QED corrections, the weak collision kernels, the coupled decoupling solver that produces $N_{\rm eff}$ and the per-species spectra, the exponential-midpoint hybrid solver, the coupling into LINX for light-element abundances, and the flavour-mixing refinement (the quantum-kinetic-equation (QKE) density-matrix transport, which is Standard-Model neutrino-oscillation physics), together with the simplest beyond-Standard-Model handle, the inert dark-radiation parameter $\Delta N_{\rm eff}$ (extra relativistic energy density that redshifts like radiation and does not interact).

The interacting dark-sector extensions (dark boson decays, the thermal majoron, and the $\chi \leftrightarrow \nu$ contact sector) live on the repository's `dark` branch, which carries its own manual.

Genuinely out of scope, because the JAX port does not implement them, are BURST's internal nuclear reaction network and its recombination / sound-horizon (`sdratio`) machinery; abundances instead come from LINX (see Appendix A for the full ported-vs-not table).

1.4 How the Fortran mapping is presented

Two boxed annotations recur throughout:

> **[Fortran ↔ JAX]** identifies the Fortran BURST routine, file, or variable that a piece of `neutt` corresponds to, so the reader can cross-read the two trees.

> **[Port note]** flags a place where the port omits a BURST feature, or deliberately takes a different numerical approach than the Fortran (and why).

A note on provenance: `neutt` is validated against a single, read-only Standard-Model Fortran reference, a snapshot of the BURST code (modules `mainvar`, `transvar`, ...) that ships in `fortran_reference/`, which compiles, runs, and serves as the oracle the port is checked against. Wherever the port deliberately departs from it numerically — the corrected wide-regime lepton-kernel quadrature (Section 6.4) and the default-on NLO $\mathcal{O}(e^3)$ QED term (Section 5.4.4) — a **[Port note]** box says so where the code appears. Port notes also mark choices where the port deliberately matches the reference bit-faithfully, such as the high-occupation interpolation clamp (Section 3.3) and the combined precision gate with its noise floor (Section 3.4).

1.5 Notation and conventions

These conventions follow the BURST paper. The most important point, and the most common cross-group misreading, is the **two-temperature split**:

- $T_{\rm cm}$: the **comoving temperature parameter**. It is not a physical neutrino temperature; it is a redshift clock, $T_{\rm cm}(a) = T_{\rm cm}^{\rm init}\, a_{\rm init}/a$ (a the scale factor; $a_{\rm init}$ and $T_{\rm cm}^{\rm init}$ the values at the start of the solve), i.e. an energy scale that redshifts exactly as $1/a$. Comoving momenta are frozen in units of $T_{\rm cm}$.
- $T_{\rm pl} \equiv T_\gamma$: the physical **plasma** (photon $+\, e^\pm +$ baryon) temperature. After $e^\pm$ annihilation transfers entropy to the photons, $T_{\rm pl}/T_{\rm cm} \to (11/4)^{1/3} \simeq 1.401$.
- T_ν: a physical neutrino temperature; only meaningful in the thermal-fluid approximation (LINX), where the spectrum is assumed Fermi–Dirac. BURST/`neutt` resolve $f(\varepsilon)$ and never assume it.
- $\varepsilon \equiv E_\nu/T_{\rm cm}$: the dimensionless **comoving energy** variable that labels the momentum grid. Defined with the energy E; for the relativistic neutrinos in the transport window it equals $p/T_{\rm cm}$. The grid is linear in ε on $[0, \varepsilon_{\rm max}]$.
- $f_{\nu_i}(\varepsilon)$: the neutrino occupation number (occupation probability) for species i. Equilibrium value $f^{\rm eq}(\varepsilon) = 1/(e^\varepsilon + 1)$ (Fermi–Dirac with chemical potential $\mu = 0$, at $T_\nu = T_{\rm cm}$).
- $\delta f \equiv \big[f(\varepsilon) - f^{\rm eq}(\varepsilon)\big]/f^{\rm eq}(\varepsilon)$: the fractional **spectral distortion**. The absolute distortion $f - f^{\rm eq} = f^{\rm eq}\,\delta f$ carries energy with an ε^3 weight: $\Delta(\mathrm{d}\rho/\mathrm{d}\varepsilon)/\rho = (120/7\pi^4)\,\varepsilon^3[f - f^{\rm eq}]$, with ρ the equilibrium energy density of that single species.

- $N_{\rm eff}$: the effective number of neutrino species, defined through the radiation energy density at photon decoupling,

$$\rho_{\rm rad} = \left[2 + \tfrac{7}{4}\left(\tfrac{4}{11}\right)^{4/3} N_{\rm eff}\right] \frac{\pi^2}{30}\, T_\gamma^4. \tag{1}$$

  Equivalently $N_{\rm eff} = \frac{8}{7}\left(\frac{11}{4}\right)^{4/3} \rho_\nu/\rho_\gamma$ evaluated late, after $e^\pm$ annihilation completes (Section 2.3).
- $Y_{\rm P}$: the ^{4}He **mass** fraction, $Y_{\rm P} \simeq 2(n/p)/(1 + n/p)$ with n/p the neutron-to-proton ratio when nucleosynthesis begins; D/H is the deuterium abundance ratio. These are the BBN observables.
- $G_{\rm F}$, $\sin^2\theta_W$, α, m_e: Fermi constant, weak mixing angle, fine-structure constant ($\alpha = e^2/4\pi$ with e the elementary charge, so $\mathcal{O}(e^3)$ counts powers of e), electron mass.

The integration variable for the decoupling ordinary differential equation (ODE) is $\ln a$ (the number of e-folds since the start); $T_{\rm cm} = T_{\rm cm}^{\rm init} e^{-\ln a}$ analytically. All energies are in MeV and energy densities in MeV^4 unless noted.

1.6 JAX concepts used in this manual

The port is written in JAX, and a handful of ideas from JAX and its compiler, XLA (Accelerated Linear Algebra), recur throughout and explain many of the design choices and timings. This is a one-page glossary for a reader fluent in physics but new to JAX; nothing here is BBN-specific.

Trace and compile.
When you first call a function wrapped in `jax.jit` (just-in-time compilation), JAX runs your Python once symbolically ("tracing") to record the array operations, hands that graph to the XLA compiler, and caches the compiled executable. Every later call reuses it. So the first call is slow (it compiles) and later calls are fast (they just run the cached graph).

Cold / maiden vs. warm.
We call the first, compile-paying call cold (or maiden); a later, graph-reusing call warm. The cold cost is one-time; the warm cost is what matters for a scan or an MCMC. This is why every solver follows a build-once, reuse pattern: build the solver, then call it many times (Section 10).

Traced vs. build-time (static).
A parameter is traced if it enters the graph as data: you can change it on every warm call with no recompile (MCMC-safe). It is build-time / static if changing it changes the shape of the computation, forcing a recompile. Which physical inputs are which is the subject of Section 10.

PyTree.
A nested container (dict / tuple / list) of arrays that JAX treats as one structured object. The evolved state $\{T_{\rm pl},\ f\}$ is a PyTree; so are the returned output dicts.

Immutable arrays.
JAX arrays cannot be mutated in place; an "edit" returns a new array. The demos use `x.at[i].add(v)` / `.set(v)` for this (it reads "x with index i incremented by v").

`vmap` / `pmap`.
Automatically vectorize a function over a new leading axis (a batch of cosmologies, say), and `pmap` spreads that batch across devices. This is the "across-calls" GPU win.

Host vs. device.
Host = CPU (where Python and some setup run); device = GPU (where the compiled kernels run). The coupled-BBN driver deliberately splits work across the two (Section 9).

`lax.scan`.
A compiled fixed-length loop (a `for` loop XLA can see through). The hybrid head is a `lax.scan` of a fixed number of steps.

`einsum`.
: Einstein-summation contraction of several arrays in one fused operation, the form the collision integral is recast into (Section 3.1).

Fusion and constant-folding.
: XLA fuses adjacent array operations into one GPU kernel; the grouping it chooses can change the floating-point reduction order, so two builds of the "same" code can round slightly differently (this is behind the executable-to-executable $N_{\rm eff}$ spread of Section 8). Constant-folding is XLA evaluating an input-independent sub-computation at compile time; usually harmless, but it can mishandle a near-cancellation (Section 3.4).

Adjoint.
: The strategy a differentiable ODE solver uses to back-propagate gradients through the integration. It affects gradient cost and compile time but not the forward solution (Section 14).

`diffrax` **PID controller.**
: `diffrax` is the JAX ODE-solver library used here; its adaptive step size is set by a PID (proportional–integral–derivative) controller on the Runge–Kutta embedded error estimate (Section 3.5).

A reader who wants to skip this can: the terms are re-explained briefly where they first do real work.

1.7 The canonical Standard-Model result

The benchmark this whole code exists to compute is the Standard-Model (SM) $N_{\rm eff}$. The **default** includes the NLO $\mathcal{O}(e^3)$ finite-temperature QED correction (Section 5.4.4), and `neutt`'s full Standard-Model decoupling produces

$$\boxed{N_{\rm eff}^{\rm SM} = 3.04332} \qquad \text{(NLO QED, the default)},$$

which reconciles with the NLO precision calculation of Cielo, Escudero, Mangano & Pisanti (arXiv:2306.05460, "$N_{\rm eff}$ in the Standard Model at NLO is 3.043"). Turning the NLO term off (`qed_nlo=False`) recovers the leading-order (LO) value

$$N_{\rm eff} = 3.0442761,$$

and evaluating that at Fortran BURST's own $\sin^2\theta_W = 0.23$ (`SIN2THW_BURST`; production uses the more precise 0.23122, which raises $N_{\rm eff}$ by $+5.1\times10^{-5}$) gives 3.0442252, the bit-faithful BURST replica (it matches a collaborator's BURST oracle to 5.5×10^{-7}, and reproduces the precision results of Akita & Yamaguchi, arXiv:2005.07047; Froustey, Pitrou & Volpe, arXiv:2008.01074; and Bennett et al., arXiv:2012.02726, all $\simeq 3.0440$). The NLO term shifts $N_{\rm eff}$ by -9.5×10^{-4}. Turning the finite-T QED correction to the plasma equation of state (EoS) off entirely (`use_renorm=False`) gives the "decoupling-only" $N_{\rm eff} = 3.034029$. These numbers recur as pins throughout the manual; keep them in mind as the correctness anchors.

1.8 Roadmap: how to read this manual

The manual is layered so a reader can stop at the depth they need.

- **Just want $N_{\rm eff}$ or a spectrum.** Skim the conventions (Section 1.5) and the canonical result (Section 1.7), then run Quickstart 1 (Section 4).
- **Want light-element abundances.** Go to the LINX coupling (Section 9) and Quickstart 3; the SM transport runs underneath.
- **Driving an MCMC or a parameter scan.** Read which arguments are traced vs. build-time (Section 10) and the "build once, reuse the compiled closure" pattern (Quickstart 2; Sections 8, 9).

- **Cross-reading the Fortran.** The inline [**Fortran ↔ JAX**] and [**Port note**] boxes map each piece to its BURST ancestor; the full enumeration is the mapping appendix (Appendix A).
- **Want the numerical reasoning first** (why the kernel tensors, the precision gate, the hybrid solver): read the numerical-methods background (Section 3) before the API.
- **Hit a problem.** See the troubleshooting quick reference (Section 12.2) and the caveats (Section 12).

The body proceeds physics (Section 2) → numerics (Section 3) → install and quickstarts (Section 4) → a module-by-module reference (Sections 5–6) → the coupled and hybrid solvers (Sections 7, 8) → the LINX coupling (Section 9) → parameters (Section 10), validation (Section 11), and caveats (Section 12); then the QKE flavour-mixing refinement (Section 13) and directions for extending the code (Section 14); closing with the mapping appendix, glossary, and references.

2 Physics background

This section sets out the physics the code implements, at the level of the BURST paper. Readers fluent in weak decoupling can skim to the conventions in Section 1.5 and the collision integral in Section 2.2.

2.1 Weak decoupling and neutrino spectral distortions

In the early universe ($T \gtrsim$ few MeV) neutrinos are kept in equilibrium with the $e^{\pm}/\gamma$ plasma by charged- and neutral-current weak interactions, which have rate $\Gamma \sim G_{\rm F}^2 T^5$. As the universe expands, $\Gamma/H \propto T^3$ (H the Hubble expansion rate) falls below unity near $T \simeq 2$–3 MeV and the neutrinos decouple. In the idealized "instantaneous decoupling" picture, decoupling happens before $e^{\pm}$ annihilation, the neutrinos retain a perfect Fermi–Dirac spectrum, and the subsequent $e^+e^- \to \gamma\gamma$ heats only the photons, giving $T_\gamma/T_\nu = (11/4)^{1/3}$ and exactly $N_{\rm eff} = 3$.

Decoupling is not instantaneous. At the few-MeV epoch where the weak rates are shutting off, $e^{\pm}$ annihilation has already begun, so a small amount of the entropy released by annihilating pairs leaks into the still-marginally-coupled neutrinos before the channel closes. Because the weak cross-section grows with energy, this leakage preferentially heats the high-energy tail of the neutrino spectrum: the distribution develops a non-thermal **spectral distortion** $\delta f(\varepsilon)$, positive and growing with ε. The extra energy carried by the distorted seas raises the radiation density slightly above the instantaneous-decoupling value, i.e. $N_{\rm eff} > 3$. Capturing $\delta f(\varepsilon)$ (not just an averaged T_ν) is exactly what BURST and `neutt` are for.

[**Fortran ↔ JAX**] The distortion is what the Fortran writes to its per-species output units `nu_72.dat` … `nu_75.dat`. In `neutt` the same information is the trajectory array `f_traj` returned in memory by the `solve` of `make_decoupling_solver` (Section 7.2); the port writes no files (Section 7.6).

2.2 The Boltzmann transport equation and the collision integral

Each neutrino occupation evolves under

$$\left[\frac{\partial}{\partial t} - H(a)\, p\, \frac{\partial}{\partial p}\right] f_{\nu_i}(p,t) = C_{\nu_i}\left[\{f_j\}\right], \tag{2}$$

where H is the Hubble rate and C_{ν_i} is the collision integral summing all $2\to2$ weak processes the species participates in. In the comoving variable $\varepsilon = p/T_{\rm cm}$ the free-streaming (redshift) term on the left is absorbed: a decoupled neutrino sits at fixed ε, so in ε-space Eq. (2) becomes simply $\mathrm{d}f/\mathrm{d}t = C$ (and the code integrates in $\ln a$, dividing C by H).

For a representative process r, $1+2\to3+4$, with four-momenta $P_k = (E_k, \mathbf{p}_k)$ of the legs $k = 1..4$, the collision integral is

$$C^{(r)}_{\nu_1} = \frac{1}{2E_1}\int \frac{\mathrm{d}^3p_2}{(2\pi)^3 2E_2}\frac{\mathrm{d}^3p_3}{(2\pi)^3 2E_3}\frac{\mathrm{d}^3p_4}{(2\pi)^3 2E_4}(2\pi)^4\delta^4(P_1+P_2-P_3-P_4)\, S_r\, \langle|M_r|^2\rangle\, F_r, \tag{3}$$

with the Pauli-blocked **gain** − **loss** phase-space factor

$$F_r = \underbrace{(1-f_1)(1-f_2)f_3f_4}_{\text{gain}} - \underbrace{f_1f_2(1-f_3)(1-f_4)}_{\text{loss}}. \tag{4}$$

$\langle|M_r|^2\rangle$ is the spin-summed squared matrix element, S_r a symmetry factor. At equilibrium ($f_i = f^{\rm eq}$) detailed balance makes $F_r = 0$ identically, so $C = 0$: the equilibrium spectrum is a fixed point.

The processes are:

- $\nu\nu \to \nu\nu$ and $\bar\nu\bar\nu \to \bar\nu\bar\nu$ elastic scattering (neutral current among neutrinos);
- $\nu\bar\nu \to \nu\bar\nu$ scattering and the flavour-changing annihilation $\nu_i\bar\nu_i \to \nu_j\bar\nu_j$;
- $\nu + e^\pm \to \nu + e^\pm$ scattering (two angular reductions, called R1 and R2 in BURST);
- $\nu\bar\nu \to e^-e^+$ annihilation (and its inverse).

The first two families exchange energy only among the neutrino seas; the ν–$e^\pm$ families exchange energy with the plasma and therefore drive the distortion. This split is reflected in the code's `leg4` bookkeeping, BURST's $e^\pm$ partner slot, not a momentum leg (Section 6.2).

The angular (δ^4) integrals in Eq. (3) can be done analytically, reducing each process to a two-dimensional integral over outgoing energies for every output bin ε_1. Energy conservation fixes the fourth leg ($E_4 = E_1+E_2-E_3$, i.e. $p_4 = p_1+p_2-p_3$ for the relativistic legs), which generally lands off the ε-grid and must be interpolated. How the port turns this structure into a GPU tensor contraction is the subject of Section 3.1.

2.3 The two-temperature description and $N_{\rm eff}$

After $e^\pm$ annihilation completes, the neutrino energy density is frozen in comoving units and the photon temperature has been boosted. $N_{\rm eff}$ packages the total neutrino (plus any extra-radiation) energy density relative to photons through Eq. (1), equivalently

$$N_{\rm eff} = \frac{8}{7}\Big(\frac{11}{4}\Big)^{4/3}\frac{\rho_\nu}{\rho_\gamma}, \qquad \rho_\nu = \frac{T_{\rm cm}^4}{2\pi^2}\sum_{i\in\{\text{flavour}\},\,\text{cp}}\int_0^{\varepsilon_{\max}}\varepsilon^3 f_i(\varepsilon)\,\mathrm{d}\varepsilon, \qquad \rho_\gamma = \frac{\pi^2}{15}T_{\rm pl}^4. \tag{5}$$

The sum runs over the three flavours and both charge-parity (particle / antiparticle) states; each contributes $g = 1$. Instantaneous decoupling gives exactly 3; the spectral distortions and finite-T QED corrections push it to ≈ 3.044 (LO), or 3.0433 with the NLO QED term that is on by default (Section 5.4.4).

[Fortran ↔ JAX] Eq. (5) is `decoupling.neff_from_spectrum(grid, occ, T_cm, T_pl)` (the second argument `occ` is the spectrum $f[3, 2, N]$: 3 flavours × 2 cp states × N grid bins; and `rho_nu_from_spectrum(grid, occ, T_cm)`); the prefactor $1/2\pi^2$ matches BURST's `drho` moment normalization. In Fortran the analogous number is written to unit 82 (`params_82.dat`).

Initial vs. end-point evaluation. LINX (and Escudero-style fluid codes) report a $\Delta N_{\rm eff}$ defined at the initial time, $\Delta N_{\rm eff}^{\rm init} = \frac{8}{7}(\frac{11}{4})^{4/3}\rho_{\rm ext}^{\rm init}/\rho_\gamma^{\rm init}$ (with $\rho_{\rm ext}$ the extra-radiation energy density), distinct from the late-time $N_{\rm eff}$; the coupling layer (Section 9) is careful about which is which, and the public $\Delta N_{\rm eff}$ knob uses LINX's initial-time convention.

2.4 Finite-temperature QED corrections

At $\mathcal{O}(\alpha)$ the surrounding $e^\pm/\gamma$ plasma shifts the electron and photon dispersion relations (thermal masses δm_e^2, δm_γ^2) and therefore the plasma equation of state $\rho_{\rm pl}$, $P_{\rm pl}$, $\mathrm{d}\rho_{\rm pl}/\mathrm{d}T$. Because $N_{\rm eff} \propto T_{\rm pl}^{-4}$ at fixed ρ_ν, this ∼per-cent correction to the plasma energy budget moves $N_{\rm eff}$ at the third decimal; it is the difference between the "decoupling-only" 3.034 and the LO-QED 3.044 (a further NLO term, on by default, trims it to 3.0433; Section 5.4.4). BURST computes the LO correction as an additive $\delta\rho/\delta P$ built from the thermal self-energies δm_e^2, δm_γ^2 (which enter BURST's seven Gauss–Laguerre plasma-moment integrals `glag1` … `glag7`, the $e^\pm/\gamma$ energy density, pressure, $\mathrm{d}\rho/\mathrm{d}T$, and their finite-$T$ QED corrections), not as a mass shift inside the free-gas integrand; the port follows this faithfully (Section 5.4.3).

2.5 BBN observables and the $n \leftrightarrow p$ rates

The light-element abundances ($Y_{\rm P}$, D/H, ^{3}He/H, ^{7}Li/H) are set by the neutron-to-proton ratio at freeze-out and the subsequent nuclear network. The $n \leftrightarrow p$ interconversion rates ($n + \nu_e \leftrightarrow p + e^-$, $n + e^+ \leftrightarrow p + \bar{\nu}_e$, and neutron decay) depend on the electron-neutrino occupation at physical energy, precisely the distorted $f_{\nu_e}(\varepsilon)$ that transport produces. A distortion in f_{ν_e} thus feeds directly into $Y_{\rm P}$ and D/H. In the `neutt` + LINX coupling, this is the role of `f_e_occupation` (Section 9): it evaluates the transport-distorted ν_e spectrum at the energies LINX's weak-rate integrals request.

[Port note] Fortran BURST contains its own nuclear reaction network and computes $Y_{\rm P}$, D/H internally. The JAX port does not re-implement the network; it ports only the neutrino transport and hands the distorted spectrum to LINX's abundance model. See Appendix A.

3 Numerical methods background

The port reproduces BURST's physics but reorganizes its numerics for the GPU and for differentiability. This section explains the non-trivial numerical ideas at a level the reader will want before reading the API.

3.1 The kernel-tensor reformulation

The collision integral (3), after the analytic angular reduction, has the schematic form (illustrated for ν–ν)

$$\mathrm{net}[i] = \mathrm{prefac} \sum_j \mathrm{kinem}(i,j) \sum_{p_3} \mathrm{glw}(i,j,p_3)\, \mathrm{w}_3(i,j,p_3)\, F\big(f_1, f_2, f_3, f_4\big), \tag{6}$$

where $f_1 = f(\varepsilon_i)$, $f_2 = f(\varepsilon_j)$ are on-grid (so $p_1 \equiv \varepsilon_i$, $p_2 \equiv \varepsilon_j$), $f_3 = f(p_3)$ and $f_4 = f(p_1{+}p_2{-}p_3)$ are off-grid, and F is the gain − loss tensor of Eq. (4). The schematic symbols (prefac the process prefactor, kinem the kinematic weight, glw the Gauss–Legendre weights, w_3 the leg-3

quadrature measure) stand for the host-static factors enumerated below. The key observation (anticipated by the Fortran authors in `trans_init.f90`) is that every factor except F depends only on the ε-grid and the quadrature nodes, never on the occupation numbers. The prefactor, kinematic weights, Gauss weights, segment boundaries, and node positions are all **host-static**, computable once.

Moreover the off-grid reads are done by interpolating $\log f$ with a fixed-order polynomial, which is linear in the sampled $\log f$ values: $\log f(p) = \sum_a L[p, a] \log f_a$ over a 6-point window, with static Lagrange weights L. So the only nonlinear step is the final pointwise exp. The runtime therefore collapses to:

1. gather $\log f$ through two static sparse interpolation tensors to get f_3, f_4 (a `jnp.take` plus weighted sum, then exp);
2. form F pointwise (multilinear in f_1, f_2, f_3, f_4);
3. contract F against one static weight tensor W with an `einsum`.

The depth-3 Fortran loops over (bin × partner-bin × node) become a single batched contraction over the (flavour × cp × bin) axis, exactly the axis the Fortran's MPI (Message Passing Interface) build already splits across ranks. This is the central GPU idea and the reason the port can be both faithful and fast.

Static objects (grid-only kernels). With N bins, $n_{\mathrm{GL}} = 5$ Gauss nodes, $M = 6$ interpolation points, and K quadrature slots per bin-pair, the precomputed tensors of Eq. (6) are the weight tensor W of shape (N, N, K) and the interpolation index/weight tensors $(\mathrm{idx}_3, \mathrm{lw}_3, \mathrm{idx}_4, \mathrm{lw}_4)$ of shape (N, N, K, M). Sparse 6-point windows keep each interpolation tensor at ∼7 MB; a dense (N, N, K, N) matrix would be ∼120 MB.

Kinematic case-branches → union grid + masks. The energy integral runs over segments split at kinematic boundaries, and the ν–$e^{\pm}$ kernels further split the bin range into "regimes" with different segment structure. Rather than data-dependent control flow (illegal under `vmap`/`jit`), every segment/regime is materialized into a fixed slot/tile dimension and the inactive ones are masked, baked to $W = 0$ for the grid-only kernels, and selected by runtime regime masks for the lepton kernels.

[**Port note**] This "union grid + mask" is the port's structural departure from the Fortran, which uses ordinary `if`-branches on the kinematic regime. The two agree in value; the JAX version simply evaluates a fixed superset and zeroes what the Fortran would have skipped. A faster variant (Section 6.5) gathers each bin's active tiles instead of masking.

3.2 Quadrature: what rule is used where

Three quadrature families appear, exactly as in the BURST paper:

- **Gauss–Legendre** for inner energy integrals over finite intervals;
- **Gauss–Laguerre** for semi-infinite $[a, \infty)$ tails (the exponentially-decaying Boltzmann factors). The port uses the plain variant ($\alpha = 0$ in the generalized weight $x^{\alpha} e^{-x}$; this α is not the fine-structure constant) with e^x folded into the weights so the rule integrates a bare $\int_0^{\infty} f \, \mathrm{d}x$ (`grid.gauss_laguerre_plain`); the genuinely generalized ($\alpha \neq 0$) routine exists but is unused;
- **composite Boole's rule** (5-point Newton–Cotes) for the outer integral over the neutrino energy grid (this is why N must be $4n{+}1$ for integer n).

The classical Gauss rules are tiny and fixed, so the port precomputes their nodes and weights on the host with NumPy/SciPy and carries them as constants.

[**Port note**] This is the one deliberate algorithmic substitution in the auxiliaries: BURST generates Gauss nodes at runtime via the Golub–Welsch eigensolve (`gauss.f90:imtqlx`). The port does not port the eigensolve; it precomputes nodes with `numpy.polynomial.legendre.leggauss` and `scipy.special.roots_genlaguerre`, and ports only the affine interval rescale `scqf`, which the kernels apply inline to the canonical $[-1, 1]$ nodes (`grid.rescale_legendre` is the standalone form), because the collision-integral limits move with the running momentum and must stay jittable.

3.3 Off-grid interpolation in log space

The off-grid legs f_3, f_4 are read by BURST's `interp_occfrac`: a 5th-order (6-point) polynomial interpolation of $\log f$, clamped to avoid log of a non-positive number. Working in $\log f$ matches the near-exponential shape of the Fermi–Dirac tail. The port reproduces this two ways:

- for static query points (the grid-only kernels, whose p_3, p_4 positions are fixed) the 6-point window indices and Lagrange weights are precomputed on the host (`interp.lagrange_window`) and the runtime is a gather + weighted sum + clamp (`interp.apply_log_interp`, which returns clamped $\log f$), followed by the pointwise exp in the kernel;
- for dynamic query points (the lepton kernels, whose nodes move with $m_e/T_{\rm cm}$) the window is found at runtime with `searchsorted` (`interp.interp_log_occ_dynamic`).

[**Port note**] The high-occupation clamp follows the Fortran reference (`transvar.f90`): an interpolation overshoot to $\log f > 0$ is sent to the floor ($\log f = -300$, i.e. $f \to 0$) rather than clamped to $f = 1$. This matters only for unphysical overshoots; on the smooth SM spectrum it never fires.

[**Port note**] **Port-only off-grid closures.** `interp.py` carries three optional off-grid tail closures, the Fermi–Dirac envelope cap $f(p) \le C \cdot f^{\rm eq}(p)$ (`fd_cap_logC` $= \ln C$), `fd_decay` and `monotone_read`, used by the flavour-mixing (QKE) solve to tame a tail amplification (Section 13.5). All three default off (`fd_cap_logC`$= +\infty$ is a bitwise no-op), so the classical SM/$\Delta N_{\rm eff}$ solve does not use them.

3.4 The net/frs decomposition and the precision gate

The gain − loss factor (4) is a catastrophic cancellation near equilibrium: gain and loss are each $\mathcal{O}(1)$ but their difference is $\mathcal{O}(10^{-9})$ or smaller. BURST tracks both the signed difference and the cancellation-free sum of the forward (gain) and reverse (loss) rates:

$$\text{net} = \text{forward} - \text{reverse}, \qquad \text{frs} = \text{forward} + \text{reverse}. \tag{7}$$

`net` is the physical rate; `frs` is the total throughput, used to normalize the cancellation. Because the off-grid interpolation is only $\mathcal{O}(10^{-9}$–$10^{-13})$ accurate (a degree-5 polynomial is not exact for log of a Fermi–Dirac), the raw `net` near equilibrium is dominated by interpolation noise. BURST suppresses it with a **precision gate**: for each bin it forms a noise floor `prec` (the value

of |net|/frs at exact equilibrium, where the true rate is zero) and keeps a rate only if it stands out above the floor,

$$\text{test_ratio} = \frac{|\text{net}|}{\text{frs} \cdot \text{prec}} \;\geq\; \text{nuratiotol} = 3.0 \;\Rightarrow\; \text{keep}; \qquad \text{else set the rate to 0.} \tag{8}$$

The floor (`prec_trans` in the code) is a one-time static precompute at the equilibrium spectrum. This gate is essential to reproduce BURST's $N_{\rm eff}$; it is what stops interpolation noise from random-walking the spectrum during the long near-equilibrium phase.

[**Port note**] **The `optimization_barrier`.** The noise floor is computed from a fixed (input-independent) equilibrium spectrum, so under full `jit` XLA would recognize it as a compile-time constant and evaluate it through its constant-folding path. That path reorders/aggregates the large gain and loss terms differently from the runtime kernel, and so would lose the 10^{-9} cancellation that is the floor, collapsing `prec`$\to 1$, inverting the gate, and shifting $N_{\rm eff}$ (in the decoupling-only configuration) from 3.034 to 3.004. (A deterministic precompute can give two answers because the two evaluation paths use different floating-point arithmetic, and a near-cancellation is exactly where that matters.) Wrapping the equilibrium occupation in `jax.lax.optimization_barrier` marks it opaque, forcing XLA to use the same runtime arithmetic in both places. At runtime the barrier is an identity, so the floor stays bit-faithful (Section 12).

3.5 ODE integration: diffrax, Tsit5, Kvaerno5, PID control

The coupled state is the pair $\{T_{\rm pl},\ f[3,2,N]\}$ (a JAX PyTree), integrated in $\ln a$ by `diffrax`. Two solvers are offered:

- **Tsit5**: an explicit 5th-order Runge–Kutta (Tsitouras), the default. Accurate and cheap per step, but stability-limited in the stiff high-T phase, where the thermally averaged collision loss rate is $\Gamma/H \sim 10^3$ at 10 MeV (the stiffest bin's is $\approx 8.6 \times 10^3$ per e-fold, Section 3.6).
- **Kvaerno5**: an L-stable implicit ESDIRK (explicit-first-stage, singly diagonally implicit Runge–Kutta) method, an alternative for a stiff high-T start. Note the default explicit Tsit5 with `atol` $= 10^{-30}$ already completes the 10 MeV start (it does not blow up there); it is merely stability-limited (slow, not unstable) in the stiff head, which is precisely what the hybrid of Section 3.6 cures.

[**Port note**] BURST integrates in time t with an adaptive Cash–Karp RK5(4) of $\sim 3 \times 10^4$ steps. The port integrates in $\ln a$ (with $T_{\rm cm} = T_{\rm cm}^{\rm init} e^{-\ln a}$ analytic) using diffrax's adaptive solvers and a PID step controller. The physics window is the same.

Step size is set by a **PID controller** on the embedded error estimate, the difference between the high- and low-order solutions the Runge–Kutta pair produces for free at each step, which estimates the local error the controller keeps near the requested tolerance. The defaults are a damped PI law, $(p_{\rm coeff}, i_{\rm coeff}, d_{\rm coeff}) = (0.4, 0.3, 0.0)$, with a relative tolerance `rtol` $= 10^{-6}$ and an absolute tolerance `atol` $= 10^{-30}$. Two non-obvious facts about these defaults (both measured, both documented in Section 10):

- `rtol` $= 10^{-6}$ is faster than the looser 10^{-4}, because at 10^{-4} the explicit controller runs right at the stability edge and spends $\sim 25\%$ of attempts on rejected steps;

- `atol` $= 10^{-30}$ makes the error norm effectively pure-relative, matching BURST. Raising it toward 10^{-7} leaves the stiff $\bar{\nu}$-tail bins error-uncontrolled and silently biases $N_{\rm eff}$ by $+1.5 \times 10^{-3}$ (Section 12). Do not raise it.

These tolerances govern the classical (diagonal) transport ODE only. The QKE flavour-coherence sector (Section 13.3) is deliberately not integrated this way: its off-diagonal substep is an exact matrix exponential carrying no per-component tolerance (the stiff flavour oscillation is exponentiated exactly rather than error-controlled), so `rtol`/`atol` never act on the coherences, and the adaptive-integrator failure mode (a tiny imaginary off-diagonal drifting against a loose, trace-scaled tolerance) cannot arise.

3.6 The exponential-midpoint hybrid solver

In the stiff $10 \to 2.2$ MeV "head" of the solve the collision loss rate is very large (at the 10 MeV start the largest B over all modes and bins is $\approx 8.6 \times 10^3$ per e-fold, where B is the per-bin relaxation coefficient $B = (g{+}l)/H$ of the affine right-hand side (RHS) below, with g and l the bin's gated gain and loss rates per unit time). The spectrum therefore relaxes to the instantaneous collision fixed point (the **quasi-static-equilibrium (QSS) attractor** $f^\star = A/B$, the value at which the collision RHS $\mathrm{d}f/\mathrm{d}\ln a = A - Bf$ vanishes) far faster than the universe expands, and sits on it to one part in 10^{17}. An explicit method is stability-limited there, not accuracy-limited: it burns 65–92% of all steps fighting a stiffness that carries no information (the attractor it is chasing barely moves). The remedy exploits the exact affine structure of the gated collision RHS,

$$\frac{\mathrm{d}f}{\mathrm{d}\ln a} = A - B\,f, \qquad A = g/H \geq 0, \quad B = (g+l)/H \geq 0, \tag{9}$$

which is pure algebra on the kernels' forward/reverse rates (every forward rate carries a Pauli factor $1 - f_1$, every reverse carries f_1, so forward $= (1-f)g$, reverse $= fl$). An exponential integrator treats the linear part exactly:

$$f' = f\,e^{-Bh} + \frac{A}{B}\big(1 - e^{-Bh}\big), \tag{10}$$

for a step h in $\ln a$: a convex combination of f and the attractor A/B, unconditionally stable and positivity-preserving, with no step-size ceiling. (Equation (10) is the 1-stage "exponential-Euler" building block; the head uses its 2-stage midpoint form (A and B re-evaluated at the step midpoint) to reach second order, while both treat the stiff $-Bf$ linear part exactly.) The **hybrid** runs this fixed-step exponential-midpoint integrator over the stiff head (a `lax.scan` of ~ 1600 steps), then hands off to the production adaptive Tsit5 for the accuracy-demanding annihilation tail, where stiffness is gone and order-5 earns its keep (~ 96 tail steps, zero rejections).

The handoff is **qualified**: at the switch $\ln a$ the solve checks that the spectrum is on the attractor ($\max|f - A/B| \leq$ `qual_tol` $= 10^{-4}$) and sane (finite, no negative or super-unity bins). If the check fails, it silently falls back to the full adaptive solve from $\ln a = 0$; both modes share one compiled executable (the fallback is selected by a traced `where` on the tail solve's initial time and state, t_0, y_0). A mis-qualified configuration therefore costs speed, never accuracy. Measured: the hybrid's $N_{\rm eff}$ matches the tight-tolerance (`rtol` $= 10^{-8}$) arbiter to -6.9×10^{-8} (comfortably inside the $|\Delta N_{\rm eff}| \leq 2 \times 10^{-6}$ acceptance bar), at 2.36× the speed of the plain transport solve. The full rationale and the no-go for a pure exponential solve (the order-2 energy budget would need $\sim 18{,}000$ steps; high-order exponential schemes go unstable against the discontinuous precision gate) is in Section 8.

3.7 float64, NaN guards, and differentiability

float64 is **mandatory**: the collision operator is a forward−reverse near-cancellation that float32 cannot resolve (the conservation sum rules fail). The package enables it at import:

```
import neutt # runs jax.config.update("jax_enable_x64", True) on import
```

Import `neutt` before creating any JAX array, or x64 will not take effect for arrays made earlier in the process.

Two robustness guards deserve mention because they are visible in the API:

- `nanrobust_rms_norm` maps a NaN error estimate to $+\infty$ so diffrax's controller routes it into the designed reject-and-shrink path instead of freezing at $\mathrm{d}t = \mathrm{NaN}$ (the diffrax NaN freeze, Section 12); bit-identical on every finite step;
- the kernels use double-`where` guards on `sqrt` and on masked denominators so a masked-off tile cannot emit a NaN that $0 \cdot \mathrm{NaN}$ would propagate. These make every kernel `grad`-safe, a capability the Fortran does not have.

3.8 The GPU-throughput strategy

There are two axes of parallelism, used differently.

Within a call (automatic). A single `solve()` already runs the whole computation on the GPU: the collision kernels are one batched tensor contraction over the (flavour × cp × bin) axes (Section 3.1), so the parallelism lives inside each solve and there is nothing for the user to arrange.

Across calls (batch a scan with `vmap`). A single solve is small and dispatch-bound: its wall time is dominated by the per-call GPU launch overhead, not by arithmetic, so the way to use the GPU efficiently for a scan is to `vmap` the built solver over a parameter array, so one launch does many solves at once:

```
import jax, jax.numpy as jnp
from neutt.hybrid import make_hybrid_solver
solve, _ = make_hybrid_solver() # build / compile ONCE
dneff = jnp.linspace(0.0, 0.5, 1024) # 1024 dark-radiation values
neffs = jax.vmap(lambda d: solve(delta_neff_init=d)["neff_final"])(dneff) # ONE batched launch
```

The background ODE alone drops from ∼60 ms/solve to ∼96 μs/cosmology this way; the full transport solve amortizes its per-call launch overhead the same way, at a larger absolute cost. (`pmap` instead spreads the batch across several GPUs; reach for it only when a batch is too large for one device; a single solve is far too small to shard usefully, Section 14.)

Coupling to LINX: the device split, and collecting the outputs. LINX's background and abundance models compile prohibitively slowly on the GPU, so the coupled driver runs LINX on the CPU and the transport solve on the GPU in one process (Section 9). For a single cosmology `run()` handles the hand-off internally. For a batch, the GPU results must be **collected to the host before the CPU LINX side can use them**; you cannot call a CPU model from inside a GPU `vmap`. The pattern is: batch the transport on the GPU, pull the whole array back to the host in one transfer (optionally save it), then drive LINX over the stored spectra:

```
import numpy as np, jax
specs = jax.vmap(transport_solve)(params) # batched on the GPU
```

```
specs_h = jax.device_get(specs) # ONE GPU -> host transfer
np.savez("transport_batch.npz", **specs_h) # optional: persist for a CPU-only LINX run
for s in specs_h: # LINX (CPU) consumes them one at a time
    abundances = linx_abundances(s)
```

Here `params` is the batched array of per-call inputs, `transport_solve` the GPU spectrum solver and `linx_abundances` the CPU abundance step; the one rule is that the GPU batch is materialized to the host (one transfer, or a saved file) before LINX reads it.

4 Installation, environment, and a first run

4.1 Where the code lives

`neutt` is a pure-Python package; there is nothing to build (no compiled extension). It is installed from PyPI with `pip install neutt` (CPU) or `pip install "neutt[cuda12]"` (NVIDIA GPU), or used from a checkout by putting the repository root on `PYTHONPATH` (or `pip install -e .`). The API reference is a Sphinx site built from `docs/sphinx/` (`pip install "neutt[docs]"`, then `sphinx-build docs/sphinx docs/sphinx/_build`). Throughout this manual `$BURST_ROOT` denotes your checkout of the BURST repository (the `neutt` package is `neutt/` inside it) and `$LINX_ROOT` the LINX checkout used for the abundance coupling (Section 9). A read-only Standard-Model Fortran reference (a snapshot of the Fortran BURST code, against which the port is validated) ships in `fortran_reference/`.

The package at a glance. The modules layer from foundation to coupling (a reader's first map of the code):

layer / module	role
`neutt` (`__init__`)	enables float64 at import; the compile banner `compiling()`
`constants`, `grid`, `interp`	foundation: physical constants, the ε-grid with its Gauss/Boole rules, equilibrium occupations and momentum moments, log-interpolation
`thermo`	the plasma: free-gas $e^{\pm}/\gamma$ EoS, the charge-neutral electron degeneracy $\phi_e = \mu_e/T_{\rm pl}$ (μ_e the electron chemical potential), finite-T QED, Hubble / decoupling clock
`transport/` (`nunu`, `nunubar`, `nuer1`, `nuer2`, `nuepma`, `assemble`)	the SM weak collision kernels (each lepton kernel with its reference and scheduled implementation, Section 6.5) and the gated RHS assembly
`decoupling`, `hybrid`	the coupled solvers: the production RHS, `make_decoupling_solver`, the exponential-midpoint hybrid
`transport/qke/`	the QKE density-matrix solver, the SM flavour-mixing refinement (Section 13)
LINX adapter (separate repo)	`transport_adapter` / `coupled_bbn`: the seam that hands the spectra to LINX for abundances

4.2 Environment (Perlmutter)

Everything that touches JAX runs on a Perlmutter GPU compute node, never the login node. Request an interactive GPU allocation and activate the conda environment:

```
salloc --nodes=1 --qos=interactive --time=04:00:00 \
       --constraint=gpu --gpus=1 --account=<your-account>
module load conda && conda activate <your-jax-env>
export OMP_NUM_THREADS=1
```

```
export XLA_PYTHON_CLIENT_PREALLOCATE=false
export MPICH_GPU_SUPPORT_ENABLED=1
export BURST_ROOT=/path/to/BURST # your BURST checkout
export LINX_ROOT=/path/to/LINX # your LINX checkout (coupled BBN only)
export PYTHONPATH=$BURST_ROOT
```

For the coupled-BBN demos you additionally need the LINX checkout on the path (Section 9). The Python environment must carry JAX with GPU support and `diffrax`.

4.3 Quickstart 1: standalone $N_{\rm eff}$

The most useful entry point. This evolves the coupled (plasma + neutrino-spectrum) system through $e^{\pm}$ annihilation and returns $N_{\rm eff}$:

```
# on a GPU node, with PYTHONPATH=$BURST_ROOT
from neutt.decoupling import make_decoupling_solver
solve, grid = make_decoupling_solver() # builds the kernels and the noise floor
out = solve() # compiles on the first call, then solves
print(float(out["neff_final"])) # -> 3.043322 (NLO QED default; 3.044276 with qed_nlo=False)
```

Building the solver and its first call pay a one-time kernel-build + XLA-compile cost (the slow "cold"/"maiden" call, ∼40–80 s), and the compile prints the banner (a newt, "neutt is compiling…"). Later calls of the same `solve` reuse the compiled executable (the fast "warm" call, no banner) with new values of its per-call arguments; Section 10 says which arguments are which. The result is the canonical Standard-Model number of Section 1.7.

[Port note] This single Python call replaces the transport sector of a 600–1000 CPU-hour Fortran BURST run (the full Fortran run also does the nuclear network and recombination; the transport collision integrals dominate its cost). It computes $N_{\rm eff}$ and the spectra, but not abundances; for those, couple to LINX (Quickstart 3).

4.4 Quickstart 2: a repeated transport solve

For repeated solves (a scan, an MCMC) use the production hybrid solver, which is 2.4× faster than the adaptive `make_decoupling_solver` (Section 8). Every solver has the same shape: a factory `make_*_solver(...)` that builds the kernels and returns a jitted `solve`, whose arguments are the per-call physics inputs:

```
from neutt.hybrid import make_hybrid_solver
solve, grid = make_hybrid_solver() # builds kernels; compiles on the first call
out = solve() # ∼2.8 s warm
print(float(out["neff_final"]), bool(out["qualified"])) # qualified=True: fast hybrid path used
out2 = solve(delta_neff_init=0.1) # extra dark radiation, traced: later values reuse this
  ↪ compile
```

(`qualified=False` would mean the hybrid silently fell back to the full solve, still correct, just slower; Section 8.)

4.5 Quickstart 3: coupled BBN (abundances + $N_{\rm eff}$)

This couples transport to LINX. It needs both repos on the path:

```
export PYTHONPATH=$BURST_ROOT:$LINX_ROOT
```

```python
import neutt # FIRST: enables x64
from linx.coupled_bbn import make_coupled_bbn

run, info = make_coupled_bbn(max_steps=8192) # build ONCE, reuse for a whole chain
res, neff = run(delta_neff_init=0.0) # ~80 s maiden (cold), ~4.2 s warm
Yp = float(res[5] * 4.0) # res[5] = Y(He4); Yp = mass fraction
DH = float(res[2] / res[1]) # res[2]/res[1] = Y(D)/Y(p) = D/H
print(neff, Yp, DH) # -> N_eff 3.043322 (NLO); Yp ~0.2470, D/H ~2.44e-5 (network-dependent)
```

`res` is LINX's `AbundanceModel` yield vector, indexed (0:n, 1:p, 2:D, 3:T, 4:^{3}He, 5:^{4}He, ...) as number abundances $Y_i = n_i/n_b$ (n_b the baryon number density); the helium mass fraction is $Y_{\rm P} = 4\,Y(^4\mathrm{He})$ and $\mathrm{D/H} = Y(\mathrm{D})/Y(p)$.

4.6 Building the Fortran oracle (optional, for cross-checks)

The Standard-Model Fortran reference (`fortran_reference/`) builds with `gfortran` and is what the port is validated against. You only need to build it to produce a new reference output: the committed snapshot outputs already cover the standard checks and are loaded by the test suite (Section 11).

5 Core module reference

This section documents the four "core" modules, `constants`, `grid` (the grid, its quadrature, the occupation numbers, and the moments), `interp` (the interpolation), and `thermo` (the background plasma thermodynamics), everything the transport kernels (Section 6) and the decoupling solver (Section 7) rest on. Each subsection gives the purpose, the public API and the Fortran origin, and most add a short runnable demo. A complete routine-by-routine mapping index is in Appendix A.

5.1 `constants`: physical constants

Purpose. Module-level physical and mathematical constants, ported bit-for-bit from BURST (except $\sin^2\theta_W$ and the last digits of $\zeta(3)$, below) so that parity checks are never polluted by a constant mismatch. Pure values, no functions.

The Standard-Model–relevant constants:

symbol	value	meaning / Fortran source
M_ELECTRON	0.510998928	electron mass [MeV] (xmelec, mainvar.f90)
GFHBAR	0.20668190937	$G_F^2/\hbar$ [$s^{-1}MeV^{-5}$], weak collision-rate scale (transvar.f90)
SIN2THW	0.23122	$\sin^2\theta_W$ ($\overline{\text{MS}}$ scheme at the Z mass M_Z); enters the ν–lepton kernels (and their QKE matrix counterparts) only. Fortran BURST uses 0.23 (SIN2THW_BURST), which the Fortran-parity tests pin
ALPHA_FS	$7.2973525698\times10^{-3}$	fine-structure α
MPL	1.220932×10^{22}	Planck mass [MeV]
HBAR	$6.582119514\times10^{-22}$	$\hbar$ [MeV·s]
PI	3.1415926535897932	π
ZETA3	1.202056903159594	$\zeta(3)$
BR_COEFFS	$(64, 24, 64, 28)/45$	interior Boole's-rule weights (brcoeffs, mainvar.f90)
BR_END	14/45	Boole's-rule end-point weight (brends)

[**Fortran ↔ JAX**] Values mirror the snapshot's mainvar.f90 (pi, mpl, hbar, zeta3, xmelec, alphafs at lines 25–37; brcoeffs, brends at 102–103), verified exact except ZETA3: mainvar.f90:34 truncates $\zeta(3)$ to 1.2020569031, while the port carries it to double precision (it enters only the baryon density behind ϕ_e and the QKE n_p). GFHBAR and SIN2THW_BURST mirror transvar.f90:26--27 (gfhbar, sin2thw; production SIN2THW is the more precise 0.23122).

[**Port note**] The module also defines QKE flavour-mixing constants (GFERMI= G_F, the weak-potential scales GFW2HBAR = $G_F/m_W^2\hbar$ and GFZ2HBAR = $G_F/m_Z^2\hbar$ with m_W, m_Z the W and Z boson masses, and the PMNS (Pontecorvo–Maki–Nakagawa–Sakata) mixing angles and mass-squared splittings Δm^2_{21}, Δm^2_{31}). These feed the QKE flavour-mixing sector (Section 13); the classical SM transport never reads them.

5.2 grid: the comoving-energy grid, quadrature, occupations, and moments

The module holds everything that lives on, or integrates over, the momentum grid: the grid itself, the Gauss rules of the inner collision and plasma integrals, the equilibrium occupations, and the Boole-weighted moments.

5.2.1 The ε-grid

Purpose. Defines the linear $\varepsilon = p/T_{\rm cm}$ grid on which every distribution is discretized, with baked-in composite Boole's-rule weights so a momentum integral is just a weighted sum $\int g\,\mathrm{d}\varepsilon \approx \sum_i w_i\, g(\varepsilon_i)$.

API.

```
class EpsGrid(eqx.Module):
    abscissas : (nbins,) # eps_i
    weights : (nbins,) # composite-Boole w_i ; sum(weights) == eps_range
    nbins, eps_range, eps_offset # static

make_eps_grid(nbins=101, eps_range=25.0, eps_offset=0.0,
              bin1_weight_frac=0.0) -> EpsGrid
```

The grid has $N =$`nbins` points on $[\varepsilon_{\text{offset}}, \varepsilon_{\text{offset}} + \varepsilon_{\text{range}}]$ (`eps_offset`, `eps_range`); `bin1_weight_frac` adds $\varepsilon_{\text{offset}} \cdot$`bin1_weight_frac` to the first weight (BURST's offset term). The defaults (101 bins, $\varepsilon \in [0, 25]$, no offset) are BURST's. `nbins` must be $4n+1$ (Boole requirement) or a `ValueError` is raised. The spacing is $h = \varepsilon_{\text{range}}/(N-1)$; interior weights cycle through `BR_COEFFS`$\times h$ and the endpoints carry `BR_END`$\times h$.

[Fortran ↔ JAX] `make_eps_grid` ≡ `part_lib.f90::part_make_bins`; defaults from the `part_bin_var` namelist (`nbins=101`, `epsrange=25`). Bit-faithful transcription.

Demo.

```
from neutt.grid import make_eps_grid
import numpy as np
g = make_eps_grid()
print(g.nbins, float(g.abscissas[-1]), float(np.sum(g.weights))) # 101 25.0 25.0
```

5.2.2 Equilibrium occupations

Purpose. Equilibrium occupation numbers on the grid and the log-space clamp used by interpolation. A species in equilibrium at temperature T_s and degeneracy $\xi = \mu/T_s$ has $f(\varepsilon) = 1/(e^{\varepsilon/r - \xi} \pm 1)$ (+ for fermions, − for bosons) with $r = T_s/T_{\text{cm}}$ (the `escale_ratio` argument).

API.

```
fermi_dirac(eps, escale_ratio=1.0, xi=0.0) # 1/(exp(eps/r - xi)+1) (fermions)
bose_einstein(eps, escale_ratio=1.0, xi=0.0) # 1/(exp(arg)-1), guarded (bosons)
log_occupation(occ) # clip(log(occ), -300, 0)
LOG_OCC_FLOOR = -300.0
```

`fermi_dirac`, the Fermi–Dirac (FD) occupation, is overflow-safe in float64; `bose_einstein` returns 0 outside arg $= \varepsilon/r - \xi \in (0, 300]$ (avoiding the $\varepsilon \to 0$ divergence) and is a reference massless-boson occupation; the SM neutrinos use `fermi_dirac`.

[Fortran ↔ JAX] Maps to Fortran `fd_equil_calc` / `be_equil_calc`; the log clamp `LOG_OCC_FLOOR`$= -300$ is the floor of `interp_occfrac` (`transvar.f90`) (the same value as `interp`'s `LOG_OCC_CLAMP`, Section 5.3).

5.2.3 Gauss nodes and rescaling

Purpose. Gauss–Legendre and (generalized) Gauss–Laguerre nodes/weights for the collision-integral inner integrals and the thermo/QED momentum integrals. Classical rules are precomputed on the host; only the affine rescale is jittable.

API.

```
legendre_canonical(n) -> (x, w) # Gauss-Legendre on [-1,1]
gauss_laguerre(n, alpha=0.0) -> (x, w) # generalized GL: int x^a e^-x f
gauss_laguerre_plain(n) -> (x, w) # weights * exp(x): int_0^inf f (workhorse)
rescale_legendre(x, w, a, b) -> (x', w') # affine [-1,1] -> [a,b] (jittable)
gauss_legendre(n, a=-1.0, b=1.0) -> (x, w) # canonical, rescaled to static [a,b]
```

`gauss_laguerre_plain` (e^x folded into the weights so $\sum w_i f(x_i) \approx \int_0^\infty f$) is the rule used by `thermo` (free-gas EoS, ϕ_e, finite-T QED); it is BURST's `cgqf` with rule kind 5 (generalized Laguerre) followed by `wts *= exp(absc)` (the node count is a separate argument: 25 for the ϕ_e and QED integrals, 100 for the free-gas $e^{\pm}$ integrals). `rescale_legendre` is the `scqf` equivalent and is pure arithmetic, so it is `jit`/`vmap`-friendly; `gauss_legendre` uses it for a static interval, and the kernels apply the same affine map inline to `legendre_canonical`'s nodes, broadcast over their moving collision-integral limits.

[**Port note**] The Golub–Welsch eigensolve (`gauss.f90:imtqlx`) is **not ported**: nodes come from NumPy/SciPy on the host, only the rescale is ported. See Section 3.2.

5.2.4 Momentum-space moments

Purpose. Boole's-rule weighted moment integrals over an `EpsGrid`; the building blocks for number/energy densities and the conservation sum rules.

API.

```
integrate(grid, fvals) # sum_i w_i f_i = int f deps
moment(grid, occ, k) # sum_i w_i eps_i^k occ_i = int eps^k occ deps
number_density(grid, occ, g=1.0, t_cm=1.0) # n = g/(2pi^2) T^3 int eps^2 occ deps
energy_density(grid, occ, g=1.0, t_cm=1.0) # rho = g/(2pi^2) T^4 int eps^3 occ deps
```

All are vectorized and `jit`/`vmap`-friendly. The $k = 2$ moment gives number, $k = 3$ gives energy (and feeds $N_{\rm eff}$).

Demo: a thermal cross-check.

```
import neutt
from neutt.grid import make_eps_grid, energy_density, fermi_dirac
import numpy as np
grid = make_eps_grid()
f_eq = fermi_dirac(grid.abscissas) # FD at T_cm
rho = energy_density(grid, f_eq, g=2.0, t_cm=1.0) # the keyword g=2.0 is the dof, not the grid
# analytic relativistic fermion: (7/8)(pi^2/30)*g*T^4, here T=1, g=2
print(float(rho), 7/8 * np.pi**2/30 * 2) # agree to ~1e-10
```

5.3 `interp`: log-space occupation interpolation

Purpose. The order-5 (6-point) log-space interpolation of the occupation, the port of BURST's `interp_occfrac`. Recast as a sparse linear (Lagrange) operator on $\log f$ for static query points, with a fully-dynamic variant for the moving lepton-kernel queries (Section 3.3).

API.

```
ORDPOLINT = 5 # 6-point window
LOG_OCC_CLAMP = -300.0 # log-occupation floor (= grid.LOG_OCC_FLOOR)
FD_ENVELOPE_C = 10.0 # the QKE cap VALUE (env NEUTT_FD_ENVELOPE_C overrides); default
  ↪ fd_cap_logC=inf leaves it unused

lagrange_window(abscissas, query, ord=5) -> (idx, weights) # HOST precompute
apply_log_interp(logf_grid, idx, weights,
       fd_decay_base=None, bracket=None) -> logf # jittable gather+clamp
interp_log_occ_dynamic(logf_grid, abscissas, query, ord=5,
       fd_cap_logC=inf, fd_decay=False,
       monotone_read=False) -> logf # jittable, traced query
```

For the SM kernels: the grid-only kernels use `lagrange_window` (host) + `apply_log_interp` (runtime); the lepton kernels use `interp_log_occ_dynamic`. An overshoot $\log f > 0$ is always sent to the -300 floor (the fermion clamp); the off-grid closures (`fd_cap_logC`, `fd_decay`, `monotone_read` in the dynamic read, and their `fd_decay_base`/`bracket` counterparts in the static one) default off, the SM-faithful setting.

[Port note] The high-occupation clamp sends an interpolation overshoot to $\log f > 0$ down to the floor ($f \to 0$) rather than clamping it to $f = 1$; it fires only on unphysical overshoots and is bit-faithful to the validated Fortran reference (Section 3.3). `fd_cap_logC` (the envelope $f \le C\, f^{\rm eq}$ with $\log C =$`fd_cap_logC`), `fd_decay` (off grid, $\log f$ capped at the FD shape anchored to the grid-edge value) and `monotone_read` (the interpolant clamped to its two bracketing nodes; off grid both are the edge node, so the read is pinned to the edge value, which `fd_decay` then lowers to the FD tail) are the tail closures used by the QKE solve (Section 13.5); all default off, so they do nothing in the classical SM solve.

5.4 `thermo`: plasma thermodynamics, finite-T QED, and the background

The module holds the $e^{\pm}/\gamma$ plasma: its free-gas equation of state, the finite-T QED mass renormalization, the QED equation-of-state corrections (the LO terms built from it, and the NLO term), the decoupling-background clock, and the charge-neutral electron degeneracy $\phi_e \equiv \mu_e/T_{\rm pl}$ (μ_e the electron chemical potential). `solve_phie` solves charge neutrality $n_{e^-} - n_{e^+} = q_{\rm baryon}\, n_b$ (n_b the baryon number density, set by the present baryon-to-photon ratio η_0, argument `eta0`; $q_{\rm baryon}$ the net charge per baryon, argument `q_baryon`, default 0.5) by fixed Newton steps from the non-degenerate asinh seed; it is what `phie="auto"` evaluates inside the transport RHS (Section 12).

```
solve_phie(tplmev, tcmpl, eta0, q_baryon=0.5, m_e=M_ELECTRON,
          nodes=None, weights=None, n_newton=6) # phi_e; NaN below T_pl ~m_e/300
phie_linear(tplmev, tcmpl, eta0, q_baryon=0.5, m_e=M_ELECTRON,
           nodes=None, weights=None) # BURST's linearized phi_e
net_electron_number_over_T3(phi, mt, nodes=None, weights=None) # (n_e- - n_e+)/T_pl^3, mt = m_e
  ↪ /T_pl
nb_over_Tcm3(eta0) # n_b/T_cm^3 = eta0 * N_GAMMA * 11/4
```

Here `tcmpl`$= T_{\rm cm}/T_{\rm pl}$, and `nodes`/`weights` override the default 25-node plain Gauss–Laguerre rule. `phie_linear` is the single linear step from $\phi_e = 0$, a validation reference that equals `solve_phie` to machine precision at high T ($\phi_e \sim 10^{-9}$).

5.4.1 The free-gas equation of state

Purpose. The free-gas $e^{\pm}/\gamma$ plasma equation of state: photon and $e^{\pm}$ energy densities, pressures, and $\mathrm{d}\rho/\mathrm{d}T$, by Gauss–Laguerre momentum integrals. Uses the **bare** electron mass; the finite-T QED correction is added separately (Section 5.4.3).

API.

```
rho_gamma(T) # photon rho = (pi^2/15) T^4 (g=2)
pressure_gamma(T) # rho_gamma/3
rho_electron(T, phi=0.0, m=M_ELECTRON, nodes=None, weights=None) # e- + e+
pressure_electron(T, phi=0.0, m=M_ELECTRON, nodes=None, weights=None)
rho_plasma(T, phi=0.0, m=M_ELECTRON) # rho_gamma + rho_electron
pressure_plasma(T, phi=0.0, m=M_ELECTRON)
drho_plasma_dT(T, m=M_ELECTRON) # analytic drho/dT at phi=0
drho_plasma_dT_phi(T, phi, m=M_ELECTRON) # at fixed phi != 0 (autodiff)
```

The argument `phi` is ϕ_e; the standard background uses $\phi = 0$ (the baryon asymmetry $\sim 10^{-10}$ is negligible). The $e^{\pm}$ integrals default to a 100-node plain Gauss–Laguerre rule.

[**Fortran ↔ JAX**] Integrands are the leading (free-gas) terms of BURST's `glag4` (energy) and `glag7` (pressure) in `bbn_therm.f90`. $\mathrm{d}\rho/\mathrm{d}T$ uses the analytic $f(1-f)$ form, cross-checked against `jax.grad`.

[**Port note**] Two intentional deltas: (i) the default uses 100 Gauss–Laguerre nodes vs. BURST's 25 (accuracy margin; pass BURST's 25-node rule via `nodes/ weights` to match BURST bit-for-bit); (ii) the integrand uses the bare mass; folding a thermal mass shift in here would double-count and send $N_{\rm eff}$ the wrong way (Section 5.4.3).

Demo.

```
import neutt
from neutt import thermo
T = 2.0 # MeV
print(float(thermo.rho_plasma(T)), float(thermo.drho_plasma_dT(T)))
```

5.4.2 Finite-T QED mass renormalization

Purpose. The thermal self-energies δm_e^2, δm_γ^2 (momentum independent) and the momentum-dependent electron self-energy. These feed the QED EoS correction; physically they encode that the plasma shifts the effective electron and photon masses.

API.

```
get_eg_mass_renorm(tplmev, phie, nodes=None, weights=None, m_e=M_ELECTRON)
    -> (dmeeps2, dmedteps, dmedt2, dmgeps2, dmgdteps, dmgdt2) # 6 dimensionless
delta_me2(tplmev, phie=0.0, ...) # dmeeps2 * T_pl^2 [MeV^2]
delta_mg2(tplmev, phie=0.0, ...) # dmgeps2 * T_pl^2 [MeV^2]
m_e_eff(tplmev, phie=0.0, ...) # sqrt(m_e^2 + delta_me2) [MeV]
get_e_mass_pdep(tplmev, phie, peps, ...)
    -> (dmep1, dmep2, dmep3, dmep4, dmep5) # momentum-dependent, shaped like peps
```

Here `tplmev`= $T_{\rm pl}$ in MeV and `phie`= ϕ_e. The six outputs of `get_eg_mass_renorm` are exactly the Fortran out-args, in order: `dmeeps2`= $\delta m_e^2/T_{\rm pl}^2$, then `dmedteps` and `dmedt2`, its first- and second-T-derivative terms, then the same three for δm_γ^2; they combine the $2\pi\alpha/3$ thermal-photon piece with the $e^{\pm}$ thermal integral $(2\alpha/\pi)\int\ldots$. `get_e_mass_pdep` does the momentum-dependent piece at `peps`= $p/T_{\rm pl}$ (outputs $\delta m_e^2(p)/T_{\rm pl}^2$ and its T, p, T^2 and pT derivative terms), integrating over the loop momentum $y = k/T_{\rm pl}$ with a reflected-Legendre rule about the singularity at $y =$`peps` plus a shifted Laguerre tail (the principal-value construction).

[**Fortran ↔ JAX**] `get_eg_mass_renorm` ≡ `renorm.f90:29--114` (verified line-for-line); `get_e_mass_pdep` ≡ `renorm.f90:119--282`. The 25-node default rule matches BURST's `glag_bins=25`.

5.4.3 The finite-T QED equation of state

Purpose. BURST's `glag1`–`glag7` correction terms: the additive $\delta\rho$, δP, $\delta(\mathrm{d}\rho/\mathrm{d}T)$ that turn the bare-mass free-gas EoS into the $\mathcal{O}(\alpha)$ QED plasma EoS. Built from the thermal self-energies of Section 5.4.2, not from a shifted mass in the free-gas integrand.

API.

```
qed_eos_corrections(tplmev, phie=0.0, ...)
    -> (d_rho_gamma, d_P_gamma, d_drho_gamma_dT, d_rho_e, d_P_e, d_drho_e_dT)
qed_eos_pdep_corrections(tplmev, phie=0.0, ..., **pdep_rules)
    -> (d_rho_e_pdep, d_P_e_pdep, d_drho_e_dT_pdep)
qed_plasma_deltas(tplmev, phie=0.0, ..., pdep=True, m_e=M_ELECTRON, nlo=False)
    -> (d_rho_plasma, d_P_plasma, d_drho_plasma_dT) # photon + e+- summed
rho_plasma_qed_delta(..., nlo=False), pressure_plasma_qed_delta(...), drho_plasma_dT_qed_delta
  ↪ (...)
    -> its three components, one each
```

`qed_plasma_deltas` is the one-call form used in the coupled-solve RHS (it evaluates the renorm integral once). `pdep=True` (default, BURST-faithful) includes the momentum-dependent second-loop terms; the solvers pass it as `renorm_pdep`, and `renorm_pdep=False` (with `qed_nlo=False`) reproduces the momentum-independent pin $N_{\mathrm{eff}} = 3.044313$. `nlo=True` adds the NLO term of Section 5.4.4; it defaults to `False` at this level, while every solver passes `qed_nlo=True`. Add the deltas to the corresponding `thermo` quantities.

[**Fortran ↔ JAX**] The `glag1`–`glag7` correction terms in `bbn_therm.f90` (momentum-independent block plus the second-loop pdep block), verified.

[**Port note**] **Not ported:** the `glag6` ($\mathrm{d}\rho_e/\mathrm{d}\phi_e$) EoS-derivative term. The degeneracy ϕ_e itself is carried (settable, or auto-solved from charge neutrality; Section 12); only this QED-EoS feedback on ϕ_e is omitted, and the physical $\phi_e \sim 10^{-10}$ makes it negligible.

Demo: the QED shift in the plasma energy.

```
import neutt
from neutt import thermo
T = 1.0
rho0 = float(thermo.rho_plasma(T))
drho = float(thermo.rho_plasma_qed_delta(T, pdep=True))
print(rho0, drho, drho/rho0) # per-cent-level QED correction to rho_plasma
```

5.4.4 NLO QED (the `qed_nlo` default)

Everything above is the leading-order $\mathcal{O}(\alpha)$ finite-T QED correction (the thermal self-energies and the `glag` terms), which is all Fortran BURST has. The port adds the **next-to-leading-order** $\mathcal{O}(e^3)$ correction on top, and it is on by default.

The flag. `qed_nlo` (boolean) toggles the NLO term, and it defaults **on** at every entry point and builder: `make_decoupling_solver`, the RHS/kernel builders (`make_rhs`, `make_split_rates`), `make_hybrid_solver`, the full density-matrix QKE solver (`make_full_coupled_solver`, Section 13), and the coupled-BBN driver (next paragraph) all run NLO by default. Passing `qed_nlo=False` anywhere recovers the bit-faithful LO BURST replica (the Fortran-parity oracle tests set it explicitly).

The physics. The NLO term is the $\mathcal{O}(e^3)$ finite-T QED interaction pressure that the LO renormalization omits, a port of LINX's `explicit_P3` (Bennett et al., arXiv:1911.04504, Eq. 4.24),

$$\delta P_{e^3} = \frac{e^3}{12\pi^4}\, T^4\, J^{3/2}, \qquad J = \sum_i w_i\, \frac{2x_i^2 + (m_e/T)^2}{E_i}\, \frac{2}{e^{E_i}+1},$$

with $e^3 = (4\pi\alpha)^{3/2}$, $T = T_{\rm pl}$, $E_i = \sqrt{x_i^2 + (m_e/T)^2}$, and x_i, w_i the nodes and weights of the 25-node plain Gauss–Laguerre rule ($x = p/T$). The energy density and heat-capacity corrections follow from the thermodynamic relations $\delta\rho = T\,\mathrm{d}P/\mathrm{d}T - P$ and $\delta(\mathrm{d}\rho/\mathrm{d}T) = T\,\mathrm{d}^2P/\mathrm{d}T^2$, with the T-derivatives taken by forward-mode autodiff (so the term stays differentiable and varies coherently with a traced m_e). The occupation carries BURST's $E/T > 300$ cutoff, as the $\mathcal{O}(\alpha)$ terms do, so the term is exactly zero, with zero derivatives, once every node is cut ($T < m_e/300$). It shifts $N_{\rm eff}$ by -9.5×10^{-4} (3.0442761 → 3.0433222; 3.0442252 → 3.0432716 at `SIN2THW_BURST`), reconciling with Cielo et al. (arXiv:2306.05460). The implementation is `thermo.qed_eos_e3_pressure` / `qed_eos_e3_deltas`, added by `qed_plasma_deltas(..., nlo=True)`.

The double-count guard (coupled BBN). In `make_coupled_bbn` with `transport="full"` (Section 9; under `"qke"` set `qke_kwargs["qed_nlo"]` to match) the transport `qed_nlo` is tied to LINX's own `BackgroundModel NLO` flag (default `True`). This matters: the transport NLO acts on the neutrino-heating / $N_{\rm eff}$ sector, while LINX's native NLO acts on the (disjoint) charged-plasma EoS, and the coupling discards LINX's own neutrino sector and swaps in the transport one, so the QED→ ν effect must be supplied exactly once. Tying the two flags guarantees that; an explicit `solver_kwargs["qed_nlo"]` (or `qke_kwargs["qed_nlo"]` in qke mode) that disagrees with LINX's `NLO` still works but emits a mismatch warning. (The coupled $\Delta N_{\rm eff}$ between NLO and LO is -9.5×10^{-4}, the same as standalone, confirming no double-count, which would have given $\approx -1.9\times10^{-3}$.)

[**Port note**] NLO QED is a documented improvement over Fortran BURST (which carries only the LO correction), ported from LINX. `qed_nlo=False` restores exact LO BURST parity.

5.4.5 The decoupling "clock"

Purpose. The standalone plasma-temperature and expansion evolution, with neutrinos as an instantaneously-decoupled relativistic bath. Photons reheat through $e^\pm$ annihilation, recovering $T_\gamma/T_\nu \to (11/4)^{1/3}$ and $N_{\rm eff} \to 3$. This is the reference background without neutrino heating; the coupled solver (Section 7) adds the collision energy-exchange that makes $N_{\rm eff} > 3$.

API.

```
rho_nu(T_nu, n_nu=3.0) # n_nu (7/8)(pi^2/15) T_nu^4
hubble(rho_tot) # sqrt(8 pi rho /(3 MPL^2))/HBAR [1/s]
neff(T_nu, T_gamma, n_nu=3.0) # (8/7)(11/4)^{4/3} rho_nu/rho_gamma
evolve_background(T_init=100.0, lna_max=10.0, n_save=256,
                  rtol=1e-10, atol=1e-12) -> dict
comoving_plasma_entropy(T_pl, lna) # (rho_pl+P_pl)/T_pl * a^3 (conserved)
```

`evolve_background` integrates $\mathrm{d}T_{\rm pl}/\mathrm{d}\ln a = -3(\rho_{\rm pl}+P_{\rm pl})/(\mathrm{d}\rho_{\rm pl}/\mathrm{d}T)$ (covariant energy conservation of the decoupled, $\mu = 0$ plasma; H drops out of the T-equation) with diffrax Tsit5. The returned dict has `lna`, `T_pl`, `T_nu` (=`T_init` $e^{-\ln a}$, the decoupled-neutrino temperature), `ratio` ($= T_{\rm pl}/T_\nu$), `neff_final`, `ratio_final`.

[**Fortran ↔ JAX**] `hubble` matches the snapshot's `bbn_yes_evolve_v2.f90:156` (`hubcst = sqrt(8/3 pi/mpl^2 thm(10)/mev4tocgs)/hbar`, `thm(10)` the total energy density), verified; the port works in MeV^4 and skips BURST's `mev4tocgs` unit conversion. The $\mathrm{d}T_{\rm pl}/\mathrm{d}\ln a$ form replaces BURST's thermo-driven temperature stepping.

Demo: the instantaneous-decoupling sanity check.

```
import neutt
from neutt.thermo import evolve_background
bg = evolve_background()
print(float(bg["ratio_final"]), float(bg["neff_final"])) # ~1.401019, ~3.000007
```

The ratio matches $(11/4)^{1/3} = 1.401020$; $N_{\rm eff} \to 3$ to 7×10^{-6}. The gap to 3.044 is exactly the transport heating the coupled solver adds.

6 The Standard-Model transport kernels

The five weak collision kernels and their assembly into the RHS live under `neutt/transport/`. This is the hard core of the port, the dominant cost in BURST and the part that the kernel-tensor reformulation (Section 3.1) exists to accelerate.

6.1 File inventory and Fortran mapping

`neutt` module	physical channel	snapshot Fortran (oracle)
`transport/nunu.py`	$\nu\nu$, $\bar\nu\bar\nu$ elastic scatter	`trans_nunu.f90`
`transport/nunubar.py`	$\nu\bar\nu$ scatter + $\nu_i\bar\nu_i \to \nu_j\bar\nu_j$ annih	`trans_nunubar.f90`
`transport/nuer1.py`	$\nu e^\pm \to e^\pm\nu$ (R1 reduction)	`trans_nuer1.f90`
`transport/nuer2.py`	$\nu e^\pm \to e^\pm\nu$ (R2 reduction)	`trans_nuer2.f90`
`transport/nuepma.py`	$\nu\bar\nu \to e^-e^+$ annihilation	`trans_nuepma.f90`
`transport/assemble.py`	precision gate + RHS assembly	`trans_evolve.f90`

6.2 `assemble`: precision gate and RHS

The main RHS.

```
assemble_rhs(grid, occ, tcmev, tplmev, phie, prec_trans,
             nunu_kernel, nunubar_kernel, nuratiotol=3.0,
             m_e_eff=M_ELECTRON, use_sched=False, gate_width=0.0, lag_tail_fix=True)
    -> (doccfracdt[3,2,N], drhonudt) # [s^-1], [MeV^4/s]
```

Here `occ` is the occupation $f[\text{flavour} = 3, \text{cp} = 2, \text{bin} = N]$, and this index convention is used throughout: **flavour** $\in \{0{:}e,\ 1{:}\mu,\ 2{:}\tau\}$, **cp** $\in \{0{:}\nu,\ 1{:}\bar\nu\}$, **bin** the ε-grid index (so `f[0,0]` is the ν_e spectrum, `f[0,1]` the $\bar\nu_e$, etc.). `tcmev`= $T_{\rm cm}$, `tplmev`= $T_{\rm pl}$ in MeV; `phie`= ϕ_e; `prec_trans` the noise floor; the two `*_kernel` are the precomputed grid-only kernels (Section 6.3); `nuratiotol` is the gate threshold of Eq. (8), `use_sched` the lepton-kernel path (Section 6.5), and `gate_width` an optional smooth gate (below). The internals follow `trans_evolve.f90` exactly:

1. form meps $= m_e/T_{\rm cm}$ (the `m_e_eff` argument is the bare mass, default `M_ELECTRON` despite its name, distinct from `thermo.m_e_eff`, the thermal effective mass, which is not fed to the kernels) and tcmpl $= T_{\rm cm}/T_{\rm pl}$;
2. evaluate all five kernels to get, per (flavour, cp, bin), numer $= \sum$net (all channels), denom $= \sum$frs, and leg4 $=$ net$_{R1}$ + net$_{R2}$ + net$_{\rm pma}$ (only the three electron-coupled lepton kernels carry plasma exchange);

3. apply the precision gate (8): doccfracdt $= T_{\rm cm}^5$ net where the gate is open, else 0;
4. form drhonudt $= \frac{T_{\rm cm}^9}{2\pi^2}\sum_i w_i\varepsilon_i^3 \sum_{\rm flavour,cp}$(gated leg4), the energy exchanged with the plasma.

`doccfracdt` is $\mathrm{d}f/\mathrm{d}t$ in s^{-1}; the division by H to get $\mathrm{d}f/\mathrm{d}\ln a$ happens in the decoupling driver, not here.

The `leg4` convention. The name comes from BURST's rate arrays `ratescatt(4,2,3,2,nbins)` and `rateannih(4,3,2,nbins)` (`transvar.f90`, allocated in `trans_init.f90`), whose leading index labels the other species in the process (the scattering partner, or the pair an annihilation produces): slots 1–3 a neutrino flavour, slot 4 the $e^{\pm}$. Only `nuer1` and `nuer2` ($\nu e^{\pm}$ scattering) and `nuepma` ($\nu\bar{\nu} \leftrightarrow e^- e^+$) fill slot 4; `nunu` and `nunubar`, whose partners are all neutrinos, fill slots 1–3. Slot 4 is therefore the part of the net rate that exchanges energy with the plasma, and `trans_evolve.f90` sums it alone into `drhonudt`. The port encodes this by summing only the three lepton-kernel nets into `leg4`. It is unrelated to the fourth momentum leg p_4 of a $1+2 \to 3+4$ process, the off-grid leg read by interpolation (Section 3.1).

The affine split (for the hybrid head).

```
assemble_split_rhs(grid, occ, tcmev, tplmev, phie, prec_trans,
                   nunu_kernel, nunubar_kernel, nuratiotol=3.0,
                   m_e_eff=M_ELECTRON, use_sched=False, fd_cap_logC=inf, lag_tail_fix=True,
                   fd_decay=False, monotone_read=False)
    -> (A[3,2,N], B[3,2,N], drhonudt) # df/dt = A - B*occ, both ≥0
```

This recasts the gated RHS as $\mathrm{d}f/\mathrm{d}t = A - Bf$ (Eq. (9)), $A = T_{\rm cm}^5\,\text{forward}/(1-f)$, $B = A + T_{\rm cm}^5\,\text{reverse}/f$, pure algebra on what `assemble_rhs` already computes (reproduces it to $\leq 3.5 \times 10^{-12}$ relative). The $(1-f)$ division floors at `SPLIT_ONEMF_FLOOR`$= 10^{-3}$ (exact for $f \leq 0.999$); A, B are clamped ≥ 0 so the exponential step (10) cannot push f below the occupation floor. Used by the exponential-midpoint steps of the hybrid head (Section 8) and of the QKE collision substep (Section 13.3).

Equilibrium occupation and the noise floor.

```
equilibrium_occ(grid, xi=(0.0,0.0,0.0)) -> f0[3,2,N] # 1/(exp(eps - parfact*xi)+1)
compute_prec_trans(grid, nunu_kernel, nunubar_kernel,
                   xi=(0,0,0), use_sched=False, lag_tail_fix=True) -> prec_trans[3,2,N]
compute_prec_trans_at(grid, nunu_kernel, nunubar_kernel, occ_eq, meps, tcmpl,
                      phie=0.0, use_sched=False, lag_tail_fix=True) -> prec[3,2,N]
```

`equilibrium_occ` builds the Fermi–Dirac initial condition, or IC (`parfact`$= +1$, i.e. $+\xi$, for cp$= 0$; $-\xi$ for cp$= 1$; the $\varepsilon = 0$ bin is exactly 0.5 at $\xi = 0$, more generally $1/(e^{\mp\xi}+1)$). `compute_prec_trans` runs all five kernels at that equilibrium spectrum (with meps $= 0$, tcmpl $= 1$, $\phi_e = 0$) and records $|\sum \text{net}|/\sum \text{frs}$ per bin, the per-bin interpolation-cancellation noise floor. It is a static precompute (grid $+\,\xi$ only), wrapped in the `optimization_barrier` of Section 3.4. It calls `compute_prec_trans_at`, which measures the same floor at any given equilibrium occupation `occ_eq` and background (`meps`, `tcmpl`, `phie`); the $\varepsilon = 0$ bin is 0 and bins with $\sum \text{frs} \leq 0$ get `PREC_FLOOR`.

[**Fortran ↔ JAX**] `assemble_rhs` ≡ snapshot `trans_evolve.f90:195--208` (the combined gate); `compute_prec_trans` ≡ `trans_init.f90:268--286`; `equilibrium_occ` ≡ `trans_init.f90:185--196`. Module constants: `NURATIOTOL = 3.0` (from `trans_params.ini`), `OCC_FLOOR = 1e-300`, `PREC_FLOOR = 1e-13`.

[**Port note**] **A single combined precision gate.** The port applies one gate over the summed `number/denom` with a single `prec_trans` noise floor, bit-faithful to the Fortran reference and validated to $N_{\rm eff} = 3.044225186$ (LO, at `SIN2THW_BURST`$= 0.23$). The alternative, gating each collision channel separately, is not used; the combined gate is what the reference does and what the port reproduces.

[**Port note**] **Port-only knobs in `assemble`.** `gate_width>0` switches the hard gate (8) to a smooth sigmoid weight (a continuous-RHS robustness lever, default 0 = BURST's hard gate); `fd_cap_logC` is the FD-envelope cap (default ∞, a no-op). `OCC_FLOOR` clamps f to $[10^{-300}, 1]$ before any log, a positivity guard BURST does not have (it relies on the interp clamp instead). All three are no-ops for the SM solve. The module also exposes `decoherence_rate` (ungated $g + l$, for the QKE off-diagonal damping) and `compute_sumrules` (below); both are diagnostics/QKE, not part of the core SM RHS.

The QKE damping rates.

```
decoherence_rate(grid, occ, tcmev, tplmev, phie, nunu_kernel, nunubar_kernel,
                 m_e_eff=M_ELECTRON, use_sched=False, fd_cap_logC=inf,
                 fd_decay=False, monotone_read=False, lag_tail_fix=True) -> Gamma[3,2,N] # [s
                   ↪ ^-1]
decoherence_rate_split(...same arguments...) -> (gamma_nue, gamma_nunu)
```

`decoherence_rate` is the ungated $\Gamma_a = g_a + l_a$, the B of `assemble_split_rhs` without the gate, which the QKE solve uses as the scalar coherence damping (Section 13.4). `decoherence_rate_split` splits it into the ν–$e^{\pm}$ part (`nuer1`+`nuer2`+`nuepma`) and the ν–ν part ($\nu\nu$ and $\nu\bar{\nu}$), whose sum reproduces it on a physical state (each part is clamped at 0 separately); `collision="matrix_nue"` adds only the ν–ν part as a scalar rate.

6.3 The grid-only kernels: `nunu` and `nunubar`

These two depend only on the ε-grid (no m_e, $T_{\rm cm}/T_{\rm pl}$, or ϕ_e), so they are precomputed once into static tensors and never rebuilt.

```
build_nunu_kernel(grid, n_gl=5, ordpolint=5) -> NuNuKernel # host precompute
nunu_rates(kernel, occ, n_chunks=None, fd_cap_logC=inf, fd_decay=False, monotone_read=False) ->
  ↪  (net, frs) # (3,2,N) each

build_nunubar_kernel(grid, n_gl=5, ordpolint=5) -> NuNuBarKernel
nunubar_rates(kernel, occ, n_chunks=None, fd_cap_logC=inf, fd_decay=False, monotone_read=False)
    -> (net_s, frs_s, net_a, frs_a) # scatter + annihilation channels

nunu_dfdt(kernel, occ, t_cm_mev, n_chunks=None) # T_cm^5 * net: ungated df/dt [s^-1]
nunubar_dfdt(kernel, occ, t_cm_mev, n_chunks=None) # T_cm^5 * (net_s + net_a)
```

`NuNuKernel` carries the static W, $\mathrm{idx}_3/\mathrm{lw}_3/\mathrm{idx}_4/\mathrm{lw}_4$, and the flavour factor (1 on the diagonal, $\frac{1}{2}$ off). `nunu` has $K = 15$ slots (three p_3 segments $\times 5$ Gauss nodes); `nunubar` has $K = 10$ (two segments) and yields both a scattering and a flavour-changing-annihilation channel from one weight tensor, with the $\bar{\nu}$ partner read at the opposite cp. Neither has an $e^{\pm}$ partner, so neither contributes to `leg4`. The `*_dfdt` helpers return one kernel's ungated $\mathrm{d}f/\mathrm{d}t$, for tests and diagnostics.

`n_chunks` bounds the peak memory of the $\mathcal{O}(N^2)$ partner-bin gather: `1` is the verbatim path (bit-faithful), `None` auto-picks (small $N \to 1$), `>1` sums the partner-bin axis in blocks via `scan`. Performance only; bit-identical at `n_chunks=1`.

Demo: the equilibrium sum rule. At equilibrium every net rate must vanish to the noise floor. This is the cheapest correctness check and needs no Fortran run:

```
import neutt, numpy as np
from neutt.grid import make_eps_grid
from neutt.transport.assemble import equilibrium_occ
from neutt.transport.nunu import build_nunu_kernel, nunu_rates
g = make_eps_grid()
f0 = equilibrium_occ(g) # FD equilibrium
K = build_nunu_kernel(g)
net, frs = nunu_rates(K, f0)
print(float(np.max(np.abs(net)))) # ~1e-9 or smaller: net -> 0 at equilibrium
```

6.4 The lepton kernels: `nuer1`, `nuer2`, `nuepma`

These carry the energy exchange with the plasma, so they depend on the background through meps $= m_e/T_{\rm cm}$, tcmpl $= T_{\rm cm}/T_{\rm pl}$, and ϕ_e. They are not static tensors but jittable functions:

```
nuer1_rates(grid, occ, meps, tcmpl, phie, n_gl=5, n_lag=5, fd_cap_logC=inf, fd_decay=False,
  ↪ monotone_read=False, lag_tail_fix=True, lag_tail_gap=6.0) -> (net, frs)
nuer2_rates(...same keywords...) -> (net, frs)
nuepma_rates(...same keywords...) -> (net, frs)
nuer1_dfdt(grid, occ, meps, tcmpl, phie, t_cm_mev, **kw) # T_cm^5 * net; likewise nuer2_dfdt,
  ↪ nuepma_dfdt
```

Here $p_1 = \varepsilon_i$ is the output bin and E_2, E_3 the incoming and outgoing $e^{\pm}$ energies in units of $T_{\rm cm}$ (for `nuepma`, the e^- and e^+ energies $E_{\rm out}$, $E_{\rm in}$); `n_gl` and `n_lag` are the Gauss–Legendre and Gauss–Laguerre node counts per tile (BURST's 5). The double integral over these energies is organized as a fixed table of tiles (E_2/E_3 sub-intervals using Legendre on finite stretches and Gauss–Laguerre on $[a,\infty)$ tails) and a small number of regimes that become runtime masks: `nuer1` has 2 regimes (split at $p_1 = $ meps$/2$), `nuer2` and `nuepma` have 4, R1–R4 (split at the `pcase` energies, the $e^{\pm}$-mass thresholds meps$(\sqrt{5}-1)/4$, meps$/(2\sqrt{2})$, meps$/2$ for `nuer2` and meps$/2$, meps$(1+\sqrt{5})/4$, meps for `nuepma`). The off-grid leg $p_4 = p_1 + E_2 - E_3$ ($p_4 = E_{\rm out} + E_{\rm in} - p_1$ for `nuepma`) is read by the dynamic interpolator (Section 5.3), because its node positions move with the background. The coupling weights carry $\sin^2\theta_W$ and the ν_e-vs-$\nu_{\mu,\tau}$ charged-current factor.

The quadrature fix (`lag_tail_fix`, default on). Fortran BURST has a latent quadrature bug wherever an energy cut diverges: it applies a fixed 5-point Gauss–Legendre rule over a near-infinite interval and mis-resolves the exponentially decaying integrand. In `nuer1` and `nuepma` the cut diverges as a grid momentum approaches a regime boundary, so the per-bin rate is wrong by 2–4× in a narrow window and the kernel is discontinuous at the boundary (by $\sim 80\%$ of peak). In `nuer2` it diverges inside regime R1 (the lowest-p_1 regime, not the R1 reduction) as $p_1 \to 0$: with the kinematic cuts ecut1 $= p_1 + $ meps$^2/(4p_1)$ and ecut2 $= p_1 + $ meps$(p_1 + $ meps$)/(2p_1 + $ meps$)$, R1's outer segment $[$ecut2, ecut1$]$ (`trans_nuer2.f90`) has width $\approx$ meps$^2/(4p_1)$, tens of $T_{\rm cm}$ at the lowest bins once meps $\gtrsim 5$ ($T_{\rm cm} \lesssim 0.1$ MeV), and there the rate is off by 23% at meps $= 7$, rising to $\sim 100\%$ at meps $= 20$ (against a converged 40×40-node reference). The port reproduces the Fortran to $\le 2.5 \times 10^{-12}$ per bin on the `nuer2` oracle, including those bins at meps $= 12.7$ and 33.9, so the defect is BURST's.

The port fixes all three, and the fix is the default (`lag_tail_fix=True` in every function that takes the argument; the solvers that do not expose it, the QKE solver and the LINX adapter's, run the kernels at that default): the wide bins switch to a Gauss–Laguerre tail. In `nuer1` and `nuepma` it is the rule BURST already uses on the other side of the boundary. In `nuer2`, R1 bins with ecut1 $-$ ecut2 $\ge 6\,T_{\rm cm}$ integrate $E_3 \in [$ecut2$,\infty)$ with one Laguerre rule in place of

BURST's Legendre [ecut2, ecut1] plus Laguerre [ecut1, ∞) pair, with the pair's own E_2 tiles: below ecut1 the E_2 integral ends at its kinematic upper limit `elim1`, above it `elim1` has diverged and the tail $[E_3, \infty)$ applies. The `nuer1` meps-sweep discontinuity collapses ($\sim 0.81 \to 0.025$), the lowest `nuer2` bins come within 0.4–6% of the converged rate (the accuracy of BURST's 5-point rules at those bins), detailed balance is preserved exactly, and the scheduled kernels reproduce the reference ones to 10^{-16}. The end-to-end $N_{\rm eff}$ impact is $\leq 8.4 \times 10^{-7}$ for `nuer1/nuepma` and below 10^{-9} for `nuer2` (the `rtol=1e-8` pins agree to nine digits with and without it): local-correctness fixes that are $N_{\rm eff}$-neutral. `lag_tail_fix=False` recovers the bit-faithful BURST form, and is used only by the Fortran-parity oracle tests.

Quadrature accuracy. Even with the fix, BURST's 5-point rules leave a per-bin error of a few per cent at the lowest bins below $T_{\rm cm} \approx 0.25$ MeV, where the $e^{\pm}$ are Boltzmann-suppressed, and $\sim 10^{-4}$ (energy-weighted) above. Raising the order to $(n_{\rm gl}, n_{\rm lag}) = (16, 10)$ moves the default $N_{\rm eff}$ by $+1.3 \times 10^{-8}$ in `nuer2` and $+3.9 \times 10^{-6}$ in `nuepma`, so the lepton-kernel quadrature is a $\sim 4 \times 10^{-6}$ systematic on $N_{\rm eff}$, dominated by the annihilation kernel. The pins (Section 1.7) are at BURST's order. The same rule makes `nuepma`'s loss operator slightly negative at its lowest bin below $T_{\rm cm} \approx 0.1$ MeV (on rates that are numerically zero there).

[**Port note**] At the exact regime boundary the Fortran crashes (a zero-width `cscgqf` rescale); the port instead gives a zero-weight tile and stays finite (values and gradients), so it is strictly more robust there. That exact coincidence is measure-zero (the continuous solve never samples it) and is the one residual the `lag_tail_fix` does not address (it does not need to).

6.5 The scheduled (`_sched`) variants, and what `use_sched` is

`use_sched` is a performance switch, not a physics one: it selects how the lepton kernels are evaluated, changing speed and bit-reproducibility but never the physics. "Never the physics" is not the same as "bit-identical", and the two scheduled modes differ on exactly that point; see the bit-reproducibility note below. Each of the three lepton kernels splits a bin into kinematic regimes: depending on where the bin energy ε_i sits relative to the $e^{\pm}$ mass thresholds (the `p case` energies), the energy integral is segmented differently (`nuer2` and `nuepma` have 4 such regimes, `nuer1` has 2). A given bin belongs to exactly one regime. But the **plain** kernels, to stay control-flow-free under `jit`/`vmap`, evaluate all regimes for every bin and then mask away the $\sim 3/4$ (or $\sim 1/2$) that do not apply, so each bin redundantly does several times the necessary work, and the launch is dominated by that waste.

The **scheduled** kernels (`nuer1_rates_sched`, `nuer2_rates_sched`, `nuepma_rates_sched`, each in the same module as its reference kernel) remove the redundancy. An integer schedule table records, for each bin, which regime's quadrature tiles it needs; from there the kernel can either gather just those active tiles (`select=True`: one batched contraction over the schedule) or rebuild the same masked union of all regimes without a gather (`select=False`). "Scheduled" refers to that table. Either way it is a pure-XLA reformulation that is ~6–17× cheaper than the plain kernels.

[**Port note**] **Bit-reproducibility caveat: the two scheduled modes are not equally faithful.** This is the one place `use_sched` can move the last digits of $N_{\rm eff}$, and it is easy to over-state as "bit-identical".

- `select=False` (the `"union"` mode) evaluates the same tiles the plain kernel does and merely skips the redundant launches, so it reproduces the reference to $\leq 10^{-15}$, bit-identical for all practical purposes.
- `select=True` (the `True` mode) gathers a per-bin subset, which re-associates the gate sum by more than a pure round-off reordering. It agrees with the reference only to $\sim$few $\times 10^{-7}$ in $N_{\rm eff}$ (measured 6.7×10^{-7} versus union in the production configuration), not to the last bit. This is below every physics threshold in the port, but it is not zero, and it is why the QKE solver defaults to `"union"`.

A user normally never sets `use_sched`; it is exposed so the bit-identical reference (`False`) and the bit-identical union path are available for cross-checks.

```
nuer1_rates_sched(grid, occ, meps, tcmpl, phie, select=True, ...) -> (net, frs)
# select=True : per-bin regime GATHER (the fast path, ~6-17x)
# select=False : regime-masked UNION (no gather; reproduces the plain kernel exactly)
```

The three values of `use_sched`:

`use_sched`	kernel path	role
`False`	plain `nuer*_rates`	bit-faithful reference (slowest)
`True`	`*_rates_sched(select=True)`	fast per-bin gather; faithful to $\sim 6.7 \times 10^{-7}$ in $N_{\rm eff}$, not bit-identical (classical default)
`"union"`	`*_rates_sched(select=False)`	regime-masked batch, no gather; bit-identical ($\leq 10^{-15}$) to the reference (the QKE default)

Both scheduled modes perturb the gate sum relative to the plain kernel: `"union"` only at round-off ($\leq 10^{-15}$), `True` more ($\sim 10^{-7}$, per the caveat above), enough that on the adaptive (Tsit5) solvers a full solve must cap its maximum step `dtmax` (in $\ln a$) at ≤ 0.10 (the solver defaults handle this). The fixed-step QKE `lax.scan` has no adaptive controller to cascade, so it is immune to that hazard and is free to pick whichever mode is more faithful rather than whichever survives the controller.

Defaults. The user-facing solvers default to `True` (select): `make_decoupling_solver`, `make_hybrid_solver`, and the LINX adapter's `make_transport_solver` (Section 9.5). The QKE solver (Section 13) defaults to `"union"`. The bare low-level builders (`assemble_rhs`, `make_rhs`, `make_split_rates`) default to `False` (the reference). In `make_coupled_bbn`, `use_sched=None` (the default) resolves per transport mode: `True` for full / semi, `"union"` for `transport="qke"`. The QKE choice of `"union"` over `True` is deliberate, not a fallback: that solve is collision-bound (the lepton kernels are not its bottleneck), so the 1.18× that the gather buys is negligible, while `"union"` keeps the kernel bit-identical to the reference; it is the fast and bit-faithful path, and the fixed-step scan removes the dtmax hazard that would otherwise argue for `True`.

6.6 Conservation sum rules

BURST monitors number, energy, and ν–$\bar{\nu}$ (CP) symmetry through five "sum rules" (`sumrule1`=number $\int \varepsilon^2$ net and `sumrule2`=energy $\int \varepsilon^3$ net of the ν-only processes, `sumrule3`–`5`= the ν–$\bar{\nu}$ symmetry of the annihilation channels; each is returned as the ratio of its gated net to its gated frs sum). The port reproduces them in `assemble.compute_sumrules` (each as net/frs over the gated bins, with the raw `sumrule1net`, `sumrule1frs`, ...), and `make_decoupling_solver(sumrules=True)` recomputes them post-hoc at the saved states (zero impact on

the solve). These are the free, first-class correctness target of the port: the ν-only processes conserve neutrino number and energy, so `sumrule1` and `sumrule2` vanish to quadrature error at any spectrum, and at equilibrium every net rate vanishes. (Near equilibrium the forward and reverse rates cancel; the precision gate of Section 3.4 zeroes the bins where that cancellation leaves only noise.)

Demo: per-mode rates at a non-equilibrium spectrum.

```
import neutt, numpy as np
from neutt.grid import make_eps_grid
from neutt.transport.assemble import (equilibrium_occ, compute_prec_trans,
                                      assemble_rhs, compute_sumrules)
from neutt.transport.nunu import build_nunu_kernel
from neutt.transport.nunubar import build_nunubar_kernel
g = make_eps_grid()
Kn = build_nunu_kernel(g); Kb = build_nunubar_kernel(g)
prec = compute_prec_trans(g, Kn, Kb)
f = equilibrium_occ(g) * 1.0
f = f.at[0, 0].add(0.01 * np.exp(-(np.asarray(g.abscissas) - 5.0)**2)) # a bump in nu_e
docc, drhonu = assemble_rhs(g, f, 2.0, 2.0, 0.0, prec, Kn, Kb)
print(float(np.max(np.abs(docc))), float(drhonu)) # nonzero: the bump relaxes
sr = compute_sumrules(g, f, 2.0, 2.0, 0.0, prec, Kn, Kb) # dict of sumrule1 ... sumrule5
print(float(sr["sumrule1"]), float(sr["sumrule2"])) # number / energy monitors
```

7 The coupled decoupling solver

This is the headline of the port: evolve the plasma temperature $T_{\rm pl}$ together with the per-mode neutrino spectrum $f[3,2,N]$ through $e^{\pm}$ annihilation, so $N_{\rm eff}$ emerges self-consistently from the spectral distortions. The state is the PyTree $\{T_{\rm pl},\ f\}$; the integration variable is $\ln a$, with $T_{\rm cm} = T_{\rm cm}^{\rm init} e^{-\ln a}$. The module is `neutt/decoupling.py`.

[**Fortran ↔ JAX**] The temperature ODE is faithful to `bbn_yes_evolve_v2.f90`; the collision RHS to `trans_evolve.f90` (through `transport.assemble`).

7.1 The system of equations

It is worth writing the evolved system out in full, since it is rarely seen in one place. The state is $\{T_{\rm pl},\ f_i(\varepsilon_a)\}$, one plasma temperature plus the occupation of every neutrino mode $i \in \{\text{flavour}\} \times \{\text{cp}\}$ (6 modes) on every grid bin ε_a, and the independent variable is $s \equiv \ln a$, with the comoving clock $T_{\rm cm}(s) = T_{\rm cm}^{\rm init} e^{-s}$ fixed analytically.

(1) The neutrino transport. For each mode i and bin ε_a,

$$\frac{\mathrm{d}f_i(\varepsilon_a)}{\mathrm{d}s} = \frac{1}{H}\, C_i(\varepsilon_a), \qquad C_i(\varepsilon_a) = T_{\rm cm}^5\ \mathrm{net}_i(\varepsilon_a)\ \Theta_{\rm gate}, \tag{11}$$

where C_i is the collision rate $[\mathrm{s}^{-1}]$ summed over the five weak processes (Section 2.2). After the analytic angular reduction each process contributes (illustrated for one process)

$$\mathrm{net}_i(\varepsilon_a) = \mathrm{prefac} \sum_b \mathrm{kinem}(a,b) \sum_{p_3} w(a,b,p_3)\, F\big(f_1, f_2, f_3, f_4\big), \tag{12}$$

with prefac and kinem as in Eq. (6), $w(a,b,p_3) = \mathrm{glw}\cdot \mathrm{w}_3$ the combined Gauss and leg-3 quadrature weight, $f_1 = f_i(\varepsilon_a)$, f_2 the partner occupation at ε_b, $f_3 = f(p_3)$, $f_4 = f(\varepsilon_a+\varepsilon_b-p_3)$ (the off-grid leg), and the Pauli-blocked gain−loss factor

$$F = \mathrm{flafact}\Big[(1-f_1)(1-f_2)f_3f_4 - f_1f_2(1-f_3)(1-f_4)\Big] \equiv \mathrm{forward} - \mathrm{reverse}. \tag{13}$$

Here flafact is the per-process flavour/spin statistical weight (the combinatorial factor counting the contributing ν and $e^{\pm}$ states). Θ_{gate} is the precision gate (Section 3.4); it exists because near equilibrium forward and reverse nearly cancel, so a bin's net rate can be dominated by round-off. It is 1 where $|\sum \mathrm{net}|/(\sum \mathrm{frs}\cdot\mathrm{prec}) \geq \mathrm{nuratiotol} = 3$ and 0 otherwise, with frs = forward+reverse the cancellation-free throughput and prec the per-bin equilibrium-noise floor. Equation (11) is exactly the gated collision RHS, and it can be written in the affine form $\mathrm{d}f/\mathrm{d}s = A - Bf$ used by the hybrid head (Section 3.6).

(2) The plasma temperature. The decoupled $\mu = 0$ $e^{\pm}/\gamma$ plasma conserves comoving entropy except for the energy it leaks to the neutrinos, which gives

$$\frac{\mathrm{d}T_{\mathrm{pl}}}{\mathrm{d}s} = \frac{-3\,(\rho_{\mathrm{pl}} + P_{\mathrm{pl}}) - \dfrac{1}{H}\dfrac{\mathrm{d}\rho_\nu}{\mathrm{d}t}\Big|_{\mathrm{exch}}}{\mathrm{d}\rho_{\mathrm{pl}}/\mathrm{d}T_{\mathrm{pl}}}, \qquad \frac{\mathrm{d}\rho_\nu}{\mathrm{d}t}\Big|_{\mathrm{exch}} = \frac{T_{\mathrm{cm}}^9}{2\pi^2}\sum_{i,a} w_a\,\varepsilon_a^3\,\big(\text{gated leg4}\big)_i, \tag{14}$$

where the energy-exchange rate $\mathrm{d}\rho_\nu/\mathrm{d}t|_{\mathrm{exch}}$ collects only the three ν–$e^{\pm}$ channels (`leg4`, BURST's $e^{\pm}$ partner slot), the ones that move energy across the ν/plasma boundary. The $-3(\rho_{\mathrm{pl}} + P_{\mathrm{pl}})$ term is adiabatic dilution; the second term is the extra cooling because the plasma is feeding the distorting neutrinos. $\mathrm{d}\rho_{\mathrm{pl}}/\mathrm{d}T_{\mathrm{pl}} \equiv C_{\mathrm{pl}}$ is the plasma heat capacity.

(3) Closure: the Friedmann equation and the constitutive relations.

$$H = \frac{1}{\hbar}\sqrt{\frac{8\pi}{3M_{\mathrm{Pl}}^2}\left(\rho_{\mathrm{pl}} + \rho_\nu + \rho_{\mathrm{dr}}\right)}, \qquad \rho_\nu = \frac{T_{\mathrm{cm}}^4}{2\pi^2}\sum_i \int_0^{\varepsilon_{\max}} \varepsilon^3 f_i(\varepsilon)\,\mathrm{d}\varepsilon,$$

$$\rho_{\mathrm{pl}} = \rho_\gamma + \rho_{e^\pm} + \delta\rho_{\mathrm{QED}}, \qquad \rho_\gamma = \frac{\pi^2}{15}T_{\mathrm{pl}}^4, \qquad \rho_{\mathrm{dr}} = \Delta N_{\mathrm{eff}}\left(\tfrac{4}{11}\right)^{4/3}\frac{7}{8}\frac{\pi^2}{15}T_{\mathrm{cm}}^4. \tag{15}$$

M_{Pl} is the Planck mass (`MPL`), $\rho_{e^\pm}$ the Gauss–Laguerre free-gas integral (Section 5.4), $\delta\rho_{\mathrm{QED}}$ the finite-T QED correction (Section 5.4.3), and ρ_{dr} the optional inert dark radiation, with ΔN_{eff} here the input `delta_neff_init` (Section 7.5). The headline observable is the late-time $N_{\mathrm{eff}} = \frac{8}{7}\left(\frac{11}{4}\right)^{4/3}\rho_\nu/\rho_\gamma$.

The QKE generalization. The flavour-mixing solver (Section 13) replaces the diagonal $f_i(\varepsilon)$ with the 3×3 density matrix $\rho(\varepsilon)$ per cp and evolves

$$\frac{\mathrm{d}\rho(\varepsilon)}{\mathrm{d}s} = \frac{1}{H}\Big(-i\,[\,H_{\mathrm{flav}}(\varepsilon), \rho(\varepsilon)\,] + C[\rho]\Big), \tag{16}$$

with the flavour Hamiltonian H_{flav} of Section 13 and the collision $C[\rho]$ the matrix generalization of the gated C_i of (11) (its kernels the matrix form of (12)), to which it reduces on a diagonal ρ.

7.2 `make_decoupling_solver`

The reference transport solve: one adaptive `diffrax` solve of the whole window. Like every solver it is a factory that builds the kernels, the noise floor, and the equilibrium initial condition once and returns a jitted `solve` that closes over them (Section 10).

Signature.

```
make_decoupling_solver(grid=None, *, n_save=64, max_steps=50_000,
                       pcoeff=0.4, icoeff=0.3, dcoeff=0.0,
                       solver=None, implicit=False,
                       nuratiotol=3.0, use_renorm=True, use_sched=True,
                       renorm_pdep=True, qed_nlo=True, tcm_cutoff=0.015, phie=0.0,
                       eta0=None, q_baryon=0.5, xi=(0.0,0.0,0.0), sumrules=False,
                       throw=True, lag_tail_fix=True) -> (solve, grid)
# solve(T_init=10.0, lna_max=7.0, rtol=1e-6, atol=1e-30, dtmax=None,
# delta_neff_init=0.0, me=None, xi=None) -> dict
```

The builder's `xi` is the build-time degeneracy (it sets the baked noise floor and initial spectrum), so a scan over ξ uses `solve(xi=...)` instead.

Build-time arguments.

- `n_save`: number of saved trajectory states.
- `implicit`: `True` uses L-stable Kvaerno5 (an alternative for a stiff start); `False` (default) uses Tsit5, which with the `atol` $= 10^{-30}$ default already completes the 10 MeV start (it is stability-limited but does not blow up; the hybrid is the faster cure).
- `use_renorm`/`renorm_pdep`: include the finite-T QED EoS correction (default `True`=BURST-faithful; `False` reproduces the decoupling-only $N_{\rm eff} = 3.034029$).
- `use_sched`: lepton-kernel path (Section 6.5; default `True`).
- `tcm_cutoff`: transport-window low edge (0.015 MeV; below it the $\propto T_{\rm cm}^5$ collision rates are negligible, so they are masked to zero and the spectrum is frozen).
- `sumrules`: also return the conservation monitors.
- `grid`: the ε-grid (default `make_eps_grid()`); `max_steps`, `pcoeff`/`icoeff`/`dcoeff`: the diffrax step budget and PID gains (Section 3.5); `solver`: an explicit diffrax solver, overriding `implicit`; `throw`: raise on a failed solve.
- `nuratiotol`, `qed_nlo`, `lag_tail_fix`: the gate threshold (Eq. (8)), the NLO QED term (Section 5.4.4), and the lepton-kernel tail quadrature (Section 6.4).
- `phie`, `eta0`, `q_baryon`: the electron degeneracy, a fixed float or `"auto"` for charge neutrality at the present baryon-to-photon ratio `eta0` (`None` is `thermo.ETA0_DEFAULT`) and net charge per baryon `q_baryon` (Section 5.4).

Per-call arguments (traced).

- `T_init`: the start temperature $T_{\rm pl} = T_{\rm cm}$ [MeV]. The default 10 is the top of BURST's transport window; neutrinos are tightly coupled there ($T_{\rm pl}/T_{\rm cm} = 1$).
- `lna_max`: integrate $\ln a \in [0, \texttt{lna_max}]$; ~ 7 lets $e^{\pm}$ annihilation finish so the $(11/4)^{4/3}$ formula is valid.
- `rtol`, `atol`, `dtmax`: PID tolerances and step cap (10^{-6}, 10^{-30}; `dtmax=None` is 0.08 with `use_sched`, 0.15 on the reference path; Section 10).
- `delta_neff_init`, `me`, `xi`: the inert dark radiation, the electron mass [MeV] (`None` is `M_ELECTRON` as a compile-time constant), and a per-call degeneracy (`None` keeps the build-time floor and initial spectrum); see Sections 7.5 and 10.

Returned dict.

```
out["lna"], out["T_pl"], out["T_cm"], out["ratio"] # background trajectory
out["f_final"] # the distorted final spectrum (3,2,N)
out["f_traj"] # full trajectory (n_save,3,2,N)
out["neff_final"] # N_eff from the distorted final spectrum
out["ratio_final"] # T_pl/T_cm at the end
out["sol"] # the raw diffrax solution
out["sumrules"] # only if sumrules=True
```

`lna` holds the `n_save` saved $\ln a$ nodes, uniform on $[0, \texttt{lna_max}]$, and `ratio`$= T_{\rm pl}/T_{\rm cm}$ at each.

7.3 The RHS builders: `make_rhs` and `make_split_rates`

`make_decoupling_solver` assembles its vector field from `make_rhs`; the hybrid head uses the affine `make_split_rates`. Both close over the grid, the kernels, and the noise floor, and take the same physics knobs.

```
make_rhs(grid, prec_trans, nunu_kernel, nunubar_kernel, T_cm_init,
        nuratiotol=3.0, use_renorm=True, use_sched=False, renorm_pdep=True,
        qed_nlo=True, tcm_cutoff=0.015, phie=0.0, ..., delta_neff_init=0.0,
        me=M_ELECTRON, lag_tail_fix=True)
    -> rhs(lna, state, args) -> {"T_pl": dT_pl, "f": df} # per d(ln a)

make_split_rates(grid, prec_trans, nunu_kernel, nunubar_kernel, T_cm_init, ...)
    -> split(lna, T_pl, occ) -> (A, B, dT_pl, H) # A - B*occ == make_rhs's df
```

Here `T_cm_init` is $T_{\rm cm}$ at $\ln a = 0$ (the solvers pass `T_init`). The temperature derivative is $\mathrm{d}T_{\rm pl}/\mathrm{d}\ln a = \big[-3(\rho_{\rm pl}+P_{\rm pl}) - \dot\rho_\nu/H\big]/C_{\rm pl}$, with $\dot\rho_\nu \equiv \mathrm{d}\rho_\nu/\mathrm{d}t|_{\rm exch}$ (`drhonudt`, Eq. (14)) and the plasma heat capacity $C_{\rm pl} = \mathrm{d}\rho_{\rm pl}/\mathrm{d}T$ (distinct from the collision integral C_{ν_i} of Section 2.2), and the Hubble rate $H = \mathrm{hubble}(\rho_{\rm pl} + \rho_\nu + \rho_{\rm dr})$ includes the inert dark-radiation term $\rho_{\rm dr} =$ `delta_neff_init` $\cdot\,(4/11)^{4/3} \cdot \frac{7}{8}\frac{\pi^2}{15} T_{\rm cm}^4$ (LINX's `Delt_Neff_init` normalization; the $(4/11)^{4/3}$ factor lives in `decoupling._DR_DNEFF_FACTOR`). The plasma EoS triple $(\rho_{\rm pl}, P_{\rm pl}, C_{\rm pl})$ carries the QED δ-corrections when `use_renorm` is on.

[**Fortran ↔ JAX**] The QED correction is BURST's additive $\delta\rho/\delta P$, not a free-gas mass shift: the lepton-kernel `meps` stays the bare $m_e/T_{\rm cm}$ (`meps = xmelec/tcmev`, snapshot `trans_evolve.f90:97`; no thermal $m_{\rm eff}$ correction is applied to the kernel mass). A varied `me` varies that bare mass coherently throughout the RHS (kernels, free-gas EoS, QED δ-terms, and the `phie="auto"` charge-neutrality solve).

7.4 The solver menu

path	warm/call	when
hybrid (default)	∼2.8 s	production standalone transport
plain Tsit5 (`hybrid=False`)	∼6.5 s	reference / debugging; transport starts below ∼6 MeV (where the stiff head the hybrid feeds on is mostly gone)
implicit Kvaerno5 (`implicit=True`)	slower	an L-stable alternative for a stiff start (the explicit default already completes the 10 MeV start, just slowly)

The `hybrid`/`implicit` selectors are the `solver_kwargs` keys on the coupled-BBN path (Section 9); standalone, the hybrid is built with `make_hybrid_solver` (Section 8) while `make_decoupling_solver` is the plain Tsit5 path (Kvaerno5 with `implicit=True`). For why the hybrid is the production default and what its one blind spot is, see Section 8.

7.5 Standalone demos

The pin values (NLO / LO / decoupling-only).

```
from neutt.decoupling import make_decoupling_solver
print(float(make_decoupling_solver()[0]()["neff_final"])) # 3.043322 (NLO QED, default)
print(float(make_decoupling_solver(qed_nlo=False)[0]()["neff_final"])) # 3.044276 (LO; 3.044225
  ↪  at SIN2THW_BURST = BURST-faithful)
print(float(make_decoupling_solver(use_renorm=False)[0]()["neff_final"])) # 3.034029 (no finite
  ↪ -T QED)
```

Each line builds and compiles its own solver; in a scan, build once and reuse the `solve`.

The spectral distortion $\delta f(\varepsilon)$. Distortions are $f/f^{\rm eq} - 1$ on the frozen comoving grid (the equilibrium reference is the same FD array at every epoch):

```
import numpy as np
from neutt.grid import make_eps_grid
from neutt.transport.assemble import equilibrium_occ
from neutt.hybrid import make_hybrid_solver
solve, grid = make_hybrid_solver()
out = solve()
eps = np.asarray(grid.abscissas)
dist = np.asarray(out["f_final"]) / np.asarray(equilibrium_occ(grid)) - 1.0
# dist[0,0] = nu_e fractional heating vs eps (positive, grows with eps,
# percent-level by eps ~10); dist[1],dist[2] = the smaller mu/tau distortions
np.savez("spectra.npz", eps=eps, w=np.asarray(grid.weights),
        lna=out["lna"], T_cm=out["T_cm"], T_pl=out["T_pl"], f_traj=out["f_traj"])
```

[**Fortran ↔ JAX**] `dist[0,0]` is the comoving-grid version of BURST's `nu_72.dat` ν_e distortion. The trajectory `out["f_traj"]` (shape $(\texttt{n_save}, 3, 2, N)$) at the nodes `out["lna"]` reproduces, in memory, what BURST streams to its `nu_7x` units over the run.

Inert dark radiation ($\Delta N_{\rm eff}$). In the standalone solver the knob is `delta_neff_init`; it is traced, so no recompile:

```
solve, grid = make_hybrid_solver()
for dn in (0.0, 0.1, 0.2):
    print(dn, float(solve(delta_neff_init=dn)["neff_final"]))
```

[**Port note**] **One convention for extra radiation.** `delta_neff_init` carries LINX's `Delt_Neff_init` normalization both standalone and coupled: $\rho_{\rm dr} = \texttt{delta_neff_init} \cdot (4/11)^{4/3} \cdot \frac{7}{8}\frac{\pi^2}{15} T_{\rm cm}^4$, so one unit is one fermionic degree of freedom (dof) at the transport start ($\Delta N_{\rm eff}^{\rm init} = 1$). BURST's bare `rhors` knob *omits* the $(4/11)^{4/3}$ factor, so the same number there is $(11/4)^{4/3} \approx 3.85\times$ more radiation; neutt folds that factor into `delta_neff_init` (`decoupling._DR_DNEFF_FACTOR`), so standalone and coupled use the identical convention and the LINX coupling passes `delta_neff_init` straight through with no conversion (Section 9).

A varied electron mass. `me` is traced and reaches the kernels' `meps`, the free-gas EoS, and the QED δ-terms coherently:

```
solve, grid = make_hybrid_solver()
print(float(solve()["neff_final"]),
     float(solve(me=0.6)["neff_final"])) # dN_eff/dme ~+0.124 / MeV
```

A lepton asymmetry (ξ). $\xi = (\xi_e, \xi_\mu, \xi_\tau)$ sets both the degenerate FD initial condition and the matching noise floor. The builder's `xi` bakes them in; `solve(xi=...)` recomputes both inside the compiled solve, so a scan over ξ does not recompile:

```
solve, grid = make_hybrid_solver()
print(float(solve(xi=(0.05, 0.0, 0.0))["neff_final"])) # degenerate-FD shift; watch out["
  ↪ qualified"]
```

[**Port note**] $\xi \neq 0$ is validated end to end at one point, $\xi = (0.05, 0, 0)$ at LO QED: the hybrid matches an `rtol=1e-8` arbiter to 7×10^{-7} in $N_{\rm eff}$, and $\Delta N_{\rm eff}$ matches the degenerate-FD prediction to 0.8%. The test suite pins no $\xi \neq 0$ solve (its only $\nu/\bar{\nu}$-asymmetric case is one spectrum in the `nuepma` kernel oracle), so run an `rtol=1e-8` arbiter comparison at any other ξ before production use (and in coupled runs use full mode; the semi path's plasma side cannot see the asymmetry).

Conservation telemetry.

```
solve, grid = make_decoupling_solver(sumrules=True)
out = solve()
# out["sumrules"] is a dict of sumrule1 ... sumrule5, each an array along the trajectory:
print(np.asarray(out["sumrules"]["sumrule1"]).shape) # (n_save,) number monitor
```

7.6 Outputs and I/O: getting at the spectra

The port writes no files: every data product is returned in memory, unlike BURST, which streams to numbered Fortran units. Knowing where each distribution lives is therefore an I/O question in itself.

What the transport solvers return. The `solve` closures from `make_decoupling_solver` (Section 7.2) and `make_hybrid_solver` (Section 8) return dicts that share the same user-facing fields (the extras differ: the decoupling `solve` adds the raw `sol`; the hybrid `solve` adds the qualification diagnostics — `qualified`, `qss_dist`, `head_*`, `tail_*`; Section 8). The fields a user reads:

- `f_final` $[3, 2, N]$: the distorted final occupation $f_{\nu_i}(\varepsilon)$ (flavour × cp × bin) on the frozen comoving grid;
- `f_traj` $[$`n_save`$, 3, 2, N]$: the trajectory at the saved $\ln a$ nodes `out["lna"]`, i.e. the in-memory analogue of BURST's `nu_72.dat` … `nu_75.dat` streams;
- `T_cm`, `T_pl`, `ratio`: the background clocks at each node; `neff_final`, `ratio_final`, the end-point scalars.

The grid abscissas and Boole weights come from the returned `grid` (`grid.abscissas`, `grid.weights`); they are what turn an occupation into a moment, a density, or a distortion.

Recovering the spectral distortion. The ε-grid is frozen comoving, so the equilibrium reference is the same Fermi–Dirac array at every epoch and $\delta f = f/f^{\rm eq} - 1$ is a single elementwise division; the demo in Section 7.5 shows this and saves the result to an `.npz` with `numpy.savez`. Densities and $N_{\rm eff}$ from any saved `f` go through the `grid` moments (Section 5.2.4) and `neff_from_spectrum` (Section 2.3).

Coupled-run outputs. In the LINX coupling the products are different: `run` returns `(abundance_tuple, N_eff)`, with Y_P = `res[5]` $\cdot$ 4 and D/H = `res[2]/res[1]` (Section 9); the distorted ν_e spectrum is reachable through `f_e_occupation` (Section 9.5). As above, persist with `numpy.savez`; there is no built-in file output to configure.

8 The exponential-midpoint hybrid solver

This is the production transport solver, in `neutt/hybrid.py`. The numerical idea is sketched in Section 3.6; here is the API and the operating envelope.

8.1 `make_hybrid_solver`

```
make_hybrid_solver(grid=None, *, lna_sw=1.5, n_head=1600, head_power=1.0,
                   n_save_head=128, n_save_tail=128,
                   use_sched=True, use_renorm=True, renorm_pdep=True, qed_nlo=True,
                   nuratiotol=None, tcm_cutoff=0.015, phie=0.0,
                   eta0=None, q_baryon=0.5, xi=(0.0,0.0,0.0), lag_tail_fix=True,
                   max_steps=50_000, pcoeff=0.4, icoeff=0.3, dcoeff=0.0,
                   throw=True) -> (solve, grid)
```

`throw=False` returns a failed tail solve's partial result with `tail_ok` False instead of raising. The kernels, the noise floor, and the equilibrium IC are built once at construction; `solve` closes over them. Arguments:

- `lna_sw`: the head→tail handoff in $\ln a$ (static). Default 1.5 means "hand off at $T_{\rm cm} = 0.22\,$`T_init`", tuned for a 10 MeV start (handoff at 2.2 MeV). Keep $\lesssim 1.5$; a later handoff puts the state too far off the QSS attractor and stalls the tail.
- `n_head`: number of fixed exponential-midpoint head steps (static). 1600 uniform is the validated configuration.
- `head_power`: grading exponent for the head step edges; 1 = uniform. Do not use $\neq 1$: graded heads have measured failure pockets and are parameter-fragile (the qualification catches them, but each failed call pays full-solve price).
- `use_sched`, `use_renorm`, `renorm_pdep`, `qed_nlo`, `nuratiotol` (`None` is BURST's 3.0), `tcm_cutoff`, `phie`, `eta0`, `q_baryon`, `lag_tail_fix` (Section 7.2): forwarded identically to the head (`make_split_rates`) and the tail (`make_rhs`), so both halves are the same physics.
- `xi`: the build-time default degeneracy; it sets the degenerate-FD initial occupation and the matching `prec_trans` noise floor once at construction (it is not a `make_split_rates`/`make_rhs` argument, but head and tail both close over `prec_trans` and start from the same IC, so it reaches them consistently). `solve` also takes a traced per-call `xi` that overrides this with no recompile (`xi=None` reuses the baked value bit-for-bit); $\xi \neq 0$ shifts the QSS attractor (watch `qualified`).
- `n_save_head`, `n_save_tail` (default 128 each): saved trajectory nodes in the head $[0, $`lna_sw`$)$ and tail $[$`lna_sw`$,$ `lna_max`$]$; the stitched `f_traj` has `n_save_head`+`n_save_tail` rows.
- `max_steps`, `pcoeff`, `icoeff`, `dcoeff`: the tail's diffrax step budget and PID gains (the production PI$(0.4, 0.3)$).

The head's building blocks are public (the QKE collision substep uses `exp_step` too):

```
phi1(z) # (e^z - 1)/z, series 1 + z/2 for |z| ≤1e-8
exp_step(f, a, b, h) # f + h*phi1(-b*h)*(a - b*f) = Eq. (expstep), exact as b -> 0
head_edges(lna_sw, n_head, power=1.0) # (n_head+1,) static numpy step edges on [0, lna_sw]
```

The midpoint head calls `exp_step` twice per step: a half step to the midpoint, then a full step with (A, B) evaluated there.

8.2 The returned `solve`

```
solve(T_init=10.0, lna_max=7.5, rtol=1e-6, atol=1e-30, dtmax=0.08,
      delta_neff_init=0.0, qual_tol=1e-4, me=None, xi=None) -> dict
```

`me=None` compiles the bare mass (`me` = `M_ELECTRON`) in as a constant, the graph the canonical pins use; a passed `me` is traced. `delta_neff_init`, `qual_tol`, and a per-call `xi` are traced (no recompile). The dict has the base keys `lna`, `T_pl`, `T_cm`, `ratio`, `f_traj`, `f_final`, `ratio_final`, `neff_final` (it does not carry the decoupling `solve`'s raw `sol`) plus the hybrid diagnostics:

```
out["qualified"] # bool: False means the full-window Tsit5 fallback ran
out["qss_dist"] # gated max|f - A/B| at the handoff
out["head_T_ok"], out["head_f_finite"], out["head_n_neg"], out["head_n_super"]
out["tail_steps"], out["tail_rejected"], out["tail_ok"]
```

The `head_*` entries check the handoff state ($T_{\rm pl}$ finite and positive, every f finite, and the numbers of bins with $f < 0$ and $f > 1$); `tail_steps`, `tail_rejected` and `tail_ok` are the tail's accepted and rejected step counts and its diffrax success flag.

8.3 Handoff qualification and silent fallback

At the handoff $\ln a$ = `lna_sw` the solve checks that the spectrum is on the quasi-static-equilibrium attractor and otherwise sane ($T_{\rm pl}$ finite and positive, every f finite and in $[0, 1]$):

$$\text{qualified} = \text{sane} \ \wedge \ \big(\text{qss_dist} \le \texttt{qual_tol}\big), \qquad \text{qss_dist} = \max_{\text{gated bins}} \big|f_{\rm sw} - A_{\rm sw}/B_{\rm sw}\big|.$$

where the subscript sw marks values at the handoff. If the check fails, the tail `diffeqsolve` (the diffrax integration call) is fed a traced start time and state t_0, y_0 that restarts it from $\ln a = 0$ at the equilibrium IC, so qualified and fallback share one compiled executable and the safe path is always available without a recompile. The final state and $N_{\rm eff}$ are exact in both modes; a mis-qualified configuration costs only the extra head work. The default `qual_tol`$= 10^{-4}$ gives $\sim 2\times$ margin at the `lna_sw` $= 1.5$ handoff (gated max $|f - A/B| = 5.6 \times 10^{-5}$ there) while rejecting every handoff later than $\ln a \approx 1.7$.

8.4 Operating envelope, which solver, when

The hybrid's win is the stiff $10 \to 2.2$ MeV head. Its one blind spot is that the head is order-2 and its qualification polices the spectrum, not accumulated $T_{\rm pl}$/energy-budget error, so accuracy-critical physics deposited in the head can slip through. The asymmetry that should drive the choice: picking the hybrid wrongly costs speed (or, for exotic high-T physics, a bias a one-time pin would catch); but the genuinely dangerous wrong choice is the semi coupling (Section 9), which silently drops the $\nu \leftrightarrow$plasma energy exchange.

scenario	recommendation
SM baseline / BBN-only params (baryon-to-photon ratio η, neutron lifetime τ_n, nuclear rates)	cached transport history (`cached_transport_history`, Section 9.5); transport is constant, solve once
SM + inert dark radiation ($\Delta N_{\rm eff}$ only)	full default (hybrid)
BSM with all dynamics below ~ 2 MeV	full default; the head is SM-equilibrium there
energy injection above $\sim$ 2 MeV, or a freeze-out/resonance/decay at 3–10 MeV	`hybrid=False` (the head's blind spot)
transport start below ~ 6 MeV	`hybrid=False`; the stiff head is mostly gone
any new model family, first time	run the family pin: solve at default and at `qual_tol=-1, rtol=1e-8` (`qual_tol=-1` forces the full adaptive fallback), require $\lvert\Delta N_{\rm eff}\rvert \leq 2\times10^{-6}$, then scan with whichever passed
debugging / bit-level cross-checks	`hybrid=False, use_sched=False, rtol=1e-8` (the bit-faithful reference)

Watch per-call wall time in scans: a hybrid call that suddenly slows to the plain-solve time (standalone $\sim$2.8 s $\to$ $\sim$6.5 s; coupled $\sim$4.2 s $\to$ $\sim$8 s) is the fallback firing (the answer is still right). If a whole parameter region does it, switch that scan to `hybrid=False`.

8.5 Validation headline

Default (1600 uniform head steps, `lna_sw=1.5`): the hybrid's $N_{\rm eff}$ sits -6.9×10^{-8} from the `rtol=1e-8` arbiter, comfortably inside the pre-registered bar ($\lvert\Delta N_{\rm eff}\rvert \leq 2\times10^{-6}$ and $\max\lvert\Delta f_{\rm final}\rvert \leq 1\times10^{-5}$), at 2.36$\times$ the speed of the plain transport solve (2.78 s vs. 6.54 s warm). The `neut` test suite passes; a harness that rebuilds the executable for each run finds an executable-to-executable $N_{\rm eff}$ spread of 7.4×10^{-7}; and all 5 LINX coupling pins are reproduced.

[**Port note**] The hybrid is a port-only construction with no BURST counterpart (BURST uses one adaptive RK integrator throughout). It is mathematically exact for the linear collision diagonal and falls back to the faithful adaptive solve whenever its premises do not hold, so it adds speed without changing the physics.

9 Coupling to LINX for abundances

The transport solver produces $N_{\rm eff}$ and the distorted spectra; to get the light-element abundances it is coupled to **LINX**, the differentiable BBN code. The integration lives in the LINX source tree, in three files: `linx/coupled_bbn.py` (the one-call driver `make_coupled_bbn`) and `linx/transport_adapter.py` (the transport↔LINX seam, imported as `ta`), plus `linx/weak_rates_transport.py` (`TransportWeakRates`, the $n \leftrightarrow p$ rates on the distorted f_e).

[**Port note**] **This entire coupling is a port-only addition.** Fortran BURST has its own internal nuclear network and computes $Y_{\rm P}$, D/H itself; the JAX port instead ports only the neutrino transport and hands the distorted spectrum to LINX's abundance model. Consequently the JAX+LINX path also has no recombination / sound-horizon (`sdratio`) stage; its $N_{\rm eff}$ is the late-time frozen value from transport, not a recombination-era observable. See Appendix A.

9.1 make_coupled_bbn

```
make_coupled_bbn(nuclear_net="key_recommended", use_fe=True, n_save=256,
                 use_sched=None, gate_width=0.0, solver_kwargs=None,
                 device_split=True, cpu=None, gpu=None,
                 transport="full", xi=(0.0,0.0,0.0), grid=None,
                 qke_kwargs=None, solver=None, **bg_kwargs) -> (run, info)
```

Key arguments:

- `nuclear_net`: LINX reaction network (default `"key_recommended"`: the key rates of the 2023 compilation of the PRIMAT BBN code, with an updated $n + p \to d + \gamma$ rate). The choice affects the abundances ($Y_{\rm P}$, D/H) but not $N_{\rm eff}$ (which is a transport result, network-independent).
- `use_fe` (default `True`): use the distorted-f_e weak rates (`TransportWeakRates`) rather than equilibrium ones. This is what threads the transport-distorted ν_e spectrum into the $n \leftrightarrow p$ rates.
- `transport` $\in$ {`"full"`, `"semi"`, `"qke"`}: the coupling mode (Section 9.3; `"qke"` is the flavour-mixing solver of Section 13; its builder options go in `qke_kwargs`, whose `lna_max`, default 7.5, goes to the solve).
- `solver`: a convenience front-end that selects the transport solver per mode without reaching into `solver_kwargs`: `"full"` accepts `hybrid` (default) / `tsit5` / `implicit`; `"semi"` only `tsit5`; `"qke"` `hybrid` (default) / `expm` / `strang`. Passing both `solver` and the matching low-level key is an error.
- `use_sched`: transport kernel path (Section 6.5). Default `None` resolves to each mode's validated default (`select` for full/semi, `"union"` for qke).
- `solver_kwargs`: forwarded to the transport builder in full and semi modes (in full mode a key that is neither a build key nor `rtol/atol/dtmax/lna_max` raises; qke mode takes its options in `qke_kwargs` and rejects `solver_kwargs`). Notably `solver_kwargs={"hybrid": False}` selects plain Tsit5 transport (neutt's `make_decoupling_solver`, ~ 8 s), and `{"lna_max": 6.6}` matches the end temperature for a lower transport start.
- `device_split` (default `True`): run LINX on CPU and the transport solve on GPU, in one process. (LINX's `BackgroundModel` compiles prohibitively slowly on GPU.) `False` runs everything on one device; with no visible GPU the split switches off by itself.
- `grid`: the transport ε-grid (every mode), fixed at build time (never vary per call; it would retrace).
- `**bg_kwargs`: forwarded to LINX `BackgroundModel`. **Pass `max_steps=8192` here**: the LINX background default (512) is marginal for the NLO plasma EoS and can fail the coupled run with a "maximum number of steps" error. This is the background ceiling; the transport solver's own `max_steps` is set separately via `solver_kwargs`.

`info` is `dict(cpu, gpu, grid, device_split)`. Everything parameter-independent (kernels, noise floor, LINX models) is built once; `run` is serialized-MCMC safe (physics scalars are traced, nothing recompiles per call).

9.2 The run closure

```
run(delta_neff_init=0.0, T_anchor=4.0, T_init=10.0, xi=None, return_diag=False,
    **am_kwargs) -> (abundance_tuple, N_eff)
```

- `delta_neff_init`: the public inert-dark-radiation input (LINX's `Delt_Neff_init` convention; see below).
- `T_init`: where the full and qke solves start (10 MeV).
- `T_anchor`: where the semi spectrum solve starts (4 MeV).

- `xi`: optional per-call neutrino-degeneracy override (Section 10); `None` reuses the build-time value, a passed triple is traced (no recompile).
- `**am_kwargs`: forwarded to LINX's `AbundanceModel`: the traced physics scalars `eta_fac` (baryon-to-photon rescale), `tau_n_fac` (neutron-lifetime rescale), `nuclear_rates_q` (per-reaction log-normal draws), `me` (electron mass, threaded coherently to the LINX background, the weak rates, the transport and the swapped `Neff_vec`; LINX's `AbundanceModel` builds its own $t(T)$ and $a(T)$ from the EM energy density at `const.me`, so a varied `me` does not reach the network's clock), plus non-traced controls like `rtol`, `sampling_nTOp`.
- `return_diag=True` adds a third return value, a CPU dict with the transport solve's own `neff_final`.

The abundance tuple is LINX's `AbundanceModel` output; the standard extractions are $Y_{\rm P}$ = `res[5]` $\cdot 4$ and D/H = `res[2]/res[1]`. `N_eff` is the late-time value at the lowest (frozen) T_γ of the swapped background, LINX's background with its neutrino sector replaced by the transport history (`ta.swap_neutrino_sector`).

9.3 Full vs. semi coupling

mode	what it does	warm	ν–plasma feedback?
`"full"` (default)	per-call full coupled transport: $T_{\rm pl}$ co-evolved with the spectrum, finite-T QED (with the momentum-dependent `renorm_pdep` term) in the RHS, 10 MeV start, BURST's own topology. $\nu \leftrightarrow$plasma energy exchange feeds $T_{\rm pl}/H$. The only mode correct for energy-exchanging physics.	∼4.2 s	yes
`"semi"`	SM-grade fast path: spectrum-only solve driven by LINX's plasma history from a 4 MeV anchor. **Discards** the $\nu \leftrightarrow$plasma exchange ($\dot\rho_\nu$). (Still produces the SM distortion and accepts inert `delta_neff_init` through LINX's background.)	∼2.4 s	no

[**Port note**] **Semi is a silent physics error if misused.** Full mode reproduces BURST's coupling topology; semi is a port-only convenience that drops $\dot\rho_\nu$. It is admissible only for SM-grade work (or SM + inert dark radiation) at $\sim 10^{-4}$ $N_{\rm eff}$ tolerance. Never use it with any sector that exchanges energy with the neutrinos. When in doubt, stay on full; nothing warns you if semi is wrong.

9.4 The $\Delta N_{\rm eff}$ convention

The single input `delta_neff_init` uses LINX's `Delt_Neff_init` meaning: $\rho_{\rm DR}$ = `delta_neff_init` $\cdot \frac{7}{8}(\frac{4}{11})^{4/3}\rho_\gamma$ at the start, redshifting as a^{-4}. The transport carries the *same* normalization natively: neutt folds the $(4/11)^{4/3}$ factor into its dark-radiation term (`decoupling._DR_DNEFF_FACTOR`), so $\rho_{\rm dr}$ = `delta_neff_init` $\cdot (4/11)^{4/3} \cdot \frac{7}{8}\frac{\pi^2}{15}T_{\rm cm}^4$ equals LINX's $\rho_{\rm DR}$ and the glue passes `delta_neff_init` straight through with no conversion at the seam. (BURST's own `rhors` omits the $(4/11)^{4/3}$ factor, so the same number there is $\approx 3.85\times$ more radiation.) The hybrid stays qualified at least to `delta_neff_init` = 0.3.

9.5 The transport↔LINX seam (`transport_adapter`)

These are the functions that map the comoving-grid spectrum onto LINX's T_γ clock and feed the distorted f_e into the weak rates. The most important for a user:

```
make_transport_solver(grid=None, n_save=256, use_sched=True, use_renorm=True,
                      qed_nlo=True, implicit=False, ..., hybrid=None, # None: not implicit
                      ...) -> (solve, grid) # build-once transport
cached_transport_history(grid=None, T_init=10.0, lna_max=7.5, ...) -> history
make_spectrum_solver(grid=None, n_save=256, use_sched=True, ..., gate_width=0.0,
                     xi=(0.0,0.0,0.0)) -> (solve_spectrum, grid) # the semi-mode solver
build_history_from_full_coupled(full_out, grid) -> history # the qke-mode history
build_history_from_transport(transport_out, grid) -> history
f_e_occupation(E_nu_MeV, T_g, history, cp=0) # distorted nu_e occupation
T_cm_of_Tg(T_g_query, history) # comoving temp on the T_g clock
r_nu_of_Tg(T_g_query, history) # rho_nu_tot/rho_gamma on the T_g clock
```

- `make_transport_solver` is the production build-once transport for serial MCMC; with `hybrid=True` (default) it delegates to `make_hybrid_solver`, with `hybrid=False` to `make_decoupling_solver`.
- `cached_transport_history` is the SM entry point: transport is constant across an SM MCMC, so solve once and reuse the history for every abundance call (zero per-call transport cost).
- `f_e_occupation` is the convenience wrapper (slice-once over T_γ then evaluate over ε); `TransportWeakRates` actually calls the two split primitives it composes (`f_e_slice_at_Tg` once per T_γ and `f_e_eval_slice` per momentum point) for compile efficiency. In the undistorted limit it returns exactly $1/(e^{E_\nu/T_\gamma}+1)$, LINX's thermal factor, which is the correctness anchor. `cp`: $0=\nu_e$, $1=\bar{\nu}_e$.
- `r_nu_of_Tg` / `T_cm_of_Tg` interpolate clamped into the transport T_γ range, so above the transport start (ν in equilibrium, r_ν flat) and below $e^\pm$ annihilation (r_ν frozen) the fills are physically exact, no spurious slope through the BBN epoch.

The rest of the public seam, which `make_coupled_bbn` composes, and the weak-rate class:

```
make_history(T_g_hist, r_nu_hist, *, ratio_hist=None, f_traj=None, eps=None,
             weights=None) -> history # the history dict, sorted ascending in T_g
linx_plasma_nodes(bg_output, T_anchor=4.0, me=None)
    -> (lna_nodes, Tg_nodes, H_nodes, lna0, lna1, Tcm_anchor) # semi-mode drive, from LINX's
      ↪ background
swap_neutrino_sector(bg_output, history, me=const.me) -> bg_output # LINX 7-tuple, transport
  ↪ rho_nu, N_eff
rho_nu_vec_on_grid(T_g_vec, rho_g_vec, history) # per-species rho_nu = r_nu*rho_g/3 [MeV^4]
neff_asymptotic(history) # (8/7)(11/4)^{4/3} r_nu at the lowest T_g
neff_vec_on_grid(T_g_vec, history) # N_eff(T_g) along LINX's grid
f_e_slice_at_Tg(T_g, history, cp=0) -> (N,) # f_e(eps) at one T_g, cubic in log T_g
f_e_eval_slice(E_nu_MeV, T_cm, f_slice, eps_grid) # the slice at E_nu, cubic in eps, 0 past the
  ↪  grid

# linx/weak_rates_transport.py
TransportWeakRates(nu_history, RC_corr=True, FM_corr=True, weak_mag_corr=True, thermal_corr=
  ↪ True)
```

`make_history` is what the two `build_history_*` functions call, and what a stored trajectory (a saved file, a test fixture) goes through. `bg_output` is the 7-tuple returned by LINX's `BackgroundModel`; `swap_neutrino_sector` keeps its electromagnetic sector and replaces `rho_nu_vec` and `Neff_vec` from the history. `TransportWeakRates` subclasses LINX's `WeakRates` and replaces only the Born neutrino factor by the transport f_{ν_e} or $f_{\bar{\nu}_e}$ (the channel's neutral lepton is a ν when `sgnq`$\cdot e_\nu > 0$ and a $\bar{\nu}$ otherwise), which the radiative correction multiplies as in LINX; the finite-mass correction and LINX's tabulated thermal corrections keep the thermal

neutrino. `make_coupled_bbn(use_fe=True)` builds it from each history.

9.6 Coupled-BBN demos

SM baseline.

```
import neutt
from linx.coupled_bbn import make_coupled_bbn
run, info = make_coupled_bbn(max_steps=8192)
res, neff = run(delta_neff_init=0.0)
print(neff, float(res[5]*4.0), float(res[2]/res[1])) # N_eff 3.043322; Yp/D-H depend on
  ↪ nuclear_net
```

A dark-radiation point.

```
res, neff = run(delta_neff_init=0.2) # traced -> no recompile
print(neff, float(res[5]*4.0))
```

Varying BBN-only parameters. `eta_fac`, `tau_n_fac` and `nuclear_rates_q` are traced, so a chain over them never recompiles; for SM-grade work the semi transport is the faster choice:

```
run, info = make_coupled_bbn(transport="semi", max_steps=8192) # SM-grade, fast
for ef in (0.98, 1.0, 1.02):
    res, neff = run(delta_neff_init=0.0, eta_fac=ef)
    print(ef, float(res[5]*4.0))
```

Plain-Tsit5 transport (reference path).

```
run, info = make_coupled_bbn(max_steps=8192, solver_kwargs={"hybrid": False}) # ~8 s warm
res, neff = run(delta_neff_init=0.0)
```

A varied electron mass, end to end.

```
res0, n0 = run(delta_neff_init=0.0)
res1, n1 = run(delta_neff_init=0.0, me=0.6) # reaches bg + weak rates + transport coherently
print(n0, n1, float(res0[5]*4.0), float(res1[5]*4.0)) # dYp/dme ~+0.227 / MeV
```

10 Parameters, tolerances, and knobs

10.1 Traced vs. build-time

A parameter is **traced** if it can change per call with zero recompile (MCMC-safe), or **build-time** if changing it rebuilds and recompiles the solver (~40–80 s). The dividing line is whether the parameter changes the shape of the computation (build-time) or only its numeric inputs (traced).

Every solver is a factory, `make_*_solver(...) -> (solve, grid)`, whose arguments are build-time and whose `solve` is an ordinary `jax.jit` function: build the solver once and call `solve` repeatedly. `solve`'s arguments are traced, so a new value reuses the compiled executable. `jax.jit` compiles one variant per call form, so the first call that passes its arguments differently (a keyword not passed before, a positional instead of a keyword, a Python number instead of a float64 array) compiles once more, and later calls of that form reuse it. In particular `me=None` and `xi=None` select build-time constants (`M_ELECTRON`; the builder's degeneracy), and the first call with a value compiles one traced variant. The neutt banner (`neutt.compiling()`, called first in each jitted `solve`) prints on every compile; a call that prints nothing ran a cached executable.

Coupled-BBN `run(...)` arguments (traced, except the abundance-ODE controls).

parameter	meaning	notes
`delta_neff_init`	extra inert dark radiation	LINX's `Delt_Neff_init` normalization; same convention standalone and coupled — no glue conversion (Section 9.4)
`eta_fac`	baryon-to-photon rescale	1 = fiducial
`tau_n_fac`	neutron-lifetime rescale	1 = fiducial
`nuclear_rates_q`	nuclear-rate draws	$q \sim \mathcal{N}(0,1)$ per reaction
`me`	electron mass [MeV]	reaches bg + weak rates + transport coherently; unset keeps transport on `M_ELECTRON`
`T_init`	transport start [MeV]	full and qke modes
`T_anchor`	spectrum anchor [MeV]	semi mode only (default 4)
`xi`	per-flavour ν degeneracy $(\xi_e, \xi_\mu, \xi_\tau)$	`run(xi=...)`/`solve(xi=...)` is traced: `None` (default) reuses the build-time value bit-for-bit, a passed triple recomputes the initial Fermi–Dirac spectrum and the noise floor inside the compiled solve with no recompile. Only `full`/`qke` carry the asymmetry into H and the $n \leftrightarrow p$ rates (the `semi` plasma is symmetric). Validated end to end at $\xi_e = 0.05$ (LO QED; matches an `rtol=1e-8` arbiter to 7×10^{-7} and the degenerate-FD prediction to 0.8%), with no pinned test (Section 7.5); run a tight-tolerance arbiter for any other or production ξ
`rtol`, `atol`, `sampling_nTOp`, ...	abundance-ODE controls	forwarded to LINX `AbundanceModel` (its own defaults `rtol`= 10^{-6}, `atol`= 10^{-9}); these are the abundance-ODE tolerances, not the transport `atol`= 10^{-30} below

Build-time arguments.

parameter	where	meaning
`nuclear_net`	`make_coupled_bbn`	LINX reaction network (default `key_recommended`: PRIMAT-2023 + updated $n + p \to d + \gamma$)
`use_fe`	`make_coupled_bbn`	distorted f_e in the $n \leftrightarrow p$ rates (default True)
`transport`	`make_coupled_bbn`	`"full"`/`"semi"`/`"qke"` (Section 9.3)

parameter	where	meaning
`solver_kwargs`	`make_coupled_bbn`	dict forwarded to the transport builder: `max_steps/implicit/pcoeff/icoeff/dcoeff/hybrid/hybrid_kwargs/use_renorm/renorm_pdep/tcm_cutoff/nuratiotol/phie/eta0/q_baryon/qed_nlo` (build-time; in full mode any other key raises; qke mode takes `qke_kwargs` instead) and `rtol/atol/dtmax/lna_max` (per-solve). The only way to set the transport tolerances on the coupled path
`gate_width`	`make_coupled_bbn`	precision-gate smoothing width on the semi path (default 0 = BURST hard gate)
`n_save`	`make_coupled_bbn`	saved output-trajectory nodes (default 256)
`use_sched`	all builders	lepton-kernel path (Section 6.5)
`qed_nlo`	transport builders / `make_coupled_bbn`	NLO $\mathcal{O}(e^3)$ QED (Section 5.4.4); default on in every builder (the low-level `thermo` QED functions default to `nlo=False`), `False` recovers the LO BURST value; in `make_coupled_bbn` tied to LINX's `NLO`
`use_renorm`, `renorm_pdep`	transport builders	finite-T LO QED in the EoS (default on = BURST-faithful)
`lag_tail_fix`	lepton kernels, `make_rhs`/`make_split_rates`/`make_decoupling_solver`/`make_hybrid_solver`	corrected wide-regime quadrature (default True; Section 6.4); False recovers the buggy bit-faithful BURST form
`phie`	transport builders	electron degeneracy $\mu_e/T_{\rm pl}$ (default 0 = symmetric; a fixed float, or `"auto"` for the charge-neutrality $\phi(T_{\rm pl})$, Section 12)
`tcm_cutoff`	transport builders	transport-window low edge (BURST 0.015 MeV)
`nuratiotol`	transport builders	precision-gate threshold (BURST 3.0)
`lna_sw`, `n_head`, `head_power`	`make_hybrid_solver`	hybrid head geometry
`xi`	all builders	per-flavour ν degeneracy $(\xi_e, \xi_\mu, \xi_\tau)$; this sets the default, and the per-call `run(xi=...)`/`solve(xi=...)` override is traced (no recompile, see the traced table above)
`nbins`, `eps_range`	`make_eps_grid`	grid resolution/extent (101, 25 = BURST's)

Electron degeneracy `phie` vs. neutrino degeneracy `xi`: independent. The electron degeneracy $\phi_e = \mu_e/T_{\rm pl}$ and the neutrino degeneracy ξ are different physical quantities and are not derived from one another. $\xi = (\xi_e, \xi_\mu, \xi_\tau)$ is the neutrino lepton asymmetry, a free input you choose. ϕ_e is instead fixed by charge neutrality with the baryons, $n_{e^-} - n_{e^+} = q_{\rm baryon}\, n_b$, i.e. it is a function of the baryon-to-photon ratio η, of $T_{\rm pl}$, and of m_e, not of ξ; it is never inferred from the neutrino sector. Their overrides are therefore separate:

- `xi`: default symmetric $\xi = 0$; set the factory default at build time, or override per call with `run(xi=...)` / `solve(xi=...)` (traced, no recompile).
- `phie`: `0.0` (symmetric, default), a fixed float, or `"auto"` for the time-dependent charge-neutrality $\phi(T_{\rm pl})$ solved inside the transport RHS (Section 12); its $N_{\rm eff}$ effect is $\ll 10^{-7}$.

Both reach the Hubble rate and the $n \leftrightarrow p$ rates only in `full`/`qke` mode (the `semi` path's external plasma is symmetric $e^{\pm}$).

10.2 Tolerances and step control (change with care)

These are the transport-solve tolerances. The standalone solvers (the `solve` of `make_decoupling_solver` and of `make_hybrid_solver`) take them directly; on the coupled path they are set through `make_coupled_bbn(solver_kwargs={...})`. The defaults are the tuned, validated state. Each non-obvious choice:

- `rtol`$= 10^{-6}$ (transport). Counterintuitively $\sim 2\times$ faster than 10^{-4}, which runs rejection-bound at the stability edge. Its $N_{\rm eff}$ differs from the 10^{-8} arbiter by $+5.7 \times 10^{-7}$. Tightening to 10^{-8} costs only $\sim 20\%$ (arbiter-grade); loosening buys nothing.
- `atol`$= 10^{-30}$ = pure-relative error norm, BURST-faithful. Never raise it. At 10^{-7} the stiff $\bar{\nu}$-tail bins go error-uncontrolled $\rightarrow$ executable-dependent stalls and a silent $+1.5 \times 10^{-3}$ $N_{\rm eff}$ bias (Section 12).
- `dtmax` $= 0.08$ with `use_sched` (default), 0.15 on the reference path (`make_decoupling_solver` only; the hybrid and coupled-path solves default to 0.08). The scheduled kernels reorder the gate sum at round-off and cascade Tsit5 at 0.15. Do not raise.
- `pcoeff,icoeff,dcoeff`$= (0.4, 0.3, 0.0)$: a damped PI law. A pure-I controller oscillates at the explicit stability limit; the PI damping changes only the step-adaptation, never the enforced tolerance.
- `use_renorm`+`renorm_pdep`= True: finite-T QED in the EoS (matches the collaborator oracle to 5.5×10^{-7}). Turn it off only for the decoupling-only configuration ($N_{\rm eff} = 3.034$, Section 11.2).
- **No QKE coherence tolerance.** The above are the diagonal transport-ODE tolerances. The QKE coherent (off-diagonal) substep is an exact matrix exponential (Section 13.3) with no per-component `rtol`/`atol`; there is nothing trace-scaled to set on the real or imaginary coherences. The only QKE-specific numerical control is `n_steps`, the number of operator-split macro-steps, and (orthogonally) the grid size N; the result is converged in both (Section 13). A step too coarse for the plasma-temperature update ($\mathrm{d}\ln a \gtrsim 0.03$ over the full window, i.e. `n_steps`$\lesssim 250$) drives $T_{\rm pl}$ unstable; the QKE solver then raises (it checks that $T_{\rm pl}$ never exceeds `T_init` and the final $T_{\rm pl}/T_{\rm cm}$ lies in $[0.99, 1.45]$) instead of returning a meaningless $N_{\rm eff}$.

10.3 Starting transport at a lower temperature

The transport start is decoupled from the BBN start: above the transport window the history is clamped to equilibrium ($r_\nu = \rho_{\nu,\rm tot}/\rho_\gamma$ constant, $T_{\rm pl}/T_{\rm cm} = 1$), which is exactly the physics there, so LINX's abundance evolution still begins at its own high $T_{\rm start}$. To start transport at, e.g., 4 MeV: `run(T_init=4.0)` (traced, no recompile). Rules of thumb:

- **Floor $\approx$ 3–4 MeV.** The equilibrium initial condition is only valid while weak interactions are still fast (ν decoupling $\sim$ 2–3 MeV); below that you erase real distortion physics.
- **Shrink `lna_max` to keep the same end temperature**: $T_{\rm end} =$ `T_init` $e^{-\texttt{lna_max}}$. Unlike the traced `T_init`, `lna_max` is a build-time solver knob: pass it via `make_coupled_bbn(solver_kwargs={"lna_max":6.6})` (one recompile) together with `run(T_init=4.0)`. The production end of 5.5×10^{-3} MeV corresponds to the hybrid/coupled default `lna_max` $= 7.5$ ($10\,e^{-7.5}$); note `make_decoupling_solver`'s `solve` defaults to `lna_max` $=$ 7.0 (ending slightly warmer, $\sim 9 \times 10^{-3}$ MeV), so matching 5.5×10^{-3} from 4 MeV needs `lna_max` ≈ 6.6.
- **Hybrid caveat:** `lna_sw` is e-folds from the start, so its 1.5 default means "hand off at

$T_{\rm cm} = 0.22\,$`T_init`". Below `T_init` ≈ 6 MeV just use `hybrid=False`; the stiff head the hybrid feeds on is mostly gone, and a mis-set `lna_sw` only costs the fallback price anyway.

11 Validation and testing

11.1 The validation philosophy

The port is gated at three levels (the BURST paper's "checksums" raised to a first-class target):

1. **Built-in sum rules (free).** At equilibrium every net rate vanishes to the noise floor and the number/energy moments ($\int \varepsilon^2$ net, $\int \varepsilon^3$ net) are zero. These validate each kernel standalone with no Fortran run (Section 6.6).
2. **Module-level diffs against the Fortran oracle.** The `fortran_reference/` snapshot compiles and runs, so each JAX module is checked against its Fortran counterpart at the intermediate level (a single $\mathrm{d}f/\mathrm{d}t$ at a fixed background, δm_e^2 from `renorm`, $e^{\pm}$ ρ/P at a given T), localizing any discrepancy to the module that introduced it.
3. **End-to-end pins.** $N_{\rm eff}^{\rm SM} = 3.0433222$ (NLO, the default), 3.0442761 (LO, `qed_nlo=False`; 3.0442252 at `SIN2THW_BURST`), and 3.034029 (decoupling-only); the coupled $N_{\rm eff} = 3.0433222$ (network-independent, equal to the transport's); $Y_{\rm P} \approx 0.2470$, D/H $\approx 2.44 \times 10^{-5}$ depend on `nuclear_net`.

11.2 Canonical numbers

quantity	value	configuration
$N_{\rm eff}$ (full, NLO, default)	3.0433222	NLO QED, `use_renorm`+`renorm_pdep`, 10 MeV
$N_{\rm eff}$ (full, LO)	3.0442761	`qed_nlo=False`
$N_{\rm eff}$ (full, LO, BURST-faithful)	3.0442252	`qed_nlo=False` at `SIN2THW_BURST`$= 0.23$
$N_{\rm eff}$ (decoupling-only)	3.034029†	`use_renorm=False`
$N_{\rm eff}$ (momentum-indep. QED, LO)	3.044313†	`renorm_pdep=False`, `qed_nlo=False`
$T_{\rm pl}/T_{\rm cm}$ asymptote (background only)	1.401019	`evolve_background`, vs. $(11/4)^{1/3} = 1.401020$
$N_{\rm eff}$ (background only)	3.000007	`evolve_background`
coupled $N_{\rm eff}$ (NLO)	3.0433222	`make_coupled_bbn(solver_kwargs={"hybrid": False, "rtol": 1e-8})`
coupled $Y_{\rm P}$, D/H	≈ 0.2470, $\approx 2.44\times10^{-5}$	network-dependent (`key_recommended`)

All rows use $\sin^2\theta_W = 0.23122$ (`SIN2THW`) except the BURST-faithful one, and come from the `rtol=1e-8` arbiter (`scripts_qke/regen_pins_weinberg.py`). The production angle raises every $N_{\rm eff}$ here by $+5.1 \times 10^{-5}$ relative to BURST's 0.23 (`SIN2THW_BURST`). The script sets XLA's deterministic GPU flags, under which every pin is bitwise reproducible; with default autotuning the LO pin can land up to 3×10^{-7} higher. †At $\sin^2\theta_W = 0.23$ these two configurations give 3.033977 and 3.044262.

11.3 Running the test suite

```
cd $BURST_ROOT && PYTHONPATH=$PWD \
  srun -n1 -c32 --gpus-per-task=1 \
  python -m pytest tests/ -q # default deselects slow-marked tests (pyproject addopts)
```

The suite includes the foundation tests (grid/quadrature/moments vs. analytic) and the per-kernel oracle comparisons (`test_nunu.py`, `test_nunubar.py`, `test_nuer1.py`, `test_nuer2.py`, `test_nuepma.py`, each diffed against the Fortran goldens in the matching `tests/oracle_*` directory), plus the equilibrium sum rules. Three caveats: (i) the default invocation deselects `slow`-marked tests, including the lepton-kernel `meps`-sweep robustness test, which you run with `pytest tests/ -m slow`; (ii) `tests/` also contains the QKE suites that a bare `pytest tests/` will collect, so the total count is not a fixed SM number; (iii) the end-to-end $N_{\rm eff}$ pins (3.0433222 NLO / 3.0442761 LO, coupled 3.0433222; Section 11.2) are reference values from a tight-tolerance arbiter run (the `rtol=1e-8` reference of Section 10), not part of the default pytest suite.

> The first `pytest` invocation triggers a long JAX compile (∼10 min) before any test runs; this is not a hang.

12 Caveats and safeguards

This section is self-contained; each item is described in full here. Each caveat below is a numerical failure mode with its symptom and the safeguard the default configuration applies, so a reader who sees the symptom, or turns the safeguard off, knows what is happening.

Lepton-kernel quadrature at a regime boundary (`lag_tail_fix`, default on). Fortran BURST mis-resolves bins where an energy cut diverges and a fixed 5-point Gauss–Legendre rule is applied over a near-infinite interval: the wide-regime bins of `nuer1` and `nuepma` near a kinematic regime boundary (the per-bin rate wrong by 2–4× in a narrow window, the kernel discontinuous at the boundary), and the lowest `nuer2` bins once $T_{\rm cm} \lesssim 0.1$ MeV (off by 20–100%). By default (`lag_tail_fix=True`, Section 6.4) the port integrates those bins with a Gauss–Laguerre tail rule instead; the end-to-end $N_{\rm eff}$ shift is $\leq 8.4 \times 10^{-7}$ (below 10^{-9} from `nuer2`). `lag_tail_fix=False` gives BURST's (inaccurate) bit-faithful form, used only by the Fortran-parity oracle tests. The one case the tail rule does not cover is the measure-zero exact boundary coincidence, where the Fortran crashes and the port instead returns a finite zero-weight tile, never sampled by the continuous solve.

Constant-folding of the precision-gate floor. The equilibrium noise floor is input-independent, so under `jit` XLA would evaluate it through its constant-folding path, which uses different floating-point arithmetic and loses the 10^{-9} cancellation that defines the floor; the gate would then invert and $N_{\rm eff}$ collapse by ~ 0.03 (in the decoupling-only configuration, $3.034 \to 3.004$). The port prevents this with an `optimization_barrier` on the equilibrium occupation (Section 3.4), a runtime identity.

The `atol` floor. Raising `atol` toward 10^{-7} leaves the stiff $\bar{\nu}$-tail bins error-uncontrolled, giving executable-dependent `max_steps` stalls and a silent $+1.5 \times 10^{-3}$ $N_{\rm eff}$ bias. The default `atol` $= 10^{-30}$ (a pure-relative norm) keeps every bin error-controlled. Do not raise it (Section 10).

NaN error estimates in the step controller. A NaN error estimate slips past diffrax's controller guards and produces $\mathrm{d}t = \mathrm{NaN}$, a permanent rejection cascade (a quasi-random `max_steps` stall). The port's error norm, `nanrobust_rms_norm`, maps the error norm's NaN to $+\infty$ so the library's designed reject-and-shrink path handles it (bit-identical on every finite step). It is the controller norm of every adaptive transport solve (`make_decoupling_solver`, the hybrid

tail, and the semi-mode `make_spectrum_solver`); the reference helper `thermo.evolve_background` and LINX's own background and abundance solves use diffrax's default norm.

12.1 Features in BURST not present in the port

For the SM/$\Delta N_{\rm eff}$ scope:

- **Sterile neutrinos** ($n_\nu > 3$). The SM transport is hardwired to 3 flavours × 2 cp.
- **The nuclear reaction network** and **recombination / `sdratio`**. Not ported; abundances come from LINX, and there is no recombination-era $N_{\rm eff}$ (Section 9).
- **The ϕ_e degeneracy: trajectory ported, closure not.** The default is $\phi_e = 0$ (symmetric $e^\pm$). The time-dependent charge-neutrality electron degeneracy is available as `phie="auto"`: it solves $\phi(T_{\rm pl})$ from charge neutrality $n_{e^-} - n_{e^+} = q_{\rm baryon}\, n_b$ inside the transport RHS each step and feeds it to the lepton kernels and the QED EoS (full/qke modes). It is valid while $T_{\rm pl} > m_e/300 \approx 1.7$ keV ($\ln a \lesssim 8.8$ from 10 MeV): below that BURST's $E/T > 300$ occupation cutoff removes every quadrature node, so `solve_phie` returns NaN and the solve fails rather than continuing on a wrong ϕ_e. Its $N_{\rm eff}$ effect is below the 10^{-7} reproducibility floor ($\phi \sim 6 \times 10^{-10}$ at 10 MeV, rising to $\mathcal{O}(0.1)$ near the window bottom), so it is a completeness knob, not an $N_{\rm eff}$ lever. What is not ported is BURST's way of closing its own ϕ_e ODE, the `glag6` ($\mathrm{d}\rho_e/\mathrm{d}\phi_e$, BURST's `drhoepdphi`) EoS-derivative term; the port gets ϕ from exact charge neutrality directly instead.
- **MPI parallelization.** Replaced by the on-GPU tensor contraction.
- **Integration in time t.** Replaced by $\ln a$ with $T_{\rm cm} = T_{\rm cm}^{\rm init} e^{-\ln a}$ analytic.

12.2 Troubleshooting quick reference

The operational pitfalls scattered through the manual, collected for lookup; each row points to the section with the full explanation.

symptom	cause / fix
x64 silently off; sum rules fail; results look float32-grade	`import neutt` before creating any JAX array; x64 is enabled at import (Section 3.7).
`pytest` appears to hang for ∼10 min before any test runs	the one-time JAX compile, not a hang (Section 11).
Every loop iteration recompiles (∼40–80 s each; the neutt compile banner prints every time)	the solver is rebuilt inside the loop (each `make_*_solver` call returns a new `solve` that compiles afresh), or a build-time argument changes per call; build once outside the loop and vary only traced `solve` arguments (Section 10).
A scan call jumps from ∼4 s to ∼8 s	the hybrid qualification failed and the full-window fallback ran (the answer is still correct); switch a whole bad region to `hybrid=False` (Section 8.4).
A single retrace fires mid-scan	the first call that passes `solve`'s arguments in a new form, e.g. a value for an argument whose default is `None` (`me`, `xi`) or a keyword not passed before: one more variant is compiled once and reused (Section 10).
$N_{\rm eff}$ biased by $\sim +1.5 \times 10^{-3}$, or executable-dependent stalls	`atol` set above 10^{-30}; keep it at 10^{-30} (Sections 3.5, 12).
Energy-exchanging BSM gives the wrong coupled $N_{\rm eff}$	`transport="semi"` silently drops $\dot\rho_\nu$; use `transport="full"` for anything that exchanges energy with the neutrinos (Section 9.3).

symptom	cause / fix
Coupled run dies with a `diffeqsolve` "maximum number of steps" error	the LINX background solver's `max_steps` (default 512) is marginal for the NLO plasma EoS; pass `make_coupled_bbn(..., max_steps=8192)`. This is the background ceiling; the transport solver's own `max_steps` is separate, via `solver_kwargs` (Section 9).

13 The QKE flavour-mixing sector

This is the Standard Model's own flavour-mixing refinement of the $N_{\rm eff}$ calculation, not a BSM extension. Three-neutrino oscillations redistribute the spectral distortions across flavours as the neutrinos decouple, so the ν_e that drives the $n \leftrightarrow p$ rates carries a flavour-redistributed distortion. It is the most elaborate piece of the SM transport: it replaces the diagonal occupation $f[3, 2, N]$ with the full flavour density matrix and evolves the off-diagonal coherences. The lineage is the `qburst` Fortran code, but its end-to-end solve is not a trustworthy oracle (its operator-split scheme is reported to be step-size unstable), so the port is validated instead by exactness-by-construction (a matrix exponential that conserves neutrino number and positivity by design), by unit oracles (the verbatim-`gfortran` weak-potential and coherent-commutator checks), and, as the central check, by the **mixing-off reduction**: with the coherences set to zero the whole machinery reduces bit-identically to the validated classical transport of the preceding sections. The modules are under `neutt/transport/qke/`: `coherent` (the density-matrix state, PMNS mixing, the Hamiltonian and $-i[H, \rho]$), `nonadiabatic` (the coherent-step propagators and the moving eigenframe), `collide_matrix`, `collide`, `collide_nue`, `collide_nunu` (the density-matrix collision), `full_coupled` (the solver), and `solve` (a standalone Strang driver over a given background).

13.1 The density-matrix state

The classical state is the real occupation $f[\text{flavour} = 3, \text{cp} = 2, \text{bin} = N]$, diagonal only. The QKE generalizes it to a 3×3 complex Hermitian flavour density matrix per (cp, bin):

$$\rho[\text{cp} = 2,\ \text{bin} = N,\ a,\ b] \in \mathbb{C}^{2\times N\times 3\times 3}, \tag{17}$$

with the diagonal real, $\rho[\text{cp}, i, m, m] \equiv f[m, \text{cp}, i]$ (the classical occupations embedded exactly), and the off-diagonal complex $\rho[\text{cp}, i, a, b]$ $(a \neq b)$ the **flavour coherences**, the new degrees of freedom, zero at the initial condition (decohered flavour states). We write ρ for the neutrino block (cp= 0) and $\bar{\rho}$ for the antineutrino block (cp= 1); ρ with a species subscript (ρ_γ, $\rho_{e^\pm}$) remains an energy density. The state evolves under $i\,\mathrm{d}\rho/\mathrm{d}t = [H, \rho] + iC[\rho]$, with H the flavour Hamiltonian ($H_{\rm flav}$ of Eq. (16); in this section H denotes it, not the Hubble rate) and $C[\rho]$ the collision term: the coherent commutator is integrated by an exact, number- and positivity-preserving matrix exponential (no per-component ODE tolerance; Section 13.3), the collision term by the classical kernels.

[**Port note**] The batch axes (cp, bin) come first and the matrix (flavour) axes last, so the commutator and `matmul` broadcast over (cp, bin) with no transposes, a deliberate layout change from Fortran's `dens_mat(a,b,bin)` (same content, reordered for JAX vectorization). Helpers: `diag_to_rho`, `rho_to_diag`, `hermitize`, `commutator`; with the off-diagonal set to zero, `rho_to_diag` recovers the classical $f[3, 2, N]$ exactly.

13.2 The Hamiltonian and what is on by default

The coherent RHS is $\mathrm{d}\rho/\mathrm{d}t|_{\mathrm{coh}} = -i[H,\rho]$ with, per (cp, bin) at momentum $p_1 = \varepsilon_i T_{\mathrm{cm}}$ and parfact $= \pm 1$ for $\nu/\bar{\nu}$,

$$H = H_{\mathrm{vac}} + H^{(1)}_{\mathrm{mat}} + H^{(2)}_{\mathrm{mat}} + H^{(1)}_{\mathrm{si}} + H^{(2)}_{\mathrm{si}}. \tag{18}$$

term	**physics**	**production default**
H_{vac}	vacuum oscillation $\propto m^2_{\mathrm{weak}}/2p_1$	**ON**
$H^{(1)}_{\mathrm{mat}}$	1st-order MSW (Mikheyev–Smirnov–Wolfenstein) matter potential $\propto (n_{e^-} - n_{e^+})$	OFF (`include_matter_asym=False`; suppressed by the baryon-to-photon ratio η)
$H^{(2)}_{\mathrm{mat}}$	2nd-order thermal MSW (Nötzold–Raffelt) $\propto \rho_{e^\pm}$	ON iff `include_matter=True`
$H^{(1)}_{\mathrm{si}}$	1st-order ν–ν self-int $\propto (\rho - \bar{\rho})$	not included (zero)
$H^{(2)}_{\mathrm{si}}$	2nd-order ν–ν self-int $\propto (\rho + \bar{\rho})$	not included (zero)

The vacuum numerator $m^2_{\mathrm{weak}} = UM^2U^\top$ uses the real PMNS (Pontecorvo–Maki–Nakagawa–Sakata) mixing matrix $U = R_{23}R_{13}R_{12}$ (no CP phase) with $\sin^2\theta_{12} = 0.307$, $\sin^2\theta_{13} = 0.0212$, $\sin^2\theta_{23} = 0.417$, and $M^2 = \mathrm{diag}(0, \Delta m^2_{21}, \Delta m^2_{31})$ with $\Delta m^2_{21} = 7.53\times10^{-17}$ MeV^2 and $\Delta m^2_{31} = 2.59\times10^{-15}$ MeV^2 (normal hierarchy); the trace/3 is removed (an $\propto$identity shift drops out of the commutator but is removed explicitly to match Fortran bit-for-bit) and $m^2_{\mathrm{weak}}(\bar{\nu}) = -m^2_{\mathrm{weak}}(\nu)$. The two MSW and two self-interaction terms carry the standard prefactors $\sqrt{2}\,G_{\mathrm{F}}(n_{e^-} - n_{e^+})$ and $-\frac{8\sqrt{2}}{3}G_{\mathrm{F}}E_\nu\rho_{e^\pm}/m^2_W$ (matter), $\sqrt{2}\,G_{\mathrm{F}}(\rho - \bar{\rho})$-type and $-\frac{8\sqrt{2}}{3}(G_{\mathrm{F}}/m^2_Z)(\rho + \bar{\rho})$-type (self-interaction). Production N_{eff} / coherence solves activate H_{vac}, plus the thermal MSW term with `include_matter=True` and the asymmetric one with `include_matter_asym=True`; the two ν–ν terms are not included. `coherent.hamiltonian` is oracle-validated (full commutator to rel 10^{-15}).

The ν–ν terms are not included. The solver carries no ν–ν refractive term: $H^{(1)}_{\mathrm{si}} = H^{(2)}_{\mathrm{si}} = 0$. Both are nonzero in the symmetric universe. $H^{(2)}_{\mathrm{si}} \propto \Sigma w\varepsilon^3(\rho + \bar{\rho})$ (w the grid quadrature weights) is sourced only by the momentum-dependent flavour misalignment with the mean field (the commutator of the mean field with $\rho(p)$ vanishes when every momentum shares one flavour structure). $H^{(1)}_{\mathrm{si}} \propto \Sigma w\varepsilon^2(\rho - \bar{\rho})$ does not vanish at $\xi = 0$ either: the solve's symmetric state is $\bar{\rho} = \rho^*$, not $\bar{\rho} = \rho$, so $H^{(1)}_{\mathrm{si}} \propto \Sigma w\varepsilon^2\,\mathrm{Im}\,\rho$, the mean-field term of the flavour-pendulum system (the ATAO, adiabatic transfer of averaged oscillations, treatment of Froustey et al., in which ρ is real, drops it). Each moves N_{eff} by $|\delta N_{\mathrm{eff}}| \lesssim 3\times10^{-7}$ (measured with the vacuum and with the thermal-MSW Hamiltonian), below the precision of every result in this manual; the coherence frames are those of the model without ν–ν refraction.

The induced ν–$\bar{\nu}$ asymmetry. $H^{(1)}_{\mathrm{mat}} \propto \sqrt{2}\,G_{\mathrm{F}}n_p$ (the "solar", linear-G_{F} MSW term, with $n_p = q_{\mathrm{baryon}}n_b \propto \eta$ the baryon charge the $e^\pm$ gas balances) is the one $\xi = 0$ term that breaks $\nu \leftrightarrow \bar{\nu}$ in the diagonal. With it off, $m^2_{\mathrm{weak}} \to -m^2_{\mathrm{weak}}$ (vacuum) and the $\bar{\nu}$ sign of the thermal term give $H_{\bar{\nu}} = -H_\nu$, hence $\bar{\rho} = \rho^*$ and equal diagonals; $H^{(1)}_{\mathrm{mat}}$ alone carries no $\bar{\nu}$ flip, the complex-conjugate (Sigl–Raffelt) equivalent of the lab frame where the asymmetric potential flips and the thermal one does not (Nötzold–Raffelt), so it seeds a genuine flavour asymmetry $\rho_{aa}(\nu) - \rho_{aa}(\bar{\nu})$ by acting on the oscillation coherence. It is switched on by `include_matter_asym` (default off, which leaves the solve bit-identical), with a per-call traced `matter_asym_scale` on n_p (0 null, 1 physical η, >1 source sweep, all on one compile). Measured: the decoupling-era ($T_{\mathrm{cm}} \sim 2$ MeV), null-subtracted asymmetry is linear in n_p over scale $1 \to 100$ (saturating by 1000), with

physical-η magnitude $A_e/N_e \sim 2\times10^{-6}$ (A_e the grid-integrated $\nu_e-\bar\nu_e$ number difference, N_e the ν_e number) and $|\delta f_e|/f_e \sim$ few $\times 10^{-4}$ (local), $\Delta N_{\rm eff} = -1.4\times10^{-7}$, negligible for $N_{\rm eff}$ but a nonzero, η-controlled induced asymmetry. Raw peak-over-time measures are contaminated at the physical scale by the late-time lepton-kernel tail amplification of Section 6.4; null-subtract / restrict to the decoupling era for the clean signal.

Lower-level API (`coherent`).

```
diag_to_rho(f) -> rho[2,N,3,3] # f[3,2,N] embedded; coherences 0
rho_to_diag(rho) -> f[3,2,N] # the real diagonal; coherences dropped
hermitize(rho) # (rho + rho^dagger)/2 over the flavour axes
commutator(A, B) # A@B - B@A, batched over leading axes
pmns_matrix(s12=sqrt(SIN2_THETA12), s13=sqrt(SIN2_THETA13), s23=sqrt(SIN2_THETA23),
           nnu=3) -> U (3,3) # real R23 R13 R12, from the sines of the angles
m2_mass_matrix(dm2_21=DELTA_M2_21, dm2_31=DELTA_M2_31, normal_hierarchy=True,
              nnu=3) -> M2 (3,3) # diag(0, dm2_21, +-dm2_31) [MeV^2]
m2_weak(s12=..., s13=..., s23=..., dm2_21=..., dm2_31=..., normal_hierarchy=True,
        nnu=3) -> (3,3,2) # U M2 U^T - Tr/3; [...,1] = -[...,0] (nubar)
matter_terms(therm_corr, npro, nflavor=3) -> (mata, mats) # the two MSW potentials (ee entry)
self_interaction_terms(tcm, nudiff, nusum) -> (sia, sis) # the nu-nu potentials
hamiltonian(eps, tcm, m2_weak, mata_mat, mats_mat, sia_mat, sis_mat) -> H[2,N,3,3] # [s^-1]
coherent_rhs(rho, eps, tcm, m2_weak, *, therm_corr=0.0, npro=0.0,
             nudiff=None, nusum=None, hermitian_project=True) # -i[H, rho]; 0 at eps = 0
```

`tcm` is $T_{\rm cm}$ [MeV], `therm_corr` the $e^\pm$ energy density [MeV^4] of $H^{(2)}_{\rm mat}$ and `npro` the proton density n_p [MeV^3] of $H^{(1)}_{\rm mat}$; `nudiff`, `nusum` are the $\nu-\bar\nu$ and $\nu+\bar\nu$ moments of `solve.nudiff_nusum` (below); the solvers pass zero ν–ν potentials.

13.3 The operator split: `hybrid` by default

The solve uses operator splitting: rather than integrating the coherent (Hamiltonian) and collision parts of $\mathrm{d}\rho/\mathrm{d}s$ ($s\equiv\ln a$) together, each of the $n_{\rm steps}$ (`n_steps`) fixed $\ln a$ macro-steps, of duration $\mathrm{d}t$ in seconds, advances them in alternation as two substeps, which lets each use the integrator best suited to it (an exact propagator for the stiff coherent oscillation, the L-stable exponential-midpoint for the stiff collision).

- **Collision substep.** The production collision advances the diagonal with the validated classical kernels in affine form (`make_split_rates` $\to A-Bf$ over the five kernels + the $\mathrm{d}T_{\rm pl}$ ODE), with the L-stable exponential-midpoint step. How the collision acts on the coherences is the `collision` option of Section 13.4: by default a scalar damping, or the density-matrix (Sigl–Raffelt) operator of `collide.py`, `collide_nue.py` and `collide_nunu.py`.
- **Coherent substep.** $\rho \leftarrow \exp(L_{\rm coh}\,\mathrm{d}t)\rho$ per bin, with $L_{\rm coh} = -i[H,\cdot] - \Lambda_{\rm off}$ the 9×9 Liouvillian (9×9 because the 3×3 density matrix is flattened to a 9-vector, turning the commutator $[H,\cdot]$ into an ordinary matrix×vector product) and $\Lambda_{\rm off}$ an off-diagonal-only dephasing: it exponentially damps the off-diagonal coherences at $\Gamma_{ab} = \frac{1}{2}(\Gamma_a+\Gamma_b)$ (Γ_a the total collision rate of flavour a, Section 13.4) while, being trace- and positivity-preserving, never pushing an occupation negative. Two choices matter: the diagonal is undamped here (its collisional loss already lives in the collision substep; damping it again double-counts), and $L_{\rm coh}$ is a Lindblad pure-dephasing $\oplus$ unitary channel, so expm **conserves $\sum_a \rho_{aa}$ (number) and keeps the diagonal ≥ 0 (positivity) by construction**, which is why it cannot trip the collision kernel's occupation-floor cliff. (`max_squarings=40`, not the JAX default 16: at late times the rotation $\omega\,\mathrm{d}t$ (ω the largest eigenvalue splitting of H) reaches $\sim 10^9$ rad/step, which the default silently clamps $\to$ NaN. At that phase the squaring is not exactly unitary in floating point; see `hybrid` below.)

The `expm` integrator is thus the dense 9×9 matrix exponential $\exp(L_{\rm coh}\,\mathrm{d}t)$ of the dissipative Liouvillian, BURST's corotating-frame Magnus idea realized as a single exact interaction-picture propagator, with no tuned subtraction and no per-bin frame-switch logic, exact for any stiffness up to a round-off that grows with the phase per step (below). It is also tolerance-free: there is no per-component `rtol`/`atol` on the coherences; the stiff phase $\omega\,\mathrm{d}t$ is exponentiated exactly rather than error-controlled, so a small imaginary off-diagonal cannot drift against a loose tolerance the way it does in an adaptive operator-split integrator (the failure mode of a corotating scheme such as `qburst`'s; see "Watch the magnitude" in Section 13.6). (The opt-in Strang split below instead realizes the unitary part as a 3×3 Hermitian eigendecomposition $U = Ve^{-i\Lambda \mathrm{d}t}V^\dagger$ (V, Λ the eigenvectors and eigenvalues of H; this U is the step unitary, not the PMNS matrix; `nonadiabatic.propagate_coherent_H`), plus a Hadamard dephase, dropping the 9×9 `expm` and its `max_squarings=40`.)

Accuracy of the coherent integrators; `hybrid`. Scaling-and-squaring is not exactly unitary at large phase: per step the dense `expm` moves occupations, per-bin number included, by $\approx 10^{-17}\,\omega\,\mathrm{d}t$, with ω the largest eigenvalue splitting of H. At low ε and late times $\omega\,\mathrm{d}t$ reaches 10^8–10^{10}, and the accumulated round-off makes the mixing-on solve ill-conditioned: a 10^{-12} change of `T_init` moves $N_{\rm eff}$ by $\sim 10^{-6}$ and the low-ε ($\varepsilon \leq 1$) flavour-differential distortion $\delta_e - \delta_\mu$ (δ_a the fractional distortion δf of flavour a) by $\sim 40\%$, and $\Delta N_{\rm eff}$ carries a round-off bias of $\approx +3 \times 10^{-6}$ (damping, $N = 101$, $n_{\rm steps} = 2000$). The Strang split (`coherent_method="strang"`: flavour-basis dephasing as half-step kicks around an exact unitary-plus-source step) has no round-off, but applies each kick between rotations of $\omega\,\mathrm{d}t \gg 1$ rad. That split error costs 1.5×10^{-6} in $\Delta N_{\rm eff}$ at $n_{\rm steps} = 2000$, spreads the low-ε $\delta_e - \delta_\mu$ by 5% across step counts and grids, and leaves a spurious rotating low-ε coherence in the mass basis of magnitude 9×10^{-8}, where the dense `expm` gives 1.6×10^{-10} for the same coherence.

`coherent_method="hybrid"` (`nonadiabatic.coherent_step_sourced_hybrid`) has neither error. Per bin it keeps the dense `expm` where $\omega\,\mathrm{d}t \leq 10^3$ (round-off $\lesssim 10^{-14}$). Elsewhere, if $\Gamma_{\rm max}\mathrm{d}t \leq 1$ ($\Gamma_{\rm max}$ the bin's largest decoherence rate), it takes a secular step in the eigenbasis of H: each component's rotation and own decay are exact phases (unitary at any $\omega\,\mathrm{d}t$), the population block of the transformed dephasing is exponentiated exactly, and the oscillating (non-secular) couplings enter averaged over the step (first Magnus term in the interaction picture). Its errors are $\mathcal{O}(\Gamma \mathrm{d}t\,(\Gamma/\omega)^2)$ on populations and $\mathcal{O}(\Gamma \mathrm{d}t\,\Gamma/\omega)$ on coherences, and both branches are accurate at the switch (moving it to 10^4 changes $\Delta N_{\rm eff}$ by $\sim 10^{-11}$). Measured (damping, $N = 101/201$, $n_{\rm steps} = 2000$–32000): the 10^{-12} nudge moves $N_{\rm eff}$ by 3×10^{-13}, $\delta_e - \delta_\mu$ at $\varepsilon \leq 1$ agrees to 0.3% across all six runs (`expm`: 41%), per-bin number is conserved to 10^{-13}, and the rotating coherence equals `expm`'s. Under `collision="matrix_full"` the nudge moves $N_{\rm eff}$ by 3×10^{-14} (`expm`: 1.1×10^{-6}) and the low-ε spread is 0.8% (`expm`: 22%). It costs ∼17% more wall time in damping mode and ∼8% in `matrix_full`. `hybrid` is the default.

The moving eigenframe of the step. The coherent step freezes H at the step midpoint. Where the thermal potential rotates the eigenbasis of H as T falls (most at high ε, where $V_{\rm th}/\omega_{\rm vac} \propto \varepsilon^2 T^6$, with $V_{\rm th}$ the thermal MSW potential and $\omega_{\rm vac}$ the vacuum splitting), a frozen H changes direction at every step boundary and leaves the coherence behind by the rotation angle, at an effectively random phase once $\omega\,\mathrm{d}t \gg 1$. Those kicks random-walk into bin-to-bin structure in the high-ε coherences that falls only as $1/n_{\rm steps}$: in the non-adiabaticity measure $\Theta = \|[H_{\rm vac}, \rho]\|^2/(\|H_{\rm vac}\|^2\|\rho\|^2)$ (Grohs et al.; Frobenius norms, unit-trace ρ), $\Theta \approx 10^{-13.5}$ at $\varepsilon = 20$–25 for $n_{\rm steps} = 6000$, against $< 10^{-16}$ converged. Each step therefore runs in the frame that carries the eigenbasis of $H(t_{\rm start})$ onto that of $H(t_{\rm end})$. With the frame generator K (Hermitian) defined by $e^{-iK\mathrm{d}t} = V_{\rm end}V_{\rm start}^\dagger$ (eigenvectors in matching order, phase-aligned; `non`

`adiabatic.eigenframe_generator`) and $\tilde{R} = e^{-iK\mathrm{d}t/2}$,

$$\rho(\mathrm{d}t) = \tilde{R}\,\mathrm{step}\big(H_{\mathrm{mid}} - K,\ \tilde{R}\rho\tilde{R}^{\dagger}\big)\ \tilde{R}^{\dagger}, \tag{19}$$

where H_{mid} is H at the step midpoint, step is the coherent substep above, and $-K$ is the geometric term of the rotating frame; in that frame the eigenbasis is nearly fixed and the coherence is transported adiabatically. The frame coincides with the flavour basis at the step midpoint, where Γ and the collision source are evaluated. $K = 0$ when the eigenvectors do not move (vacuum), which recovers the plain step bit-for-bit, and, as a guard, where the frame would turn by more than 0.5 rad in one step (an eigenvalue crossing reorders the eigenvectors). Measured with the thermal potential: at $n_{\mathrm{steps}} = 6000$ no high-ε bin has $\Theta > 10^{-15}$ (with the frozen step all twenty at $\varepsilon = 20$–25 do, and the frozen step is still worse at $n_{\mathrm{steps}} = 150\,000$); between 0.5 and 5 MeV the coherences match the frozen step at $n_{\mathrm{steps}} = 150\,000$ to $|\Delta \log_{10} \Theta| \leq 0.008$; and N_{eff} moves by $+5.0 \times 10^{-8}$ ($n_{\mathrm{steps}} = 2000$) and $+1.0 \times 10^{-8}$ (6000), the frozen step's error shrinking with the step. The unit test is the adiabatic theorem: an eigenstate of a slowly rotating H leaks $3.8 \times 10^{-9} \approx (|K|/\omega)^2$ with the frame and 7.7×10^{-6} with the frozen step.

Lower-level API (`nonadiabatic`, `solve`). Matrices are $(\ldots, 3, 3)$ complex, batched over leading axes such as (cp, bin); H and the rates are in s^{-1}, `dt` in s, `gamma_diag` is Γ_a $(\ldots, 3)$ and `src` the sourced step's s_{coll}.

```
# the coherent substep (full_coupled's coherent_method) and the moving frame
coherent_liouvillian(H, gamma_diag) -> L_coh (...,9,9) # -i[H,.] - Lambda_off, row-major vec
coherent_step(H, gamma_diag, rho, dt, max_squarings=40) # expm(L_coh dt) rho
coherent_step_sourced(H, gamma_diag, rho, dt, src, max_squarings=40) # ... + dt phi1(L_coh dt)
  ↪ src
coherent_step_strang(H, gamma_diag, rho, dt) # D(dt/2) U(dt) D(dt/2)
coherent_step_sourced_strang(H, gamma_diag, rho, dt, src) # "strang"
coherent_step_sourced_secular(H, gamma_diag, rho, dt, src) # secular step, omega dt >> 1
coherent_step_sourced_hybrid(H, gamma_diag, rho, dt, src, wdt_switch=None) # "hybrid"
HYBRID_WDT_SWITCH = 1.0e3 # its default omega_max dt switch
eigenframe_generator(H_start, H_end, dt, max_angle=0.5) -> K # the moving-eigenframe generator
unitary_exp(K, t) # e^{-iKt}
# exact unitary, frozen-diagonal coherence, collision step, diagnostics
eigendecompose(H) -> (evals, V) # of the hermitized H; evals ascending, V's columns
magnus_propagator(evals, V, dt) -> U # V e^{-i Lambda dt} V^dagger
propagate_coherent_H(rho, H, dt) # U rho U^dagger, U = e^{-iH dt}; U = I where H = 0
liouvillian(H, gamma_diag) # -i[H,.] - {diag(Gamma/2), .}: damps the diagonal too
offdiag_blocks(H, gamma_diag) -> (L_oo, L_od) # of liouvillian, (...,6,6) and (...,6,3)
qss_offdiag(L_oo, L_od, rho_diag) -> (...,6) # -L_oo^{-1} L_od rho_diag (a diagnostic)
evolve_offdiag(L_oo, s, rho_off, dt, max_squarings=40) # exact step of rho_off' = L_oo rho_off
  ↪ + s
rho_from_offdiag(rho_off, rho_diag) -> rho # the Hermitian (...,3,3)
off_diag_amps(rho, V=None, H=None) # mass-basis amplitude envelope (qburst's)
lstable_collision_step(rho, A_rate, B_rate, dt) # exact step of drho/dt = A - (B rho + rho B)
strang_step(rho, H_fn, collision_fn, t, dt) # half collision, exact unitary, half collision
# solve.py: a standalone Strang driver over a given background
make_coherent_H_fn(grid, m2_weak, bg_fn) -> H_fn(t, rho=None) # bg_fn(t) -> {"T_cm", "
  ↪ therm_corr", "npro"}
evolve(rho0, t_grid, H_fn, collision_fn=None) -> (M,2,N,3,3) # strang_step over t_grid [s]
number_density(grid, rho, T_cm), energy_density(grid, rho, T_cm) # moments of rho [MeV^3], [MeV
  ↪ ^4]
nudiff_nusum(grid, rho, diff_pow=2, sum_pow=3) -> (nudiff, nusum) # the nu-nu source moments
coherence_norm(rho) # sum over a != b of |rho_ab|
```

`strang_step` and `solve.evolve` take explicit half steps of the collision, so they suit only a resolved (non-stiff) collision; the production solve is `full_coupled`.

13.4 The collision acting on the coherences

Two options set how the collision substep reaches the density matrix: `collision_coupling` (how the classical diagonal update is handed to the coherent step) and `collision` (which operator acts on the coherences).

`collision_coupling`: `"sourced"` (default) or `"jump"`. Under `"sourced"` the collision's change of the diagonal enters the coherent step as a constant rate $s_{\rm coll} = (f_{\rm coll} - f)/\mathrm{d}t$ ($f_{\rm coll}$ the diagonal after the collision substep) inside the exponential (`nonadiabatic.coherent_step_sourced` and its `hybrid`/`strang` counterparts), so the diagonal ramps during the step and the off-diagonal's quasi-steady target $-L_{oo}^{-1}L_{od}\rho_{\rm diag}$ moves continuously ($\rho_{\rm diag}$ and $\rho_{\rm off}$ the diagonal and off-diagonal entries of ρ, L_{oo} and L_{od} the off-diagonal→off-diagonal and diagonal→off-diagonal blocks of $L_{\rm coh}$); for a constant $(L_{\rm coh}, s_{\rm coll})$ the step's fixed point is exactly $-L_{\rm coh}^{-1}s_{\rm coll}$, so a solve sitting on the quasi-steady state stays on it. `"jump"` applies the collision first and runs the coherent step on the updated state: the target then jumps once per step and $\rho_{\rm off}$ trails it by $\mathcal{O}(\mathrm{d}t)$, a lag that is generated across the transition epoch and freezes in once $\Gamma \to 0$. It puts $\mathrm{Im}\,\rho_{e\tau}$ at 2.5×10^{-7} ($n_{\rm steps} = 10^4$) where the quasi-steady value is $\sim 4\times10^{-15}$, converging only at first order; `"sourced"` gives 5.5×10^{-11}, 7.9×10^{-12}, 2.4×10^{-12} at $n_{\rm steps} = 10^4$, 5×10^4, 10^5, with $N_{\rm eff}$, `num_drift` (the per-step number-conservation residual) and `min_f` (the minimum occupation) unchanged. $N_{\rm eff}$ barely sees the difference (it rides on the diagonal, which the collision dominates).

`collision`: the operator on the coherences.

- `"damping"` (default): the scalar model $\mathrm{d}\rho_{ab}/\mathrm{d}t = -\frac{1}{2}(\Gamma_a + \Gamma_b)\rho_{ab}$, with Γ_a the ungated total collision throughput of flavour a (`assemble.decoherence_rate`; `gamma_ungated=False` substitutes the gated transport rate, a comparison mode). It is exact as a loss but drops the coherence gain, the momentum-nonlocal repopulation of $\rho_{ab}(p)$ by the other momenta.
- `"matrix"`: all five SM channels act through the density-matrix (Sigl–Raffelt) operator $\mathrm{d}\rho/\mathrm{d}t|_{\rm coll} = A_{\rm gain} - \{B_{\rm loss}, \rho\}$: $\nu\bar\nu \to e^+e^-$ (`collide.py`), ν–$e^\pm$ scattering (`collide_nue.py`) and ν–ν, ν–$\bar\nu$ (`collide_nunu.py`). They share the classical kernels' energy and angular quadrature, with the blocking factors $(1-f)$, f promoted to $(I-\rho)$, ρ inside anticommutators, so $A_{\rm gain}$ and $B_{\rm loss}$ are Hermitian, and $B_{\rm loss} \succeq 0$ on a physical state ($0 \preceq \rho \preceq I$) except at `nuepma`'s lowest bin below $T_{\rm cm} \approx 0.1$ MeV, where BURST's quadrature leaves a small negative eigenvalue (Section 6.4). On a diagonal ρ each reduces to its classical kernel, and $2\,\mathrm{diag}(B_{\rm loss})$ equals the ungated throughput, so each channel's scalar damping is replaced by $2B_{\rm loss}$ and the remainder of the operator enters the sourced step as a source: the off-diagonal right-hand side is $T_{\rm cm}^5\,\mathrm{net}_{ab}$ with no truncation in the coherence order. The diagonal is still advanced by the classical kernels.
- `"matrix_full"`: as `"matrix"`, and the diagonal is taken from the same operator, $A_{{\rm gain},aa} - \{B_{\rm loss}, \rho\}_{aa}$: the classical affine $A - Bf$ plus the $\mathcal{O}(|\rho_{\rm off}|^2)$ coherence feedback the classical kernels cannot see. Its affine halves go to the same exponential-midpoint step, so the two modes differ in the operator only. The precision gate acts on the diagonal alone, with the classical gate decision (the same open bins); gating the coherences would switch their damping off at freeze-out.
- `"matrix_nue"`: the three ν–e channels on the matrix operator, ν–ν and ν–$\bar\nu$ on the damping model (a comparison mode).

The matrix modes need `collision_coupling="sourced"` (the gain is a source, and the jump path has nowhere to put it) and `gamma_ungated=True`; the factory rejects other combinations. The matrix kernels take the same off-grid tail closure as the classical ones (Section 13.5).

Measured (vacuum mixing, $N = 101$, $n_{\rm steps} = 2000$; $N_{\rm eff}^{\rm damping} = 3.043945$):

`collision`	$N_{\rm eff} - N_{\rm eff}^{\rm damping}$	**notes**
`"matrix_nue"`	-4.5×10^{-7}	the ν–e gain alone
`"matrix"`	-9.2×10^{-5}	ν–ν gain dominates; $\lvert\rho_{\mu\tau}\rvert$ +35%
`"matrix_full"`	`"matrix"` $\pm 5 \times 10^{-7}$	feedback $\lesssim 10^{-5}$ of the throughput

The ν–ν gain is 5–15 times the ν–e one, and the `"matrix"` shift is converged to 1.5% in step count (-9.4×10^{-5} at `n_steps=6000`) and 0.7% in grid ($N = 101 \to 201$). The `"matrix_full"` shift is not resolved by the fixed-step solve: it scatters at the $\pm 3 \times 10^{-7}$ level between step counts, so it is quoted as the bound $|\delta N_{\rm eff}| \lesssim 5 \times 10^{-7}$. Number conservation is unchanged (largest `num_drift` 6×10^{-14} in every mode).

13.5 The off-grid tail closure

The QKE solve needs an off-grid tail closure, and getting it right is the most important numerical "why" in this sector.

The root cause: the lepton kernels amplify an off-grid tail. The ν–ν / ν–e kernels read an off-grid source occupation $f(p_4)$, $p_4 = p_1 + E_2 - E_3$, which reaches the Gauss–Laguerre tail past the grid edge ($25 < p_4 < 50$). There the order-5 log-occupation polynomial extrapolates tens of grid-steps past the data, so a tiny spectral perturbation is amplified into an unphysical occupation. In the QKE solve the coherent back-reaction seeds exactly such a tail perturbation $\to$ an $f(p_4)$ spike $\to$ pure gain $\to f$ pumped toward 1 $\to$ fake energy $\to T_{\rm pl}$ corrupts $\to N_{\rm eff} \to$ 0/NaN. The culprit is the SM lepton kernels (`nuer2`/$\nu\nu$) amplifying a coherence-induced super-thermal tail perturbation, the lepton-kernel boundary fragility (Section 12), H-theorem-violating on non-smooth spectra. The coherence engine is sound; the collision amplifies a seed that should have been negligible, so a well-posed solve needs a closure that bounds the overshoot without erasing real signal.

The default: the parameter-free `srcfix` closure. The QKE solver defaults to `tail_closure="srcfix"`, a parameter-free pair of closures. `fd_decay` carries the live grid-edge $f/f^{\rm eq}$ ratio outward along the FD shape (bounding the polynomial overshoot with no tunable constant while preserving a genuine super-FD edge), and `monotone_read` clamps the on-grid interpolant to its two-node bracket, killing the odd–even Nyquist mode the kernels self-amplify. With it the collision damps the spurious spike rather than amplifying it. Because the same closure acts in both mixing branches it cancels in the mixing-on$-$mixing-off $\Delta N_{\rm eff}$: it reproduces that differential (vacuum $+7.6 \times 10^{-4}$, matter $+6.7 \times 10^{-4}$) and keeps the classical (mixing-off) reduction bitwise unchanged.

The opt-in FD-envelope cap. The alternative, `tail_closure="cap"`, caps the off-grid source to a Fermi–Dirac envelope $f(p) \le C \cdot f^{\rm eq}(p)$ with $C = 10$ (i.e. $\log f \le \log C -$ logaddexp$(p, 0)$, applied with `jnp.minimum`) on both `full_coupled` kernel edges, including the non-cancelling `decoherence_rate`, which must be capped explicitly or it silently contaminates $\Delta N_{\rm eff}$, and the three scheduled lepton kernels. The mixing-on $N_{\rm eff}$ is flat in C across $\sim [5, 40]$. The cap is the numeric `fd_cap_logC` (default $+\infty$: minimum$(x, +\infty) = x$, a bitwise no-op, jit-folded at zero hot-path cost), so the classical SM transport is untouched and BURST-faithful. `srcfix` is preferred as the default because it is strictly more correct at a degenerate spectrum: at $\xi = 3 > \ln C$ the absolute $C \cdot f^{\rm eq}$ cap clips the genuine degenerate tail ($N_{\rm eff} = 13.1$,

wrong) whereas the edge-anchored closure preserves it ($N_{\rm eff} = 20.2$). A naked, no-closure solve diverges to NaN, so a closure is always load-bearing.

[Port note] The off-grid closure is a documented improvement over BURST's bare extrapolation, applied only where it is physically valid (the near-thermal QKE tail) and held off where it would move the SM collaborator oracle. Tail-accuracy ordering: `srcfix` > FD-envelope > BURST-bare. Approaches that do not work, and why: a global FD cap shifts the SM oracle by -1.2×10^{-6}; one-edge gating in `full_coupled` leaves the `decoherence_rate` edge uncapped; and a projection onto positive-semidefinite ρ or a number renormalization, interp-extrapolation caps, coherence tail-insulation, and a Γ gate are diagnostic-only or ineffective; nor is the tail pump an operator-split aliasing artefact (the behaviour is node-converged to $n_{\rm steps} = 192000$).

13.6 Headline results

medium	$N_{\rm eff}$ **(mixing on)**	**mixing** $\Delta N_{\rm eff}$
vacuum	3.044071	$+7.64 \times 10^{-4}$
matter (MSW)	3.043976	$+6.69 \times 10^{-4}$

The absolute $N_{\rm eff}$ are at the NLO QED default and $n_{\rm steps} = 6000$, with the default damping collision on the coherences; $\Delta N_{\rm eff}$ is taken against the mixing-off solve at the same $n_{\rm steps}$ ($N_{\rm eff} = 3.043307$, the classical reduction). The density-matrix collision (`collision="matrix_full"`) lowers the vacuum value by 9.4×10^{-5} (Section 13.4). For comparison, Bennett et al. (2021, arXiv:2012.02726, Table 3) obtain $N_{\rm eff} = 3.0440$ in the Standard Model with flavour oscillations and NLO QED; there oscillations raise $N_{\rm eff}$ by $+5.7 \times 10^{-4}$ with the full collision integrals, and damping terms on the coherences add a further 1×10^{-4}. The mixing $\Delta N_{\rm eff}$ (the deliverable) is QED-baseline-independent: QED acts identically on the mixing-on and mixing-off runs and cancels in the difference. MSW lowers $\Delta N_{\rm eff}$ by $\sim 10^{-4}$ (no resonance). $\Delta N_{\rm eff}$ changes by $\leq 3 \times 10^{-6}$ between `n_steps` 2000 and 6000 (the order-2 absolute-baseline error cancels in the difference because the off-grid tail closure, Section 13.5, is applied identically in both branches), so `n_steps=6000` is the default for absolute observables and 2000 a validated 3× faster delta mode.

The mixing $\Delta N_{\rm eff}$ is adiabatic. During decoupling the collision rate is far below the oscillation frequencies ($\Gamma/\omega_{31} \sim 10^{-3}$ at $T \sim 3$ MeV, $\omega_{31} = \Delta m^2_{31}/2p$), so ρ stays diagonal in the instantaneous eigenbasis of H up to $\mathcal{O}(\Gamma/\omega)$. With large vacuum mixing that state is not flavour-diagonal: its flavour off-diagonals are the image of unequal eigenbasis populations. Projecting the solver's collision $A - \frac{1}{2}\{B, \rho\}$ onto it gives, per ($\nu/\bar{\nu}$, bin), a closed equation for the eigenbasis populations p_i,

$$\frac{\mathrm{d}p_i}{\mathrm{d}\ln a} = \sum_a |U_{ai}|^2 A_a - \Big(\sum_a |U_{ai}|^2 B_a\Big) p_i, \qquad f_a = \sum_i |U_{ai}|^2 p_i,$$

with U the eigenvectors of H and A, B the classical affine-split rates. Integrated with the same exponential-midpoint step, it reproduces the full solve's mixing $\Delta N_{\rm eff}$ to 0.2% (vacuum) and 0.13% (matter) at $n_{\rm steps} = 6000$, and its $N_{\rm eff}$ to 1.5×10^{-6}. The mixing $\Delta N_{\rm eff}$ is therefore carried by the adiabatically transported eigenbasis populations, and the non-adiabatic remainder is $\lesssim 2 \times 10^{-6}$ in $N_{\rm eff}$. The pairwise flavour-conversion rate $R_{ab} = 2|H_{ab}|^2 \Gamma_{ab}/(\Delta^2_{ab} + \Gamma^2_{ab})$ (the two-flavour quasi-steady result, $\Delta_{ab} = H_{aa} - H_{bb}$) is not this limit: it assumes the flavour-basis coherence is small (small mixing, or $\Gamma \gg \omega$), and as a closure on the flavour diagonal it gives a mixing $\Delta N_{\rm eff}$ of $+3.0 \times 10^{-4}$ (vacuum), 2.5× too small.

The off-diagonal coherences are small. The largest flavour-basis coherence (the maximum of $|\rho_{ab}|$ over bins) rises to 1.8×10^{-5} and stays there to the end of the run (vacuum, $n_{\rm steps} = 6000$). This is the expected size: the off-diagonal is the eigenbasis image of the flavour differences δf (the $f_0\mathbb{1}$ part cancels under the eigenbasis projectors), and the largest $|f_a - f_b|$ is 3.5×10^{-5}. The mass-basis coherence $|\rho_{ij}^{\rm mass}|$ (maximum over bins) peaks at 1.6×10^{-7} near $T_{\rm cm} = 6$ MeV and freezes at 2.9×10^{-10} once $\Gamma \to 0$. Corner-cutting computations give coherences of order 0.1–1 instead, an artefact of either an aliased explicit operator-split dump (whose coherences grow non-convergently and violate positivity) or of the frozen-diagonal quasi-steady coherence $-L_{oo}^{-1}L_{od}\rho_{\rm diag}$ used as a closure. The matrix $|U_{ai}|^2$ is nearly singular for the PMNS angles (smallest singular value 1.1×10^{-3}, near μ–τ symmetry), so once $\Gamma \lesssim 10^{-4}\omega_{31}$ a diagonal that is not of the form $|U|^2 p$ maps to $\mathcal{O}(1)$ coherences. The full solve does neither. ν and $\bar\nu$ coincide in the symmetric case ($\bar\rho = \rho^*$).

Watch the magnitude, not the lab-frame phase. The instantaneous lab-frame real and imaginary parts of ρ_{ab} are not the right object to plot over time. The off-diagonal carries the fast vacuum phase $e^{-i\Delta t}$ ($\Delta = \Delta m^2_{ij}/2p$, the vacuum oscillation frequency, winds $\sim 10^9$–10^{10} rad per macro-step at late times), so its real and imaginary components rotate; the value sampled at frame boundaries is therefore an aliased phase, and it changes with the step count or the grid, an artifact of displaying a fast rotation, not an instability in the solve. The physically meaningful quantities are phase-/aliasing-immune and do converge: the magnitude $|\rho_{ab}|$, the mass-basis (eigenbasis-of-H) coherence, and $N_{\rm eff}$ itself. Doubling the binning ($N = 101 \to 201$ at fixed `n_steps`) leaves $N_{\rm eff}$ unchanged to 9×10^{-7}, the mixing $\Delta N_{\rm eff}$ to 3×10^{-7}, and $|\rho_{ab}|$ identical to four figures. After decoupling $\Gamma \to 0$, so the dephasing $\Lambda_{\rm off} \to 0$ and the late evolution is the undamped free precession $\rho \to e^{-iH_{\rm vac}t}\,\rho\,e^{+iH_{\rm vac}t}$, with $|\rho_{ab}|$ flat to the end of the run. The flavour off-diagonal is then a constant, essentially real offset (the image, through the real δ_{CP}=0 PMNS rotation, of the unequal mass-basis occupations) plus the frozen 2.9×10^{-10} mass-basis coherence, which precesses. Neither moves $N_{\rm eff}$: the trace is invariant under the free precession, and positivity ($\min f \geq 0$) and per-bin number (drift $\lesssim 10^{-13}$) are conserved at every step by the completely-positive trace-preserving (CPTP) coherent step, so the coherence cannot "leak into" and corrupt the diagonal. The exact CPTP exponential propagator has no per-component tolerance for the small imaginary off-diagonal to drift against, which is why the solve is free of the corotating-scheme instability (Section 13.3).

13.7 The public API and status

```
make_full_coupled_solver(grid=None, *, mixing=True, include_matter=False,
       include_matter_asym=False, use_renorm=True, renorm_pdep=True, qed_nlo=True,
       use_sched="union", tcm_cutoff=0.015, n_steps=6000,
       phie=0.0, eta0=None, q_baryon=0.5, xi=(0.0,0.0,0.0), normal_hierarchy=True,
       gamma_ungated=True, coh_eps_cut=None, n_frames=600,
       tail_closure="srcfix", fd_cap_C=None,
       coherent_method="hybrid", collision_coupling="sourced",
       collision="damping") -> (solve, grid)
# solve(T_init=10.0, lna_max=7.0, delta_neff_init=0.0, me=None, xi=None,
# matter_asym_scale=1.0) -> dict
```

The builder arguments are build-time and the `solve` arguments traced (Section 10). The returned `solve` gives `neff_final`, the background scalars, `f_final` (3,2,N), `rho_final`, the per-step histories `T_pl_hist`/`T_cm_hist`, the coherence frame arrays (`off_fr`, the off-diagonal; `ss_fr`, the frozen-diagonal quasi-steady coherence on the same diagonal, a diagnostic with the conditioning of Section 13.6; `diag_fr`; at `lna_fr`, `T_cm_fr`, and the end-of-step `T_cm_end_fr`), and the conservation diagnostics `num_drift`/`min_f`/`any_nan` (plus `lna_left`, `frame_id`

x and the closure provenance fd_cap_C/fd_cap_logC). Key knobs: mixing=False gives the exact classical reduction; include_matter adds the thermal MSW potential and include_matter_asym the asymmetric "solar" term, scaled per call by matter_asym_scale (Section 13 Hamiltonian table; eta0, q_baryon set its n_p); coherent_method selects hybrid (default), expm or strang (Section 13.3); collision and collision_coupling set the collision on the coherences (Section 13.4); tail_closure ("srcfix", default, or "cap") selects the off-grid tail closure (Section 13.5), and fd_cap_C is the FD-envelope multiplier C that takes effect only under tail_closure="cap" (default None: C = interp.FD_ENVELOPE_C = 10), threaded to the kernels as fd_cap_logC = $\log C$. The remaining arguments mirror the SM transport builders: use_renorm / renorm_pdep (finite-T QED), use_sched (kernel path, default "union" here), tcm_cutoff (window low edge), phie / xi (electron / neutrino degeneracies), normal_hierarchy (mass ordering), n_steps (operator-split macro-steps; Section 14), plus three coherence knobs best left at their defaults: n_frames (how many coherence snapshots are saved), gamma_ungated (the off-diagonal damping rate, Section 13.4) and coh_eps_cut (an optional ε above which H is zeroed, a diagnostic). It is reached from BBN by make_coupled_bbn(transport="qke", qke_kwargs={...}), which builds the QKE solve on GPU and runs LINX on CPU (the device split). Warm ∼25 s (∼ 6× the SM transport="full"), cold ∼2 min (A100, n_steps=6000).

Demo: the mixing $\Delta N_{\rm eff}$. The headline number is the difference between the mixing-on and mixing-off solves (the off-grid tail closure of Section 13.5 cancels in the difference):

```
import neutt # FIRST: enables x64
from neutt.transport.qke.full_coupled import make_full_coupled_solver

solve_on, _ = make_full_coupled_solver(mixing=True) # build ONCE (~76 s cold)
solve_off, _ = make_full_coupled_solver(mixing=False) # the exact classical reduction
neff_on = float(solve_on()["neff_final"])
neff_off = float(solve_off()["neff_final"])
print(neff_on - neff_off) # -> Delta N_eff = +7.6e-4 (vacuum)
```

Pass include_matter=True for the thermal-MSW variant ($+6.7 \times 10^{-4}$). From BBN the same solve is driven by make_coupled_bbn(transport="qke", qke_kwargs={...}).

[**Port note**] **Status.** The mixing $\Delta N_{\rm eff}$ is the validated headline result. Through make_coupled_bbn(transport="qke") at the NLO default and key_recommended, the coupled run gives $N_{\rm eff} = 3.044067$ and $Y_{\rm P} = 0.247061$: the mixing raises $Y_{\rm P}$ by 5.4×10^{-5}, and the mixing-off run (qke_kwargs=\{"mixing": False\}) reproduces the classical transport="full" $Y_{\rm P}$ to 3×10^{-7}. This manual gives no convergence-pinned $Y_{\rm P}$/D/H table for the QKE path.

14 Extending the code

This section is the practical guide to changing what the code computes: tightening the $N_{\rm eff}$ precision, taking gradients, scanning the parameters of the existing model, and adding a collision channel, a background term or an option, with the invariants each change must preserve and the tests that cover it. It closes with the structure that is fixed throughout the code.

14.1 Increasing the precision of $N_{\rm eff}$

The default tolerances and resolutions are tuned, not maximal; for an arbiter-grade number you can tighten them, at a (usually modest) cost.

- **Tolerance.** Lower `rtol` from 10^{-6} to 10^{-8} (it costs only $\sim$20%; the result moves by $\sim 5 \times 10^{-7}$). Do not touch `atol`: it is already at the pure-relative floor 10^{-30}; raising it silently biases $N_{\rm eff}$ (Section 12).
- **Solver.** For a reference solve, use the plain adaptive path `hybrid=False` (and `use_sched=False`, the bit-faithful kernels) with `rtol=1e-8`; the implicit `Kvaerno5` path (`implicit=True`) is the L-stable cross-check. For the hybrid, increase `n_head` (more fixed exponential-midpoint steps), though its blind spot is the order-2 head, so the plain path is the cleaner arbiter. A fifth-order method is required, not optional: the discontinuous precision gate fools the lower-order embedded error estimators (a third-order solver mis-estimates $N_{\rm eff}$ by $\sim 9 \times 10^{-3}$), so the default `Tsit5` (order 5) is the floor.
- **Grid and quadrature resolution.** Raise `nbins` (must stay $4n{+}1$) and/or `eps_range` by passing `grid=make_eps_grid(...)` to the builder. The Gauss orders are not builder arguments: `n_gl`/`n_lag` are arguments of the kernel functions (and `n_gl` of `build_nunu_kernel` / `build_nunubar_kernel`), which `assemble._kernel_rate_groups` and the builders call at their defaults, and the Gauss–Laguerre node counts of `thermo` are module constants (100 for the free gas, 25 for the ϕ_e and QED terms), so raising them is an edit at those call sites. All of these are build-time and trade memory/compile for accuracy.
- **QKE node count.** For the QKE solve, the absolute $N_{\rm eff}$ converges only at order 2, so raise `n_steps` (default 6000) toward $\sim$18000 for an arbiter absolute number; the difference $\Delta N_{\rm eff}$ already converges by `n_steps=2000`.
- **Physics completeness.** Turn on the pieces that are implemented but off by default: the density-matrix collision on the coherences (`collision="matrix"` or `"matrix_full"`, $\delta N_{\rm eff} = -9.2 \times 10^{-5}$, Section 13.4), the thermal-MSW matter potential (`include_matter=True`), and the charge-neutrality electron degeneracy (`phie="auto"`; its $N_{\rm eff}$ effect is $\ll 10^{-7}$). The only un-modelled pieces are the QKE ν–ν refractive terms ($|\delta N_{\rm eff}| \lesssim 3 \times 10^{-7}$, Section 13), BURST's ϕ_e EoS-derivative closure, and any QED beyond the ported NLO $\mathcal{O}(e^3)$ (the next term of the QED equation of state, $\mathcal{O}(e^4)$, is estimated at $\delta N_{\rm eff} \simeq 3.5 \times 10^{-6}$ by Bennett et al., arXiv:1911.04504, Eq. 4.35, and is not included).

14.2 Gradients and gradient-based inference

`neutt` is differentiable and `jax.grad` runs through the whole solve, but one caveat must be respected before trusting a gradient. The precision gate (Section 3.4) is a hard threshold, flat on either side, so its derivative is zero almost everywhere, and the reverse-mode adjoint silently drops the gate's contribution to $\mathrm{d}N_{\rm eff}/\mathrm{d}\theta$. The forward $N_{\rm eff}$ is unaffected (and bit-faithful), but a gradient taken through the full coupled solve is not validated against finite differences and can be biased. **Before any gradient-based inference (HMC / NUTS, a Fisher forecast), check `jax.grad` against a finite-difference $N_{\rm eff}(\theta)$ for the parameters you will differentiate.** The smooth gate `gate_width`> 0 replaces the hard threshold with a sigmoid. It is an argument of `assemble_rhs` and reaches a solve only through `make_coupled_bbn(gate_width=...)` on the semi path; the decoupling, hybrid and QKE builders (and the affine `assemble_split_rhs`) use the hard gate. Keep the forward pass at `gate_width=0` for the bit-faithful number.

14.3 Scanning the parameters of an existing model

The hot path is built to be swept with no recompile. Build the solver once and vary the traced inputs per call:

- **SM / cosmology:** `delta_neff_init`, `eta_fac`, `tau_n_fac`, `nuclear_rates_q`, the electron mass `me`, and the transport start `T_init` are all traced (Section 10), and an

MCMC varies them with zero recompile. For a batch, `vmap` the built `solve` over a parameter array.

- **Lepton asymmetry:** the neutrino degeneracy `xi` is traced per call: `run(xi=...)` (coupled) or `solve(xi=...)` (standalone) recomputes the initial spectrum and the noise floor inside the compiled solve, so a ξ-scan reuses one build with no recompile (Section 10).

Parallelism: batch, do not shard. A single coupled solve is tiny (the state is a few kilobytes) and launch/bandwidth-bound, so sharding one solve across several GPUs buys nothing ($\lesssim 1.2\times$, often $< 1\times$). The useful parallelism is across independent solves: `vmap` a batch of cosmologies onto one GPU (the per-cosmology cost drops sharply once the launch overhead is amortized), or map an MCMC's walkers onto separate GPUs, one whole solve each.

14.4 Adding a channel, a background term, or an option

- **A classical collision channel.** A kernel is a function returning `(net, frs)`, each of shape $(3, 2, N)$ per $T_{\rm cm}^5$ (the caller multiplies by $T_{\rm cm}^5$). It enters through `assemble._kernel_rate_groups`, whose per-kernel tuple `_all_kernel_rates` sums into the gate numerator and denominator (and into `leg4` if the channel exchanges energy with the plasma), and which `compute_sumrules` and `decoherence_rate_split` unpack. Everything downstream reads those sums: `assemble_rhs`, the affine `assemble_split_rhs`, the noise floor `compute_prec_trans`, the QKE damping `decoherence_rate`, and through `make_rhs` / `make_split_rates` every solver.
- **A background energy density.** H and $\mathrm{d}T_{\rm pl}/\mathrm{d}\ln a$ are formed in `decoupling._background` (the inert $\rho_{\rm dr}$ is the template), which both `make_rhs` and `make_split_rates` call, so a term added there reaches the plain, hybrid and QKE solves alike. A plasma-EoS correction enters `decoupling._plasma_eos` as a $(\delta\rho, \delta P, \delta C_{\rm pl})$ triple, the pattern of `thermo.qed_plasma_deltas`.
- **A density-matrix (QKE) channel.** The matrix kernels in `transport/qke/` reuse their classical module's tiles, nodes and measure and change only the coupling and the blocking: $(1-f)$, f become $(I-\rho)$, ρ inside anticommutators (e.g. `collide_matrix.assemble_from_phi`). A new channel follows that pattern and is wired into `full_coupled`'s `collision` option (Section 13.4).
- **An option.** A switch that changes the shape of the computation is a builder argument (build-time); a number to be scanned is a `solve` argument passed to the RHS as data, never closed over as a Python constant (Section 10).

Invariants to preserve.

- **Detailed balance and the affine form.** `net` must vanish at the equilibrium spectrum (the noise floor `prec_trans` is that residual, recomputed from the same kernels), and every forward rate must carry the Pauli factor $(1-f_1)$ and every reverse rate f_1, so that the affine split (Eq. (9)) is exact.
- **No data-dependent control flow.** Kinematic branches become fixed tiles with runtime regime masks (Section 3.1), and masked square roots and denominators carry double-`where` guards so the kernel stays `grad`-safe (Section 3.7).
- **Faithful defaults.** A port-only change is default-off, a runtime identity, or reversible with one flag (Appendix A), so the Fortran-parity oracle tests (`lag_tail_fix=False`, `qed_nlo=False`, `SIN2THW_BURST`) and the canonical pins are unchanged.
- **The mixing-off reduction.** A QKE change keeps `mixing=False` bit-identical to the classical solve, and a matrix kernel reduces to its classical kernel on a diagonal ρ and sources no coherence from it.

Tests to add. A new kernel gets an oracle diff (a Fortran golden in `tests/oracle_*/` with its `test_<kernel>.py`) and the equilibrium sum rule (net $\to 0$ at $f^{\rm eq}$; Section 6.6); a scheduled form gets a parity test against its reference (`test_sched_parity.py`); a matrix kernel gets the diagonal-reduction test (the pattern of `tests/test_qke_collide*.py`). An $N_{\rm eff}$-affecting change is checked against the canonical pins with the `rtol=1e-8` arbiter (Section 11).

14.5 Fixed structure

- **Three flavours × two cp.** The flavour axis is three flavours by two cp states throughout the kernels, the noise floor and the QKE 3×3 density matrix; a sterile species ($n_\nu > 3$) means generalizing that axis in every kernel.
- **Abundances and recombination live outside.** The nuclear network is LINX's; there is no recombination / sound-horizon stage. A new observable on the BBN side is a LINX change, not a transport one.
- **The transport window.** Transport runs over comoving $T \in [0.015, 10]$ MeV (the defaults of `tcm_cutoff` and `T_init`); above it the spectrum is the equilibrium initial condition, below it the rates are masked and the spectrum is frozen, so a model with relevant dynamics outside that window needs the window and the equilibrium initial condition changed.

A BURST ↔ `neutt` mapping index

This appendix collects the complete correspondence between `neutt` and Fortran BURST in one place, so the two trees can be cross-read. The inline [**Fortran ↔ JAX**] and [**Port note**] boxes give the narrative; the tables here give the full enumeration. The **status** column uses:

status	meaning
SAME	bit-faithful transcription (value-identical to the snapshot oracle)
DIFF	same physics, deliberately different numerical route (noted why)
NONE	BURST feature not ported
ADD	port-only addition with no BURST counterpart

A note on the two Fortran trees: most rows cite the snapshot in `fortran_reference/`, against which the port is validated (the constants and the equilibrium occupations `fd_equil_calc`, `be_equil_calc` are in its `mainvar.f90`; `renorm.f90` is the same file, with the same line numbers, in both trees). Rows that name `part_lib.f90` (`part_make_bins`, `part_bin_var`, `part_var`), `bbn_therm.f90` or `get_early_derivs`, or say "refactored tree", cite the refactored BURST tree on the repository's `main` branch; the snapshot counterparts are the grid loop of `trans_init.f90` (lines 168–178, without the offset options), `bbn_yes_therm_v2.f90`, and `bbn_yes_evolve_v2.f90`.

A.1 Auxiliaries and grid

neutt	Fortran	status	note
`constants.*`	`mainvar.f90`, `transvar.f90`	SAME	values verified exact (`ZETA3`: full double precision; Section 5.1)
`grid.make_eps_grid`	`part_lib.f90::part_make_bins`	SAME	linear ε grid + Boole weights

neutt	Fortran	status	note
grid.EpsGrid	part_bin_var / part_var	SAME	data model
grid.fermi_dirac	fd_equil_calc	SAME	
grid.bose_einstein	be_equil_calc	SAME	guarded
grid.log_occupation	interp_occfrac clamp	SAME	$[-300, 0]$ clamp
grid.legendre_canonical	gauss.f90 (Legendre)	DIFF	host NumPy, not imtqlx
grid.gauss_laguerre*	gauss.f90 / cgqf	DIFF	host SciPy, not imtqlx
grid.rescale_legendre	gauss.f90::scqf	SAME	the affine rescale (ported; the kernels apply it inline)
grid moments (integrate, moment, number_density, energy_density)	inline moment integrals	SAME	Boole-weighted sums
interp.lagrange_window/apply_log_interp	interp_occfrac, polint, indpolint	SAME	static-query path
interp.interp_log_occ_dynamic	interp_occfrac (3 branches)	SAME/DIFF	snapshot clamp; fd_cap_logC/fd_decay/monotone_read are ADD

A.2 Background and finite-T QED

neutt	Fortran	status	note
thermo.rho_gamma/pressure_gamma	bbn_therm.f90	SAME	
thermo.rho_electron/pressure_electron	glag4/glag7 leading terms	DIFF	100-node default vs. BURST's 25
thermo.rho_plasma/pressure_plasma	bbn_therm.f90	SAME	bare mass; QED added separately
thermo.drho_plasma_dT	analytic $f(1-f)$	SAME	cross-checked vs. autodiff
thermo.get_eg_mass_renorm	renorm.f90:29--114	SAME	$\delta m_e^2, \delta m_\gamma^2$
thermo.get_e_mass_pdep	renorm.f90:119--282	SAME	momentum-dependent self-energy
thermo.m_e_eff	renorm.f90 (kernel mass set bare at trans_evolve.f90:97)	SAME	thermal $m_{\rm eff}$ exposed, but kernels use the bare mass
thermo.qed_eos_corrections	glag1--7 (momentum-indep.)	SAME	additive $\delta\rho/\delta P$
thermo.qed_eos_pdep_corrections	glag4/5/7 pdep (2nd loop)	SAME	
thermo.qed_plasma_deltas	bbn_therm.f90 EoS block	SAME	one-call form for the RHS
thermo.qed_eos_e3_pressure/qed_eos_e3_deltas	— (LINX explicit_P3)	ADD	NLO $\mathcal{O}(e^3)$ pressure and its $\delta\rho$, $\delta C_{\rm pl}$; default on (qed_nlo)
(glag6, $\mathrm{d}\rho_e/\mathrm{d}\phi_e$ pdep)	bbn_therm.f90	NONE	EoS-derivative term unported; ϕ_e itself is carried (settable, default 0)
thermo.hubble	bbn_yes_evolve_v2.f90:156 (hubcst)	SAME	MeV^4 units (no mev4tocgs)

neutt	Fortran	status	note
thermo.evolve_background	get_early_derivs path	DIFF	$\ln a$ ODE (diffrax) vs. time RK

A.3 Transport kernels and assembly

neutt	Fortran (snapshot)	status	note
transport.nunu	trans_nunu.f90	SAME	grid-only static tensor
transport.nunubar	trans_nunubar.f90	SAME	scatter + annih channels
transport.nuer1	trans_nuer1.f90	SAME/DIFF	background-dependent (union+mask); default lag_tail_fix=True switches the wide-regime tail to Gauss–Laguerre ($N_{\rm eff}$ shift $\leq 8.4 \times 10^{-7}$; SAME at lag_tail_fix=False)
transport.nuer2	trans_nuer2.f90	SAME/DIFF	background-dependent; default lag_tail_fix=True replaces the wide R1 segment at the lowest bins by a Gauss–Laguerre rule ($N_{\rm eff}$ shift $< 10^{-9}$; SAME at lag_tail_fix=False)
transport.nuepma	trans_nuepma.f90	SAME/DIFF	background-dependent; default lag_tail_fix=True lowers R2's genout Legendre→Laguerre switch to lag_tail_gap, so the wide segment just above pcase1 takes the Gauss–Laguerre tail ($N_{\rm eff}$ shift with nuer1 $\leq 8.4 \times 10^{-7}$; SAME at lag_tail_fix=False)
transport.nuer1/nuer2/nuepma *_rates_sched	—	ADD	launch-collapsed XLA rewrite: select=False ("union") reproduces the reference to $\leq 10^{-15}$, select=True to $\sim 6.7 \times 10^{-7}$ in $N_{\rm eff}$
assemble.assemble_rhs	trans_evolve.f90:195--208	SAME	combined gate (snapshot, not the refactored tree)
assemble.assemble_split_rhs	—	ADD	$A - Bf$ affine form (for the hybrid head)

neutt	Fortran (snapshot)	status	note
`assemble.compute_prec_trans`	`trans_init.f90:268--286`	SAME	+ `optimization_barrier` (ADD, runtime identity)
`assemble.equilibrium_occ`	`trans_init.f90:185--196`	SAME	
`assemble.compute_sumrules`	BURST sumrule1–5	SAME	diagnostic
`assemble` `gate_width`, `fd_cap_logC`	—	ADD	robustness knobs, default-off (no-ops)
`assemble` `OCC_FLOOR`	—	ADD	always-on positivity clamp (identity on physical $f > 10^{-300}$)
(combined vs. per-channel gate)	`trans_evolve.f90` (refactored tree)	DIFF	port follows snapshot's combined gate

A.4 Solver, driver, and coupling

neutt	Fortran	status	note
`decoupling.make_decoupling_solver`	`bbn_yes` driver + `trans_evolve`	DIFF	$\ln a$ / diffrax; $\{T_{\rm pl}, f\}$ state; built once, jitted `solve`
`decoupling.make_rhs`	`bbn_yes_evolve_v2.f90` + `trans_evolve.f90`	SAME	temperature ODE + gated collision RHS
`decoupling.make_split_rates`	—	ADD	affine RHS for the hybrid head
`decoupling.nanrobust_rms_norm`	—	ADD	diffrax NaN-freeze guard
`hybrid.make_hybrid_solver`	—	ADD	exp-midpoint head + Tsit5 tail
solver: `Tsit5`/`Kvaerno5`	Cash–Karp RK5(4)	DIFF	adaptive diffrax solvers
`transport_adapter.*` (LINX)	—	ADD	transport→LINX seam (the whole module)
`linx.coupled_bbn.make_coupled_bbn`	BURST internal BBN + `sdratio`	DIFF/NONE	abundances via LINX; no recombination
nuclear network	`nuc_*`, NSE, solver	NONE	LINX provides abundances
recombination / sound horizon	`sdratio*`	NONE	not ported
sterile ν ($n_\nu > 3$)	slots $4, \ldots, n_\nu$	NONE	SM transport is 3 flavours
MPI / `#ifdef prllel`	`mpi_allreduce`	NONE	replaced by on-GPU contraction
$\phi_e(t)$ co-evolution	`phie` ODE	DIFF	ported as the algebraic charge-neutrality $\phi(T_{\rm pl})$ (`thermo.solve_phie` via `phie="auto"`, default 0); the glag6 EoS-derivative closure is unported

A.5 Flavour mixing (QKE)

The QKE sector is new code with `qburst` as its lineage, not a transcription: the end-to-end solve of `qburst` is not a trustworthy oracle (Section 13), so the sector is validated against unit oracles, conservation invariants, and the mixing-off reduction rather than the snapshot.

neutt	origin / oracle	status	note
`transport.qke.coherent`	qburst `trans_init` / `get_coherent`	ADD	PMNS, m^2_{weak}, H and $-i[H,\rho]$; `gfortran` unit oracles (`oracle_qke_mixing`, `oracle_qke_coherent`)
`transport.qke.nonadiabatic`	qburst corotating step (`get_off_diag_amps` oracle)	ADD	coherent-step propagators (`expm`, Strang, `hybrid`) and the moving eigenframe
`transport.qke.collide`, `collide_nue`, `collide_nunu`	classical `nuepma`; `nuer1`/`nuer2`; `nunu`/`nunubar`	SAME/ADD	matrix generalizations (shared tensors in `collide_matrix`); on a diagonal ρ each reduces to its classical kernel and sources no coherence; the off-diagonal is new
`assemble.decoherence_rate`	—	ADD	ungated $g+l$, the scalar coherence damping Γ_a
`transport.qke.full_coupled`, `solve`	qburst (diverges e2e)	ADD	the density-matrix solver, standalone Strang driver; `mixing=False` reduces bit-identically to the classical solve

A.6 Summary of port-only additions

Things `neutt` has that BURST does not (the BURST-faithful gold gates are preserved because each is default-off, a runtime identity, or reversible with a single flag):

- the NLO $\mathcal{O}(e^3)$ finite-T QED correction (Section 5.4.4), the one default-on new-physics addition: it moves $N_{\text{eff}}^{\text{SM}}$ from the LO 3.0443 to 3.0433, and `qed_nlo=False` recovers the LO value (BURST's own 3.0442 also needs `SIN2THW_BURST`);
- the `lag_tail_fix` wide-regime lepton-kernel tail quadrature correction (Section 6.4), default on, but numerical: it fixes a latent Fortran-BURST quadrature bug in `nuer1`, `nuer2` and `nuepma` (N_{eff} shift $\leq 8.4\times10^{-7}$), and `lag_tail_fix=False` recovers the bit-faithful BURST form;
- the kernel-tensor `einsum` reformulation and the union-grid+mask scheme (Section 3.1);
- the scheduled (`_sched`) launch-collapsed kernels and `n_chunks` partner-bin chunking (performance only; the `"union"` schedule and `n_chunks=1` are bit-faithful, the `select` schedule agrees to $\sim 6.7\times10^{-7}$ in N_{eff});
- the affine split $A - Bf$ and the exponential-midpoint hybrid solver (Sections 7.3, 8);
- the `optimization_barrier` gate guard, `nanrobust_rms_norm`, and the `sqrt`/FD/denominator `where`-guards (correctness/robustness);
- the opt-in `gate_width` smooth gate and `fd_cap_logC` FD-envelope cap (default no-ops), and the always-on `OCC_FLOOR` positivity clamp (an identity on physical occupations);
- full `jit`/`grad`/`vmap` differentiability and GPU batching;
- the build-once solver factories (`make_*_solver` returning a jitted `solve`) and the compile banner (Section 10);

- the QKE flavour-mixing sector and its off-grid tail closures `fd_decay` and `monotone_read` (Section 13), which leave the classical solve untouched (`mixing=False` is bit-identical to it);
- the entire LINX coupling layer (Section 9).

B Glossary of symbols

Symbols are grouped by topic; code names are in typewriter. Energies are in MeV, energy densities and pressures in MeV^4, and rates in s^{-1} unless noted. Where one letter carries two meanings in different sections, both are listed.

symbol	meaning
Temperatures, clocks, and the grid	
$T_{\rm cm}$ (`T_cm`, `tcmev`)	comoving temperature parameter (redshift clock, $T_{\rm cm} = T_{\rm cm}^{\rm init} e^{-\ln a} \propto 1/a$); not a physical T_ν
$T_{\rm pl} = T_\gamma$ (`T_pl`, `tplmev`)	physical plasma / photon temperature
T_ν	physical neutrino temperature (thermal-fluid approximation only); $f^{\rm eq}$ is Fermi–Dirac at $T_\nu = T_{\rm cm}$
`tcmpl`, `ratio`	$T_{\rm cm}/T_{\rm pl}$ (a lepton-kernel argument), and its inverse $T_{\rm pl}/T_{\rm cm}$ returned by the solvers
$\ln a$, s	integration variable (e-folds since the start); $T_{\rm cm} = T_{\rm cm}^{\rm init} e^{-\ln a}$. Written $s \equiv \ln a$ in Sections 7.1 and 13.3 and in the affine form $\mathrm{d}f/\mathrm{d}s = A - Bf$; the QKE sources s and $s_{\rm coll}$ are below
`T_init`, `lna_max`, $T_{\rm end}$	transport start temperature ($T_{\rm pl} = T_{\rm cm}$ there; default 10), the solve length in $\ln a$, and $T_{\rm end} =$ `T_init` $e^{-\texttt{lna_max}}$
`tcm_cutoff`	transport-window low edge in $T_{\rm cm}$ (BURST's 0.015); below it the rates are masked to zero and the spectrum is frozen
$\varepsilon = E_\nu/T_{\rm cm}$	comoving energy; the grid abscissa ($= p/T_{\rm cm}$ for the relativistic neutrinos); ε_i, ε_a a bin's value
N, $\varepsilon_{\max}$, w_i, h	bin count `nbins` ($4n$+1; 101), grid extent `eps_range` (25), composite-Boole weights, and spacing $h = \varepsilon_{\max}/(N-1)$ (Section 5.2); the h of Eq. (10) is instead a step in $\ln a$
p_1, E_2, E_3, p_4	the legs of a $1{+}2 \to 3{+}4$ process in $T_{\rm cm}$ units: $p_1 = \varepsilon_i$ the output bin, E_2, E_3 the integrated energies, p_4 the off-grid fourth leg ($p_1 + E_2 - E_3$ for scattering; $E_{\rm out} + E_{\rm in} - p_1$ in `nuepma`)
Occupations, densities, and observables	
$f_{\nu_i}(\varepsilon)$, $f[3,2,N]$	neutrino occupation number, indexed flavour ($0{:}e, 1{:}\mu, 2{:}\tau$) × cp × bin; equilibrium $f^{\rm eq} = 1/(e^\varepsilon + 1)$
cp, `parfact`	charge-parity index ($0{:}\nu$, $1{:}\bar\nu$) and its sign ± 1
δf, δ_a	fractional spectral distortion $(f - f^{\rm eq})/f^{\rm eq}$; δ_a that of flavour a (e.g. $\delta_e - \delta_\mu$)
$\xi = (\xi_e, \xi_\mu, \xi_\tau)$	per-flavour ν degeneracy (lepton asymmetry); the initial spectrum is $1/(e^{\varepsilon - \mathrm{parfact}\,\xi} + 1)$
ϕ_e (`phie`)	electron degeneracy $\mu_e/T_{\rm pl}$ (settable; default 0, or `"auto"` for the charge-neutrality $\phi(T_{\rm pl})$; only its derivative QED EoS term, BURST's `glag6`, is not ported)
$q_{\rm baryon}$ ($q_{\rm b}$), n_b, η (η_b, `eta0`)	charge per baryon balanced by the $e^\pm$, $n_{e^-} - n_{e^+} = q_{\rm baryon} n_b$; baryon number density; baryon-to-photon ratio
ρ_ν, ρ_γ	neutrino (all flavours and cp) / photon energy density [MeV^4]
$\rho_{\rm pl}, P_{\rm pl}$	plasma ($\gamma + e^\pm$) energy density and pressure: the free-gas $\rho_\gamma + \rho_{e^\pm}$ plus the finite-T QED $\delta\rho_{\rm QED}$, δP
$C_{\rm pl}$	plasma heat capacity $\mathrm{d}\rho_{\rm pl}/\mathrm{d}T$ [MeV^3] (the variable `C` of `decoupling._plasma_eos`; distinct from the collision integral C_{ν_i} and the FD-cap multiplier C)

symbol	meaning
$\rho_{\rm DR} \equiv \rho_{\rm dr} \equiv \rho_{\rm ext}$	extra-radiation energy density, $\rho_{\rm dr}$ = `delta_neff_init` · $(\frac{4}{11})^{4/3}\frac{7}{8}\frac{\pi^2}{15}T_{\rm cm}^4$; standalone and coupled share LINX's `delta_neff_init` normalization (Section 9.4); BURST's `rhors` is the same quantity without the $(4/11)^{4/3}$
r_ν	ρ_ν/ρ_γ summed over flavours and cp, the transport→LINX transfer variable (`r_nu_of_Tg`); constant above the transport start, frozen after $e^\pm$ annihilation; the late-time $N_{\rm eff} = \frac{8}{7}(\frac{11}{4})^{4/3} r_\nu$
$N_{\rm eff}$	effective number of neutrino species (Eq. (1)); SM value 3.0433 (NLO QED, default) / 3.0443 (LO)
$\Delta N_{\rm eff}$	has three distinct uses, all read from context: (i) the input inert-dark-radiation knob `delta_neff_init` (LINX `Delt_Neff_init` convention), the extra radiation at the transport start; (ii) a physics-induced output excess $N_{\rm eff} - N_{\rm eff}^{\rm SM}$ (e.g. the genuine QKE $+7.6\times10^{-4}$, a dynamically generated shift, not an input); (iii) a solver-vs-arbiter $N_{\rm eff}$ residual
$Y_{\rm P}$, D/H, Y_i	^{4}He mass fraction $Y_{\rm P} = 4\,Y(^4{\rm He})$; deuterium ratio D/H = $Y({\rm D})/Y(p)$; $Y_i = n_i/n_b$ the LINX number abundances
`eta_fac`, `tau_n_fac`, `nuclear_rates_q`	baryon-to-photon and neutron-lifetime rescales (1 = fiducial) and per-reaction draws $q \sim \mathcal{N}(0,1)$
Constants, background, and finite-T QED	
$G_{\rm F}$, $\sin^2\theta_W$, α, m_e	Fermi constant, weak mixing angle (`SIN2THW`; BURST's 0.23 is `SIN2THW_BURST`), fine-structure constant, bare electron mass `M_ELECTRON` (Section 5.1)
$M_{\rm Pl}$, $\hbar$, m_W, m_Z	Planck mass [MeV], $\hbar$ [MeV·s], and the W/Z masses of the QKE potentials
H	Hubble rate [s^{-1}], $H = \hbar^{-1}\sqrt{8\pi(\rho_{\rm pl}+\rho_\nu+\rho_{\rm dr})/3M_{\rm Pl}^2}$; in Section 13 H is the flavour Hamiltonian (below)
Γ	weak interaction rate, $\Gamma \sim G_{\rm F}^2 T^5$, and the collision loss rate in Γ/H; the QKE rates Γ_a, Γ_{ab} are below
δm_e^2, δm_γ^2, $m_{\rm eff}$	thermal self-energies [MeV^2] and $m_{\rm eff} = \sqrt{m_e^2 + \delta m_e^2}$ (`thermo.m_e_eff`; not fed to the kernels)
`glag1`–`glag7`, pdep	BURST's seven Gauss–Laguerre plasma-moment integrals (ρ, P, ${\rm d}\rho/{\rm d}T$ and their QED corrections; `glag6`= ${\rm d}\rho_e/{\rm d}\phi_e$), and their momentum-dependent second-loop part
δP_{e^3}, J	the NLO $\mathcal{O}(e^3)$ interaction pressure and its Gauss–Laguerre sum (Section 5.4.4)
Collision rates and the precision gate	
C_{ν_i}, C_i, $C[\rho]$	collision integral (rate) of mode i; in the code $C_i = T_{\rm cm}^5\,{\rm net}_i\,\Theta_{\rm gate}$; $C[\rho]$ its density-matrix form. Not the heat capacity $C_{\rm pl}$ or the FD-cap multiplier C
F (F_r), `flafact`	Pauli-blocked gain−loss factor = forward−reverse (Eq. (4)), and the per-process flavour/spin weight multiplying it
S_r, $\langle\lvert M_r\rvert^2\rangle$	symmetry factor and spin-summed squared matrix element of process r
W, K, M, $n_{\rm GL}$	static weight tensor (N,N,K) of the grid-only kernels, its quadrature slots per bin pair (K = 15 `nunu`, 10 `nunubar`), the interpolation window (M = 6), and Gauss nodes per segment (5); K is also the QKE frame generator
${\rm idx}_{3,4}$, ${\rm lw}_{3,4}$, $L[p,a]$	window indices and Lagrange weights of the off-grid reads of legs 3 and 4, $\log f(p) = \sum_a L[p,a]\log f_a$

symbol	meaning
`net`, `frs`	per-bin collision decomposition. `net`=forward−reverse rate is the physical signal that drives $\mathrm{d}f/\mathrm{d}\ln a$; `frs`=forward+reverse is the total throughput, which normalizes the cancellation in the gate and its noise floor (with `net` it also gives the forward and reverse rates behind the affine A, B and the QKE Γ_a). Near equilibrium the two nearly cancel, so $\lvert\texttt{net}\rvert \ll \texttt{frs}$
`numer`, `denom`	$\sum$`net` and $\sum$`frs` over the five kernels, the gate's inputs
`prec`, `prec_trans`	equilibrium noise floor for the precision gate, $\lvert\sum \mathrm{net}\rvert/\sum \mathrm{frs}$ per (flavour, cp, bin) at the equilibrium spectrum (`PREC_FLOOR` where $\sum \mathrm{frs} \le 0$)
`test_ratio`, `nuratiotol`, Θ_{gate}	$\lvert\sum \mathrm{net}\rvert/(\sum \mathrm{frs}\cdot \mathrm{prec})$, its threshold 3, and the resulting 0/1 gate (Eq. (8))
`doccfracdt`, `drhonudt`	the gated $\mathrm{d}f/\mathrm{d}t$ [s^{-1}] and the ν–plasma energy-exchange rate $\mathrm{d}\rho_\nu/\mathrm{d}t\lvert_{\mathrm{exch}}$ [$\mathrm{MeV}^4\,\mathrm{s}^{-1}$]
`leg4`	the summed net rate of the three ν–$e^\pm$ channels, BURST's partner-species slot 4 (the $e^\pm$), whose ε^3 moment is the energy exchanged with the plasma; not the momentum leg p_4 ("leg 4" below)
A, B (g, l)	affine collision RHS coefficients: per $\ln a$ (`make_split_rates`, the decoupling/hybrid/QKE builders) $\mathrm{d}f/\mathrm{d}\ln a = A - Bf$ with $A = g/H$, $B = (g+l)/H$, g, l the gain/loss rates [s^{-1}]; `assemble_split_rhs` returns the s^{-1} form $\mathrm{d}f/\mathrm{d}t = A - Bf$, $A = g$, $B = g + l$ (carrying T_{cm}^5). Eq. (9) writes the per-$\ln a$ form as $A = g$, $B = g + l$ with g, l already divided by H
`OCC_FLOOR`	always-on clamp of f to $[10^{-300}, 1]$ before every kernel call (an identity on physical occupations)
Flavour mixing (QKE)	
ρ, $\bar\rho$	3×3 Hermitian flavour density matrix per (cp, bin), $\rho[\mathrm{cp}, i, m, m] \equiv f[m, \mathrm{cp}, i]$, its off-diagonal entries the flavour coherences (ρ_{diag}, ρ_{off}); $\bar\rho$ the $\bar\nu$ matrix. Not an energy density
H (H_{flav})	flavour Hamiltonian [s^{-1}], $H_{\mathrm{vac}} + H_{\mathrm{mat}}^{(1)} + H_{\mathrm{mat}}^{(2)} + H_{\mathrm{si}}^{(1)} + H_{\mathrm{si}}^{(2)}$ (vacuum, asymmetric and thermal MSW, ν–ν; the solvers pass the two ν–ν terms as zero)
m^2_{weak}, U, M^2	$m^2_{\mathrm{weak}} = UM^2U^\top$ (trace removed; $-m^2_{\mathrm{weak}}$ for $\bar\nu$), the real PMNS matrix $U = R_{23}R_{13}R_{12}$ (angles θ_{ij}), and $M^2 = \mathrm{diag}(0, \Delta m^2_{21}, \Delta m^2_{31})$ [MeV^2]. In Section 13.3 U also names the Strang step's unitary $Ve^{-i\Lambda \mathrm{d}t}V^\dagger$
n_p	proton (net-e^-) density of $H_{\mathrm{mat}}^{(1)}$, $n_p = q_b n_b \propto \eta_b$, scaled per call by `matter_asym_scale`
L_{coh}, Λ_{off}	the 9×9 coherent Liouvillian $-i[H, \cdot] - \Lambda_{\mathrm{off}}$ and its off-diagonal-only dephasing
Γ_a, Γ_{ab}	ungated collision throughput $g + l$ of flavour a (`assemble.decoherence_rate`) and the coherence damping rate $\frac{1}{2}(\Gamma_a + \Gamma_b)$
ω, Δ	largest eigenvalue splitting of H ($\omega\,\mathrm{d}t$ is the phase per step) and the vacuum oscillation frequency $\delta m^2/2p$
V, Λ	eigenvectors and eigenvalues of H in the Strang unitary $Ve^{-i\Lambda \mathrm{d}t}V^\dagger$ (not Λ_{off}, nor the PMNS U)
K, $\tilde R$	moving-eigenframe generator, $e^{-iK\mathrm{d}t} = V_{\mathrm{end}}V_{\mathrm{start}}^\dagger$, and the half-step rotation $\tilde R = e^{-iK\mathrm{d}t/2}$
Θ	non-adiabaticity measure $\lVert[H_{\mathrm{vac}}, \rho]\rVert^2/(\lVert H_{\mathrm{vac}}\rVert^2\lVert\rho\rVert^2)$ (Frobenius norms); not Θ_{gate}
L_{oo}, L_{od}, s	the (off-diagonal, off-diagonal) and (off-diagonal, diagonal) Liouvillian blocks and the coherent source $s = L_{od}\rho_{\mathrm{diag}}$ (`offdiag_blocks`), so the quasi-steady coherence is $-L_{oo}^{-1}s$. The sourced step's collision rate is $s_{\mathrm{coll}} = (f_{\mathrm{coll}} - f)/\mathrm{d}t$ (`src`)

symbol	meaning
$A_{\rm gain}$, $B_{\rm loss}$	Hermitian gain and loss matrices of the density-matrix collision $\mathrm{d}\rho/\mathrm{d}t\|_{\rm coll} = A_{\rm gain} - \{B_{\rm loss}, \rho\}$; on a diagonal ρ it reduces to the classical kernel and $2\,\mathrm{diag}(B_{\rm loss})$ is the ungated throughput Γ_a
$n_{\rm steps}$ (`n_steps`)	operator-split macro-steps of the QKE solve (default 6000)
`num_drift`, `min_f`	per-bin number drift and minimum occupation, the QKE conservation diagnostics
Terms	
pin	a frozen reference value the port must reproduce on demand (e.g. the LO oracle pin $N_{\rm eff} = 3.044225186$ at `SIN2THW_BURST`; the default NLO pin is 3.0433222)
oracle	the trusted source of truth a result is checked against (here, the compiled Fortran snapshot)
cold / warm	a compile-paying first call vs. a graph-reusing later call (Section 1.6)
traced / static	a per-call (no-recompile) vs. a build-time (recompile-on-change) parameter (Section 1.6)
precision gate	per-bin test that zeroes a bin's `net` rate when |`net`| falls below the equilibrium round-off floor `prec` (where forward and reverse nearly cancel); prevents the solver from integrating floating-point noise as a physical spectral distortion
gate (verb), "gated against"	to validate/anchor a computed result against a trusted reference ("gated against the oracle" = regression-checked for agreement). Distinct from the precision gate, which instead masks sub-floor bin rates
conservation sum rules	moment integrals of the net rate (number $\int \varepsilon^2$ `net`, energy $\int \varepsilon^3$ `net`, and CP-symmetry checks) that must vanish in equilibrium; a free, continuous correctness monitor
affine $A - Bf$ form	writing a stiff bin's RHS as $\mathrm{d}f/\mathrm{d}s = A - Bf$, which an exponential-midpoint step advances in closed form (exactly); the fixed point $f^\star = A/B$ is the bin's local quasi-static equilibrium (its QSS attractor). See A, B above
QSS attractor	quasi-static equilibrium: the fixed point $f^\star = A/B$ of the affine RHS onto which a fast, collision-dominated bin is driven. Tracking it lets the stiff head take large steps without losing accuracy
head / tail, qualified	the hybrid's fixed-step exponential-midpoint stretch $[0, $ `lna_sw`$)$ and adaptive Tsit5 remainder; `qualified` reports that the handoff passed its QSS check (`qss_dist`$\leq$`qual_tol`) rather than falling back to the full adaptive solve
full / semi / qke	the coupled-BBN transport modes (Section 9.3): full co-evolution of $T_{\rm pl}$ and the spectrum, a spectrum-only solve on LINX's plasma history, and the genuine QKE solve
mixing-off reduction	the QKE solve with `mixing=False`, bit-identical to the classical transport; the anchor of the QKE validation
`tail_closure` (`"srcfix"`)	the default off-grid tail closure of the QKE solver, parameter-free: `fd_decay` carries the live grid-edge $f/f^{\rm eq}$ outward along the FD shape, `monotone_read` clamps the on-grid interpolant to its node bracket. The opt-in alternative is the FD-envelope `"cap"` below
FD-envelope cap (`fd_cap_logC`, `fd_cap_C`)	caps an off-grid source occupation at a multiple C of Fermi–Dirac, $f(p) \leq C\, f^{\rm eq}(p)$. The kernels' `fd_cap_logC`$= \log C$ defaults to $+\infty$, a bitwise no-op; under the opt-in QKE closure `tail_closure="cap"`, $C =$`fd_cap_C`, default `interp.FD_ENVELOPE_C`$= 10$
golden	a frozen reference test (input + expected output) a result is regression-checked against, as opposed to a single pin value
arbiter	a separate, ultra-tight-tolerance reference run treated as ground truth to measure another run's error
L-stable	property of a stiff integrator whose step amplitude stays bounded (in fact $\to 0$) even for arbitrarily fast-decaying modes, so a too-large step cannot blow up the stiff bins

symbol	meaning
`optimization_barrier`	a JAX/XLA directive marking a value opaque so the compiler may not pre-evaluate it at build time with a different floating-point summation order; needed where a near-perfect cancellation makes the answer summation-order sensitive
scaling-and-squaring (`max_squarings`)	the algorithm JAX uses for the matrix exponential e^M (scale down, exponentiate, square back up); a very fast rotation needs enough squarings or the result silently underflows to `NaN`
order-p convergence	the discretization error of a stepper falls like $1/n_{\text{steps}}^p$; order 2 means doubling the step count cuts the error by $\sim 4\times$
dispatch-bound	wall time dominated by per-call GPU kernel-launch (dispatch) overhead rather than arithmetic; the reason batching many solves with `vmap` amortizes the cost
leg 4 (momentum)	the fourth momentum leg p_4 of a 1+2 $\to$ 3+4 kernel ($p_4 = p_1+E_2-E_3$ for scattering, $E_{\text{out}}+E_{\text{in}}-p_1$ in `nuepma`): it lands between grid points and $f(p_4)$ is read by off-grid interpolation. Not `leg4`, BURST's $e^{\pm}$ partner slot above
Lepton kernels	
`meps`	the lepton kernels' "mass/T_{cm}" argument, m_e/T_{cm} with the bare mass
regimes	a lepton-kernel bin's kinematic case: A/B in `nuer1` (split at $p_1 =$ `meps`$/2$), R1–R4 in `nuer2` and `nuepma` (split at the `pcase` momenta); not BURST's R1/R2 angular reductions that name `nuer1`/`nuer2`
`ecut1`–`ecut3`, `etrans1,2`, `elim1`	kinematic cut energies [T_{cm} units] bounding a lepton kernel's tiles, defined per kernel (e.g. `ecut3` $= \sqrt{p_1^2 + \texttt{meps}^2}$ in `nuer1`/`nuer2`; `nuer2`'s and `nuepma`'s `ecut1` $= p_1 + \texttt{meps}^2/(4p_1)$); `etrans1,2` are `nuepma`'s inner cuts per outer node, `elim1` `nuer2`'s inner E_2 limit
tiles, `genout`/`genin`	the fixed quadrature sub-intervals (Gauss–Legendre on $[a, b]$, Gauss–Laguerre on $[a, \infty)$); `nuepma`'s `genout`/`genin` segments switch from Legendre to Laguerre when wider than 200 (BURST), or than `lag_tail_gap` under `lag_tail_fix`
`n_gl`, `n_lag`	Gauss–Legendre / Gauss–Laguerre orders of the kernels (BURST's 5, 5)
`lag_tail_fix`, `lag_tail_gap`	the default-on wide-regime quadrature correction of `nuer1`, `nuer2` and `nuepma`, and the segment width ($6\,T_{\text{cm}}$) above which a segment takes the Gauss–Laguerre tail (Section 6.4)
`use_sched`, "sched"	performance-only switch selecting the lepton-kernel implementation (per-bin `select` gather, `union` mask, or `False` reference); all give identical physics, trading speed against bit-reproducibility, never accuracy
`pcase`	the $e^{\pm}$-mass-threshold momenta (`nuepma`: `meps`$/2$, `meps`$(1+\sqrt{5})/4$, `meps`; `nuer2`: `meps`$(\sqrt{5}-1)/4$, `meps`$/(2\sqrt{2})$, `meps`$/2$) that segment a lepton-kernel bin's energy integral into the kinematic regimes R1–R4

C References

- E. Grohs, G. M. Fuller, C. T. Kishimoto, M. W. Paris, A. Vlasenko, "Neutrino energy transport in weak decoupling and big bang nucleosynthesis," Phys. Rev. D **93**, 083522 (2016), arXiv:1512.02205. (The BURST paper.)
- E. Grohs, G. M. Fuller, "Big Bang Nucleosynthesis," Handbook of Nuclear Physics (2023), arXiv:2301.12299.
- E. Grohs, G. M. Fuller, C. T. Kishimoto, M. W. Paris, "Lepton asymmetry, neutrino

spectral distortions, and big bang nucleosynthesis," arXiv:1612.01986.

- C. Giovanetti, M. Lisanti, H. Liu, S. Mishra-Sharma, J. T. Ruderman, "LINX: A Fast, Differentiable, and Extensible Big Bang Nucleosynthesis Package," arXiv:2408.14538.
- C. Giovanetti, M. Lisanti, H. Liu, J. T. Ruderman, "Joint CMB and BBN Constraints on Light Dark Sectors with Dark Radiation," Phys. Rev. Lett. **129**, 021302 (2022), arXiv:2109.03246.
- K. Akita, M. Yamaguchi, "A precision calculation of relic neutrino decoupling," arXiv:2005.07047; J. Froustey, C. Pitrou, M.-C. Volpe, "Neutrino decoupling including flavour oscillations and primordial nucleosynthesis," arXiv:2008.01074; J. J. Bennett et al., "Towards a precision calculation of $N_{\rm eff}$ in the Standard Model II," arXiv:2012.02726.
- NLO QED: J. J. Bennett et al., "Towards a precision calculation of the effective number of neutrinos $N_{\rm eff}$ in the Standard Model," arXiv:1911.04504 (the $\mathcal{O}(e^3)$ interaction pressure ported here); G. Cielo, M. Escudero, G. Mangano, O. Pisanti, "$N_{\rm eff}$ in the Standard Model at NLO is 3.043," arXiv:2306.05460.

The list above is a curated key-reading list rather than an exhaustive bibliography; sources used only in one place are linked inline where they appear. This manual is intended to be self-contained: every numerical choice and caveat is explained where it is used, so no familiarity with the repository's internal development notes is assumed.